Major Project (IT898) Report on

AGENTIC AI APPLICATION FOR INTELLIGENT PORTFOLIO CREATION

Lucid: Intelligent Investment Portfolio Platform for Indian Retail Equities

Submitted in partial fulfillment of the requirements for the degree of

MASTER OF TECHNOLOGY
in
INFORMATION TECHNOLOGY

by

ANKUR PATEL
(252IT004)

under the guidance of

Dr. Biju R Mohan



DEPARTMENT OF INFORMATION TECHNOLOGY
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA, SURATHKAL
MANGALORE - 575025

November 2026

DECLARATION

by the P.G. M.Tech.(IT) Student

I hereby *declare* that the Report of the P.G. IT898 - Major Project Work entitled "**Agentic AI Application for Intelligent Portfolio Creation (Lucid)**", which is being submitted to National Institute of Technology Karnataka, Surathkal in partial fulfillment of the requirements for the award of the Degree of Master of Technology in the Department of Information Technology, is a *bonafide report of the work carried out by me.* The material contained in this report has not been submitted at any University or Institution for the award of any degree.

ANKUR PATEL (252IT004)
Dept. of Information Technology

Place: NITK, SURATHKAL
Date: November 2026

CERTIFICATE

This is to *certify* that the Report entitled "**Agentic AI Application for Intelligent Portfolio Creation (Lucid)**", submitted by **ANKUR PATEL (252IT004)** as the record of the work carried out by him, is *accepted as the Project Work report submission* in partial fulfillment of the requirement for the award of the degree of **Master of Technology in Information Technology** in the Department of Information Technology, NITK Surathkal, India.

Dr. Biju R Mohan
(Signature of the guide with Date and Seal)

Chairman - DPGC
(Signature with Date and Seal)

ACKNOWLEDGEMENT

I would sincerely like to express my profound gratitude to my project supervisor, **Dr. Biju R Mohan**, for his unconditional support, technical guidance, and constructive critique throughout the research, design, implementation, and evaluation phases of this Major Project.

I also express my gratitude to the faculty and staff of the Department of Information Technology, National Institute of Technology Karnataka, Surathkal, for providing the necessary computational infrastructure, academic resources, and encouragement that enabled the successful completion of this work.

ANKUR PATEL (252IT004)

ABSTRACT

Indian retail equity investors face severe systemic barriers to disciplined wealth creation: cognitive information overload, emotional behavioral biases leading to mistimed market exits, and the lack of transparent, personalized portfolio guidance. Commercial quantitative robo-advisors operate as opaque "black boxes" offering zero fiduciary justification, while modern conversational Large Language Models (LLMs), when deployed naively in finance, suffer from catastrophic arithmetic hallucinations, generating non-normalizing allocations, negative weights, and economically unviable fractional share quantities on Indian stock exchanges.

This project designs, engineers, and evaluates **Lucid**, an institutional-grade, transparent, agentic AI-driven portfolio construction, continuous monitoring, and rebalancing platform engineered specifically for Indian retail investors across the National Stock Exchange (NSE) and Bombay Stock Exchange (BSE). Lucid bridges the gap between qualitative financial reasoning and numerical rigor through a strict architectural decoupling: a **13-node LangGraph StateGraph multi-agent pipeline** (powered by Google Gemini 3.8 Flash with deterministic rule fallbacks) synthesizes macroeconomic monetary policies, market volatility regimes (India VIX), balance-sheet fundamental metrics (ROE, P/E, P/B, Debt/Equity), and corporate news.

Crucially, all numerical computations are strictly prohibited within the LLM; Markowitz mean-variance risk-adjusted weights are computed deterministically via **SciPy's Sequential Least Squares Programming (SLSQP)** solver subject to hard institutional diversification bounds ($\leq 25\%$ single stock, $\leq 40\%$ sector). To bridge the gap between theoretical weights and Indian exchange mechanics, Lucid executes a **discrete whole-share integer lot sizing algorithm** that computes legally executable share quantities while preserving cash balances.

Lucid features a continuous, unattended background drift radar running every 5 minutes during Indian market hours (09:15–15:30 IST) to detect asset and sector drift ($\geq 5\%$), triggering friction-aware rebalancing previews factoring in statutory Securities Transaction Tax (STT), exchange turnover fees, SEBI charges, stamp duty, and GST. The underlying database architecture comprises **62 domain application tables** (+1 alembic tracking table) in PostgreSQL 16 housing a synchronized exchange universe of **2,639 listed equities** with **504 liquid recommendation candidates**. Empirical backtesting over historical test scenarios demonstrates that Lucid's AI SLSQP strategy achieves an annualized return of

10.72% with an annualized volatility of **9.00%**, a Sharpe ratio of **0.41**, and an institutional diversification score of **95.51/100**, strictly outperforming the market benchmark while providing transparent, dual-level fiduciary explanations for every investment decision.

Keywords: Agentic AI, LangGraph, Markowitz Mean-Variance Optimization, SciPy SLSQP, Indian Equities (NSE/BSE), Integer Lot Sizing, Demat CAS Diagnosis, Drift Radar, Explainable AI, Argon2id, PostgreSQL 16.

CONTENTS

LIST OF FIGURES xii

LIST OF TABLES xiii

ABBREVIATIONS xv

1 Introduction 1

1.1 Background . . . 1
1.2 Problem Statement . . . 2
1.3 Challenges in Retail Portfolio Construction . . . 2
1.4 Motivation . . . 3
1.5 Project Objectives . . . 4
1.6 Scope of the Project . . . 5
1.7 Proposed Solution: Lucid . . . 5
1.8 Key Contributions and Differentiators . . . 6
1.9 Report Organization . . . 6

2 Literature Review and Competitive Analysis 9

2.1 Theoretical and Academic Foundations . . . 9
2.1.1 Modern Portfolio Theory and Mean-Variance Optimization . . . 9
2.1.2 Factor Investing and Behavioral Finance . . . 10
2.1.3 Agentic AI and LLMs in Financial Decision Support . . . 10
2.2 Competitive Landscape in Indian Retail Investing . . . 11
2.2.1 smallcase . . . 11
2.2.2 Univest . . . 11
2.2.3 Motilal Oswal — Intelligent Advisory Portfolios (IAP) . . . 12
2.2.4 Savart . . . 12
2.2.5 Wright Research . . . 12
2.2.6 Jarvis Invest . . . 13
2.2.7 MarketSenseAI (International Academic Benchmark) . . . 13
2.3 Comprehensive Comparative Matrix . . . 13
2.4 Outcome of the Competitive Review and Design Rationale . . . 14

3 High-Level Design (HLD) 15

3.1 HLD Overview . . . 15
3.2 Major System Components . . . 15

3.2.1 Client Presentation Subsystem 16
3.2.2 API Gateway and Security Subsystem 16
3.2.3 Business and Application Service Subsystem 17
3.2.4 Multi-Agent AI Orchestration Subsystem 17
3.2.5 Quantitative and Optimization Subsystem 17
3.2.6 Data Persistence and Schema Subsystem 17
3.2.7 External Market Data and Feeds Subsystem 18
3.3 Component Interaction and Data Flow 18
3.4 User-Level End-to-End Workflows 19
3.4.1 Workflow 1: User Onboarding and Risk Profiling 19
3.4.2 Workflow 2: Goal-Driven Portfolio Generation 19
3.4.3 Workflow 3: Fiduciary "Stock-Why" Explainability 19
3.4.4 Workflow 4: Demat Statement (CAS) Upload and Diagnosis 20
3.4.5 Workflow 5: Continuous 5-Minute Background Monitoring . 20
3.4.6 Workflow 6: Friction-Aware Rebalancing Order Preview . . 20
3.5 Key Architectural Design Decisions 20

4 Low-Level Design (LLD) 23
4.1 LLD Overview 23
4.2 Frontend Architecture and Component Hierarchy 23
4.2.1 Component Structure and Responsibilities 23
4.2.2 Type-Safe Client API Layer (`frontend/src/lib/api.ts`) . 24
4.3 Backend Architecture and API Subsystem 25
4.3.1 Dependency Injection and Session Management 26
4.4 Agent-Level Design: 13-Node LangGraph StateGraph 26
4.4.1 Agent Nodes and Execution Topology 27
4.5 Quantitative Optimization and Lot Sizing Engine 28
4.6 Monitoring Scheduler and Distributed Locking 28
4.7 Sequence Flow Diagrams 29

5 Database Design and Entity-Relationship Architecture 33
5.1 Database Architectural Overview 33
5.1.1 Database Schema Scale and Domain Breakdown 33
5.2 Master Entity-Relationship Architecture 34
5.3 Domain-Wise Schema Specifications 34
5.3.1 Domain 1: Identity, Authentication and Security Governance 34

5.3.2 Domain 2: Investor Suitability, Profiling and Financial Goals 36
5.3.3 Domain 3: Security Master and Corporate Registry 38
5.3.4 Domain 4: Market Data, Historical Quotes and Macro Regime 40
5.3.5 Domain 5: Portfolio Engine, Quantitative Optimization and Tracking 42
5.3.6 Domain 6: Broker Integration, Execution and Order Lifecycle 44
5.3.7 Domain 7: Demat CAS Diagnosis and Portfolio Analytics . . 46
5.3.8 Domain 8: Multi-Agent AI Telemetry, Reasoning Traces and Verification 48
5.4 Database Design Decisions and Performance Optimization 49

6 System Architecture and Runtime Topology 51
6.1 System Architecture Overview 51
6.2 The Eight Architectural Layers 52
6.3 Technology Stack Manifest 53
6.4 Data Pipeline and Ingestion Architecture 53
6.5 Runtime Execution Flow: Portfolio Generation Request 55
6.6 Security and Privacy Architecture 56

7 End-to-End Application Methodology 59
7.1 Methodology Overview 59
7.2 Investor Onboarding and Suitability Profiling 61
7.3 Natural Language Intent and Constraint Parsing 62
7.4 Market Regime and Volatility Classification 62
7.5 Stock Universe Discovery and Liquidity Tiering 63
7.6 Fundamental Financial Metric and Factor Extraction 63
7.7 Multi-Agent Reasoning: LangGraph StateGraph 64
7.8 Mathematical Portfolio Optimization (SciPy SLSQP) 64
7.8.1 Objective Function 65
7.8.2 Subject to Hard Linear and Non-Linear Constraints 65
7.9 Discrete Whole-Share Lot Allocation and Residual Cash Conservation 66
7.9.1 Floor Allocation Phase 66
7.9.2 Heuristic Remainder Allocation Phase 66
7.10 Statutory Indian Transaction-Cost Modeling 67
7.11 Existing Demat Portfolio (CAS) Diagnosis 67
7.12 Continuous Background Monitoring and Drift Radar 67

7.13 Dual-Level Fiduciary Explainability 68

8 Datasets Used and External Data Sources 69

8.1 Dataset Overview and Ingestion Pipeline 69

8.2 External Data-Source Inventory 71

8.3 Exchange Universe Coverage and Synchronization 71

8.3.1 ISIN Collision Resolution and Split Mappings 71

8.3.2 Active Universe Scale 72

8.4 Historical Market Pricing and Lookback Analysis 72

8.5 Exact Data Payloads Delivered to Multi-Agent Nodes 73

8.5.1 Sanitized Agent Context Example 73

8.6 Dataset Quality, Limitations and Fallbacks 74

9 API Contract and Integration Design 77

9.1 API Architecture and Design Principles 77

9.2 Complete API Inventory: 82 Active Endpoints 77

9.3 Detailed Specification of Core API Endpoints 78

9.3.1 Endpoint: Create Portfolio Generation Request 78

9.3.2 Endpoint: Stock-Why Fiduciary Explanation 79

9.3.3 Endpoint: Demat CAS Statement Upload & Diagnosis . . . 80

9.4 API Sequence Interaction Flows 80

9.5 Postman Collection Integration and Verification 82

10 Work Done and Functional Requirements Implementation 85

10.1 Overview of Implemented Functionalities 85

10.2 Core Functional Requirements Implementation 85

10.2.1 FR-C01: User Onboarding and Profile Management 85

10.2.2 FR-C02: Natural-Language Input Processing 85

10.2.3 FR-C03: Market Regime Classification 86

10.2.4 FR-C04: Candidate Stock Universe Discovery and Tiering . 86

10.2.5 FR-C05: Multi-Agent Reasoning via LangGraph StateGraph 86

10.2.6 FR-C06: Mathematical Optimization and LLM Decoupling . 87

10.2.7 FR-C07: Continuous Unattended Portfolio Monitoring . . . 87

10.2.8 FR-C08: Friction-Aware Rebalancing Engine 87

10.2.9 FR-C09: Real-Time Alerts and Notifications Subsystem . . . 88

10.2.10 FR-C10: Existing Demat Portfolio (CAS) Diagnosis 88

10.3 Differentiating Functional Requirement Implementation 88
10.3.1 FR-D01: Fiduciary "Stock-Why" Explainability and Transparent Critic 88
10.4 Comprehensive Requirements Traceability Matrix 89
10.5 Application User Interface Demonstrations 89

11 Results, Testing, and Evaluation **91**
11.1 Testing Strategy and Methodology 91
11.2 Test Environment Configuration 91
11.3 Automated Test Suite Execution Evidence 92
11.4 Quantitative Benchmark Backtest Evaluation 93
11.4.1 Backtest Analysis and Findings 93
11.5 Repository Consistency and Architecture Verification 94
11.6 System Performance and Latency Benchmarks 95

12 Conclusions and Future Roadmap **97**
12.1 Summary of Research and Engineering Achievements 97
12.2 Academic and Practical Contributions 98
12.3 Current System Limitations 99
12.4 Prioritized Future Roadmap 100
12.4.1 Phase 1: Broker Integration and Multi-Asset Expansion (Short-Term) 100
12.4.2 Phase 2: Advanced Quantitative and Adaptive Modeling (Medium-Term) 100
12.4.3 Phase 3: Privacy-Preserving AI and Mobile Ecosystem (Long-Term) 101
12.5 Concluding Remarks 101

REFERENCES **103**

Appendices **105**

A Complete Master API Endpoint Inventory **107**

B Complete Database Schema Inventory **111**

C Supplementary Architectural and Workflow Diagrams **117**
C.1 System and Multi-Agent Data Flows 117

C.2 API Transaction Sequence Diagrams 120
C.3 Domain-Specific Entity-Relationship Diagrams 123

D Developer Setup, Installation, and Operational Guide 131
D.1 System Prerequisites 131
D.2 One-Command Automated Launcher 131
D.2.1 macOS and Linux (Bash/Zsh) 131
D.2.2 Windows (PowerShell) 132
D.3 Companion Operational Lifecycle Scripts 132
D.4 Manual Step-by-Step Installation 133
D.4.1 Step 1: PostgreSQL Database Setup 133
D.4.2 Step 2: Backend Environment Configuration 133
D.4.3 Step 3: Database Schema Migrations and Seeding 133
D.4.4 Step 4: Frontend Installation and Build 134
D.4.5 Step 5: Backend Service Startup 134
D.5 Testing and Verification Execution 134
D.5.1 Automated Unit and Integration Testing 134
D.5.2 Repository and Architectural Consistency Verification 135
D.5.3 Postman Automated API Collection Execution 135

LIST OF FIGURES

3.1 Lucid High-Level System Architecture and Tier Decomposition . . . 16
3.2 Lucid End-to-End Data Pipeline and Lifecycle Workflow 18

4.1 Low-Level Sequence Flow: Investor Authentication and JWT Token Lifecycle 29
4.2 Low-Level Sequence Flow: Goal-Driven Portfolio Generation and SLSQP Optimization 30
4.3 Low-Level Sequence Flow: 5-Minute Drift Radar and Friction-Aware Rebalancing Preview 30
4.4 Low-Level Sequence Flow: Demat CAS PDF Statement Ingestion and Health Diagnosis 31

5.1 Lucid Master Database Entity-Relationship Diagram (63 Tables) . . 34
5.2 Domain 1 ER Diagram: Identity, Authentication and Security Governance 35
5.3 Domain 2 ER Diagram: Investor Suitability, Profiling and Financial Goals 37
5.4 Domain 3 ER Diagram: Security Master and Corporate Registry . . 39
5.5 Domain 4 ER Diagram: Market Data, Historical Quotes and Macro Regime 41
5.6 Domain 5 ER Diagram: Portfolio Engine, Quantitative Optimization and Tracking 43
5.7 Domain 6 ER Diagram: Broker Integration, Execution and Order Lifecycle 45
5.8 Domain 7 ER Diagram: Demat CAS Diagnosis and Portfolio Analytics 47
5.9 Domain 8 ER Diagram: Multi-Agent AI Telemetry, Reasoning Traces and Verification 48

6.1 Lucid Eight-Layer Decoupled System Architecture 51
6.2 NSE/BSE Exchange Universe Ingestion and Pipeline Architecture . 54

7.1 Lucid Comprehensive 16-Step Investment Engineering Methodology 60

8.1 Data Ingestion, Transformation, and Agent Payload Delivery Pipeline 70

9.1 API Sequence Diagram: Authentication and Session Token Flow . . 81

9.2 API Sequence Diagram: Portfolio Request and SLSQP Solver Flow 81
9.3 API Sequence Diagram: Continuous Drift Radar and Rebalance Flow 82
9.4 API Sequence Diagram: Demat CAS Statement PDF Diagnosis Flow 82

10.1 Lucid Web Application: Main Investor Landing and Navigation Interface 90
10.2 Lucid Web Application: Generated Portfolio Holdings and Allocation View 90

C.1 Lucid End-to-End System Data Flow: From External Feeds to Discrete Portfolios 117
C.2 Layered API-to-Agent Data Pipeline and Orchestration Architecture 118
C.3 NSE/BSE Market Data Ingestion, Reconciliation, and Split Resolution Flow 119
C.4 Authentication and Cryptographic Session Verification Sequence Flow 120
C.5 End-to-End Portfolio Generation Sequence Flow across Agents and SLSQP 120
C.6 Continuous Drift Radar and Friction-Aware Rebalancing Sequence Flow 121
C.7 Demat CAS Statement Ingestion and Diversification Diagnosis Sequence Flow 121
C.8 Domain 2 ER: Investor Suitability, Profiling, and Financial Goals . 123
C.9 Domain 3 ER: Security Master, Corporate Taxonomy, and Exchange Listings 124
C.10 Domain 4 ER: Market Quotes, Fundamental Metrics, and Economic Indicators 125
C.11 Domain 5 ER: Portfolio Engine, Optimization Parameters, and Decisions 126
C.12 Domain 6 ER: Brokerage Accounts, Discrete Orders, and Executions 127
C.13 Domain 7 ER: Demat Statement Holdings, Diagnoses, and Suggestions 128
C.14 Domain 8 ER: Multi-Agent AI Telemetry, Reasoning Traces, and Auditing 129

LIST OF TABLES

2.1 Comprehensive Competitive Evaluation Matrix 13

5.1 Schema Specification: `app_users` 35

6.1 Lucid Active Technology Stack Manifest (Verified Versions) 53

7.1 Lucid Stock Universe Liquidity Tier Classification 63
7.2 Statutory Indian Equity Delivery Transaction Fee Schedule 67

8.1 Lucid External Data-Source Inventory and Ingestion Parameters . . 71
8.2 Exact Data Payloads Delivered to LangGraph Multi-Agent Nodes . 73

9.1 Lucid Complete API Endpoint Domain Inventory (82 Endpoints) . 78

10.1 Comprehensive Functional Requirements Traceability Matrix 89

11.1 Lucid Verification and Test Environment Specification 92
11.2 Automated Test Suite Inventory and Category Breakdown (216 Tests) 92
11.3 Quantitative Performance Evaluation: Lucid vs. Baseline Benchmarks 93
11.4 Repository Consistency and Architectural Validation Results 94

A.1 Lucid Master API Inventory (All 88 Active HTTP Operations) . . . 107

B.1 Lucid Complete Relational Database Tables Inventory (63 Tables) . 111

D.1 Lucid System Prerequisites and Runtime Dependencies 131
D.2 Lucid Operational Lifecycle Scripts 132

ABBREVIATIONS

ADTV	Average Daily Traded Volume
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BSE	Bombay Stock Exchange
CAGR	Compound Annual Growth Rate
CAS	Consolidated Account Statement (CDSL/NSDL)
CDSL	Central Depository Services (India) Limited
CIN	Corporate Identity Number
CPI	Consumer Price Index
D/E	Debt-to-Equity Ratio
DMA	Daily Moving Average
GST	Goods and Services Tax
HHI	Herfindahl-Hirschman Concentration Index
HLD	High-Level Design
HTTP	Hypertext Transfer Protocol
ISIN	International Securities Identification Number
IST	Indian Standard Time (UTC+05:30)
JWT	JSON Web Token
KYC	Know Your Customer
LLD	Low-Level Design
LLM	Large Language Model
NAV	Net Asset Value
NSDL	National Securities Depository Limited
NSE	National Stock Exchange of India
OHLCV	Open, High, Low, Close, Volume
ORM	Object-Relational Mapping (SQLAlchemy)
P/B	Price-to-Book Ratio
P/E	Price-to-Earnings Ratio
RBI	Reserve Bank of India
REST	Representational State Transfer
ROE	Return on Equity
SEBI	Securities and Exchange Board of India

SLSQP	Sequential Least Squares Programming
SQL	Structured Query Language
STT	Securities Transaction Tax
UI	User Interface
UUID	Universally Unique Identifier
VIX	Volatility Index (India VIX)

Chapter 1

Introduction

1.1 Background

The Indian equity capital market has undergone an unprecedented retail revolution over the past five years. Driven by nationwide smartphone penetration, biometric Know-Your-Customer (KYC) onboarding, zero-brokerage discount platforms, and the institutionalization of the Unified Payments Interface (UPI), individual retail investor participation across the National Stock Exchange of India (NSE) and the Bombay Stock Exchange (BSE) has surged dramatically. According to data published by the Securities and Exchange Board of India (SEBI) and the depository registries (CDSL and NSDL), the total number of operational Demat accounts in India expanded from approximately 36 million in March 2019 to over 160 million by early 2026.

However, rapid growth in retail market participation has not been matched by a corresponding growth in financial literacy, quantitative risk management capability, or analytical sophistication. Individual investors routinely face an overwhelming deluge of unstructured and conflicting financial data: quarterly corporate earnings reports, regulatory disclosures, economic policy announcements by the Reserve Bank of India (RBI), daily stock price fluctuations, and speculative social media commentary. Inexperienced retail investors frequently succumb to classic behavioral finance traps, including performance chasing near cyclical market peaks, panic selling into market downturns, and extreme portfolio concentration in high-beta momentum stocks.

To navigate this complexity, retail investors traditionally choose between actively managed mutual funds, thematic portfolio baskets (such as smallcases), or digital robo-advisory platforms. Yet, mutual funds impose recurring annual expense ratios between 1.5% and 2.5% that substantially erode long-term compounding while frequently underperforming broad market indices like the Nifty 50 or BSE Sensex. Conversely, existing commercial robo-advisors function predominantly as opaque "black boxes" that provide static, generic asset allocation buckets without personalized stock-level fiduciary justifications. When market volatility strikes, investors lack the conceptual conviction required to adhere to

their investment plans, leading to premature capital liquidation.

1.2 Problem Statement

The objective of this Major Project is formulated as follows:

> *"To design, engineer, and evaluate an intelligent, fiduciary-grade equity investment portfolio platform that empowers Indian retail investors to construct, diagnose, continuously monitor, and rebalance personalized equity portfolios across NSE and BSE securities by providing transparent, context-aware, explainable reasoning traces while strictly enforcing mathematical safety and statutory regulatory constraints."*

1.3 Challenges in Retail Portfolio Construction

Designing an autonomous portfolio decision-support system for Indian retail equities presents multiple complex challenges across computational, financial, and behavioral dimensions:

1. **Information Fragmentation and Research Asymmetry**: Retail investors must manually gather financial data scattered across exchange portals, company filings, annual reports, financial news outlets, and central bank releases. Synthesizing these heterogeneous sources into a coherent investment thesis is computationally demanding and practically impossible for individual investors.

2. **Cognitive Biases and Emotional Drift**: Behavioral finance demonstrates that loss aversion causes retail investors to experience the psychological pain of a loss approximately twice as intensely as the pleasure of an equivalent gain [5]. In the absence of continuous, unattended portfolio monitoring, individual portfolios experience unmonitored drift, accumulating severe single-stock and sector concentration risks.

3. **The Black-Box Fallacy of Quantitative Robo-Advisors**: Commercial robo-advisory algorithms calculate portfolio weights using hidden parameters, offering users no explanation of why specific equities were se-

lected or rejected. This lack of transparency undermines investor trust during market corrections.

4. **Arithmetic Hallucinations in Conversational AI**: While modern Large Language Models (LLMs) excel at qualitative document synthesis and thematic summarization, empirical research demonstrates that LLMs suffer from severe arithmetic inconsistencies when tasked with mathematical portfolio optimization [6]. LLM-generated allocations routinely violate budget normalization constraints ($\sum w_i \neq 100\%$), invent negative weights without short-selling permissions, and produce volatile, non-replicable allocations in response to minor prompt variations.

5. **Exchange-Level Whole-Share and Cash Conservation Constraints**: Unlike foreign jurisdictions that support fractional equity trading, Indian exchange regulations strictly mandate discrete, integer share execution. A theoretical weight of 3.42 shares of MRF Ltd. or Page Industries is legally impossible to execute. Converting continuous portfolio weights into whole shares without careful residual cash handling creates allocation skews and cash leakage.

6. **Dynamic Exchange-Wide Universe Scale**: Real-world Indian equity markets encompass over 2,600 actively traded companies. Ingesting, cleaning, validating, and filtering this extensive universe across market capitalization, daily turnover, and liquidity metrics requires robust data pipelines resilient to API rate limits and exchange corporate actions.

1.4 Motivation

The primary motivation behind **Lucid** is to democratize the fiduciary rigor, qualitative research depth, and mathematical discipline of professional Indian mutual fund managers for ordinary retail investors, without sacrificing execution safety or transparency.

Recent advancements in Agentic Artificial Intelligence provide a compelling architectural paradigm [4]. Rather than treating portfolio construction as a monolithic text-generation or pure black-box numerical task, the investment lifecycle can be decomposed into specialized cognitive roles: macroeconomic

monetary assessment, market regime classification, fundamental balance-sheet screening, adversarial risk critique, and fiduciary explanation.

Crucially, execution safety can be guaranteed by strictly decoupling qualitative AI reasoning from mathematical optimization. By restricting the LLM to qualitative context synthesis and tasking a deterministic mathematical solver (SciPy SLSQP) with constraint enforcement, the system achieves the best of both worlds: rich, transparent, human-readable explanations combined with provable, hallucination-free mathematical correctness [3].

1.5 Project Objectives

The specific technical and research objectives of this Major Project are:

1. To design and implement a multi-agent AI architecture using cyclical state graphs (LangGraph) that coordinates specialized cognitive nodes to evaluate macroeconomic trends, market volatility regimes, corporate fundamentals, and company news.

2. To establish a strict mathematical decoupling architecture where qualitative agent outputs are validated and optimized by a deterministic Sequential Least Squares Programming (SLSQP) solver enforcing hard diversification bounds ($\leq 25\%$ single-stock, $\leq 40\%$ sector).

3. To implement a discrete whole-share integer lot allocation algorithm that translates continuous percentage weights into executable Indian equity share quantities while conserving residual cash balances.

4. To construct an institutional-grade, normalized relational database architecture (PostgreSQL 16) comprising 62 domain application tables (+1 alembic tracking table) housing the full synchronized universe of 2,639 NSE/BSE listed equities.

5. To develop a continuous, unattended background monitoring radar running every 5 minutes during Indian market hours (09:15–15:30 IST) that audits holding drift ($\geq 5\%$) and sector drift ($\geq 8\%$), generating friction-aware rebalancing trade previews factoring in statutory transaction costs (STT, GST, SEBI fees).

6. To engineer a Demat Consolidated Account Statement (CAS) diagnostic engine capable of parsing CDSL/NSDL portfolio statements, calculating Herfindahl-Hirschman (HHI) concentration indices, and detecting holding skews.

1.6 Scope of the Project

The operational scope of Lucid encompasses:

- **Asset Class**: Equities listed and traded on the National Stock Exchange of India (NSE) and the Bombay Stock Exchange (BSE) in the cash delivery segment.
- **Target Audience**: Indian retail investors seeking customized, goal-driven equity portfolio construction, health diagnosis of existing Demat holdings, and continuous drift monitoring.
- **Execution Mechanics**: Discrete whole-share allocations; fractional lot trading is excluded as it is legally prohibited on Indian exchanges.
- **Regulatory Stance**: Lucid operates as an intelligent decision-support platform providing transparent fiduciary reasoning. Internal single-stock and sector limits represent configurable risk policies and do not constitute statutory SEBI advisory certifications.
- **Exclusions**: Intraday margin trading, derivative instruments (Futures and Options), short selling, and automated broker order placement (mock broker adapters are provided for simulation).

1.7 Proposed Solution: Lucid

Lucid is engineered as a decoupled, multi-layered intelligent platform. The client presentation layer is implemented in Next.js 16.4.2 and React 19, communicating via type-safe REST APIs with a high-performance FastAPI 0.115+ backend. The core intelligence resides in a 13-node LangGraph StateGraph that orchestrates Google Gemini 3.8 Flash (backed by deterministic rule fallbacks).

The platform continuously maintains a synchronized catalog of 2,639 listed exchange securities, categorizing them into four institutional Liquidity Tiers,

with 504 liquid securities eligible for portfolio recommendations. Portfolio generation requests undergo deterministic SciPy SLSQP optimization, followed by discrete lot sizing and statutory friction modeling. Post-creation, active portfolios are monitored by an asynchronous background scheduler during official exchange trading hours.

1.8 Key Contributions and Differentiators

The key contributions of this Major Project include:

1. **Architectural Decoupling of LLM and Arithmetic**: Complete elimination of generative arithmetic hallucinations by confining the LLM to qualitative screening and delegating numerical optimization to SciPy SLSQP.
2. **Discrete Integer Lot Allocation & Cash Conservation**: A production-tested lot-sizing algorithm ensuring 100% executable share allocations on Indian exchanges with exact residual cash accounting.
3. **5-Minute Market Hours Drift Radar**: Unattended, session-locked background monitoring evaluating single-stock drift ($\geq$ 5%) and sector drift ($\geq$ 8%) during IST trading hours.
4. **Dual-Level Fiduciary Explainability**: Transparent, toggleable explanations providing intuitive natural language justifications for retail investors and verifiable econometric factor scores for institutional review.
5. **Full NSE/BSE Universe Integration**: A verified PostgreSQL 16 schema supporting 2,639 listed equities and 2,715 exchange listings, avoiding restricted sub-universe limitations.

1.9 Report Organization

The remainder of this report is organized as follows:

- **Chapter 2** presents the Literature Review, Market Survey, and Competitive Analysis of existing robo-advisory and AI investment platforms.
- **Chapter 3** details the High-Level Design (HLD), major architectural subsystems, and end-to-end user workflows.

- **Chapter 4** presents the Low-Level Design (LLD), detailing module hierarchies, agent state graphs, and API schemas.
- **Chapter 5** provides the complete Database Design and ER Architecture, documenting all 62 domain application tables (+1 alembic tracking table) across 8 functional domains.
- **Chapter 6** explains the System Architecture, runtime execution topology, technology stack manifest, and security design.
- **Chapter 7** details the Methodology, including mathematical formulations for Markowitz SLSQP optimization, integer lot sizing, and drift calculation.
- **Chapter 8** documents the Datasets Used, external data providers, and exact payloads passed to the AI agents.
- **Chapter 9** specifies the complete API Contract across all 82 REST endpoints and Postman integration assets.
- **Chapter 10** demonstrates the Functional Requirements Implementation (FR-C01 through FR-D01) and application UI screens.
- **Chapter 11** presents Empirical Results, Testing, and Evaluation across 216 automated tests and backtest comparisons.
- **Chapter 12** concludes the report and outlines Future Scope.
- **Appendices** provide supplementary technical inventories, diagrams, test matrices, and installation manuals.

Chapter 2

Literature Review and Competitive Analysis

2.1 Theoretical and Academic Foundations

Modern financial theory and autonomous portfolio management rest upon several foundational pillars across mathematical optimization, behavioral economics, and artificial intelligence.

2.1.1 Modern Portfolio Theory and Mean-Variance Optimization

Harry Markowitz's seminal 1952 work on Modern Portfolio Theory (MPT) established the mathematical foundation of quantitative asset allocation [7]. Prior to MPT, investment decisions focused on individual security valuation in isolation. Markowitz demonstrated that an asset's risk and return should not be evaluated separately, but by how it contributes to an overall portfolio's risk-return profile. By modeling the expected return of a portfolio as the weighted sum of individual asset expected returns:

$$\mathbb{E}[R_p] = \sum_{i=1}^{N} w_i \mathbb{E}[R_i] = \mathbf{w}^T \boldsymbol{\mu} \tag{2.1}$$

and portfolio variance as a function of the pairwise covariance between assets:

$$\sigma_p^2 = \sum_{i=1}^{N} \sum_{j=1}^{N} w_i w_j \mathrm{Cov}(R_i, R_j) = \mathbf{w}^T \boldsymbol{\Sigma} \mathbf{w} \tag{2.2}$$

Markowitz formulated portfolio selection as a quadratic optimization problem. William Sharpe extended this framework in 1964 through the Capital Asset Pricing Model (CAPM), introducing the Sharpe ratio to quantify risk-adjusted excess return per unit of volatility [10]:

$$S_p = \frac{\mathbb{E}[R_p] - R_f}{\sigma_p} \tag{2.3}$$

where R_f represents the risk-free rate of return (e.g., the 91-day Government of India Treasury Bill yield or the RBI Policy Repo Rate).

2.1.2 Factor Investing and Behavioral Finance

Fama and French (1993) demonstrated that market beta alone is insufficient to explain cross-sectional equity returns, introducing size (SMB) and value (HML) factors [2]. In emerging markets such as India, momentum, quality (high Return on Equity, low Debt-to-Equity), and volatility factors demonstrate significant predictive power.

Concurrently, behavioral economists Daniel Kahneman and Amos Tversky established Prospect Theory, demonstrating that human investors systematically violate expected utility theory due to loss aversion, anchoring, and recency bias [5]. In the Indian equity market, retail investors frequently hold losing stocks for excessive periods in the hope of breaking even (disposition effect) while liquidating winning positions prematurely, resulting in suboptimal long-term compounding.

2.1.3 Agentic AI and LLMs in Financial Decision Support

Recent advances in generative Artificial Intelligence have introduced Large Language Models (LLMs) to financial document analysis and thematic synthesis. However, recent empirical investigations reveal severe limitations when LLMs are deployed in quantitative contexts. Kulpa et al. (2025) demonstrated that LLMs suffer from persistent arithmetic hallucinations, producing asset weights that fail to sum to 100% or assigning negative allocations in long-only portfolios [6].

Conversely, multi-agent architectures (such as LangGraph) coordinate specialized agents to debate investment theses, cross-examine risks, and generate explainable audit trails [3, 4]. The prevailing consensus in financial machine learning emphasizes a **hybrid architecture**: leveraging LLMs for unstructured textual and thematic reasoning while delegating numerical optimization to deterministic mathematical solvers [8].

2.2 Competitive Landscape in Indian Retail Investing

To evaluate current industry capabilities and identify existing technological gaps, a comprehensive competitive review was conducted across six prominent Indian platforms and one leading international academic AI benchmark.

2.2.1 smallcase

Overview: smallcase is an Indian fintech platform that offers curated thematic or factor-based model portfolios created by registered research analysts or mutual fund houses. Users connect their existing demat accounts (e.g., Zerodha, Upstox, Groww) to execute orders.

Strengths: Excellent broker integration, transparent holdings visibility, and accessible entry points.

Limitations: Portfolios are static and thematic rather than dynamically tailored to an individual's personal financial goals or risk appetite. Portfolio rebalancing requires manual user confirmation upon email notification; there is no continuous background drift monitoring or personalized stock-level fiduciary explanation.

2.2.2 Univest

Overview: Univest provides retail investors with stock ideas, trade alerts, and portfolio health checks powered by algorithmic scoring.

Strengths: Intuitive mobile interface with clear BUY/SELL/HOLD indicators.

Limitations: Recommendations focus on isolated stock tips rather than holistic, risk-adjusted portfolio construction. It lacks Markowitz mean-variance optimization, formal sector exposure constraints, and discrete whole-share cash conservation algorithms.

2.2.3 Motilal Oswal — Intelligent Advisory Portfolios (IAP)

Overview: IAP provides algorithmic, professionally managed equity portfolios across market capitalizations for high-net-worth retail clients.

Strengths: Robust institutional research backed by full-service broker execution and automated rebalancing.

Limitations: Demands a prohibitive minimum capital threshold ($\geq$ Rs. $5,00,000$), rendering it inaccessible to mass retail investors. Furthermore, the underlying quantitative algorithms operate as opaque black boxes offering zero explainability into stock selection rationale.

2.2.4 Savart

Overview: Savart is an AI-driven investment advisory platform utilizing a proprietary algorithm ("Vantage") to construct customized portfolios based on investor psychometrics.

Strengths: Strong focus on long-term behavioral alignment and personalized risk profiling.

Limitations: Operates as a proprietary algorithmic black box. Investors receive allocation percentages without verifiable factor scores, economic regime justifications, or adversarial risk critique.

2.2.5 Wright Research

Overview: Wright Research is a SEBI-registered quantitative investment firm specializing in factor investing, momentum models, and macroeconomic asset allocation.

Strengths: Sophisticated quantitative discipline, high historical Sharpe ratios, and systematic regime-switching rules.

Limitations: While quantitative performance is strong, the platform lacks natural language conversational onboarding, automated Demat CAS statement diagnostic tools, and transparent, plain-English "Stock-Why" explanations.

2.2.6 Jarvis Invest

Overview: Jarvis Invest is an artificial intelligence-driven advisory application that analyzes millions of data points to create personalized, risk-managed portfolios.

Strengths: Dynamic risk management and continuous monitoring with automated rebalance alerts.

Limitations: The platform is entirely black-box in nature. Users cannot inspect the qualitative or quantitative debate that led to specific security allocations, creating skepticism during inevitable drawdown phases.

2.2.7 MarketSenseAI (International Academic Benchmark)

Overview: Developed by academic researchers (Fatouros et al., 2024), MarketSenseAI represents the state-of-the-art in multi-agent LLM financial reasoning, employing specialized agents to analyze news, fundamentals, and market sentiment [3].

Strengths: Pioneered agentic debate architectures and demonstrated market-beating returns in US equities.

Limitations: Engineered strictly for US markets; it completely lacks support for Indian exchange mechanics, discrete whole-share lot constraints, statutory Indian transaction taxes (STT, GST, stamp duty), and CDSL/NSDL Demat statement ingestion.

2.3 Comprehensive Comparative Matrix

Table 2.1 provides an exhaustive comparative evaluation across fifteen technical, financial, and operational dimensions.

Table 2.1: Comprehensive Competitive Evaluation Matrix

Capability / Feature Dimension	smallcase	Univest	Motilal IAP	Savart	Wright Res.	Jarvis	MarketSense	Lucid (Ours)
Multi-Agent Cognitive AI Architecture	No	No	No	No	No	No	**Yes**	**Yes (13 Nodes)**
Strict Mathematical Decoupling (SciPy SLSQP)	No	No	No	No	Yes	No	No	**Yes**
Discrete Integer Lot Sizing (No Fractional Shares)	Partial	No	Yes	Yes	Yes	Yes	No	**Yes**
Residual Cash Balance Conservation	No	No	Yes	Partial	Yes	Partial	No	**Yes**
Continuous 5-Min Background Monitoring Radar	No	No	No	No	No	Partial	No	**Yes (IST Trading)**
Fiduciary Dual-Level "Stock-Why" Explanations	No	No	No	No	No	No	Partial	**Yes**
Demat CAS Statement (PDF) Diagnosis	No	Partial	No	No	No	No	No	**Yes (CDSL/NSDL)**
Full NSE/BSE Listed Security Master (2,600+ Stocks)	Partial	Partial	Partial	Partial	Partial	Partial	N/A	**Yes (2,639 Stocks)**
Macroeconomic Regime Adaptation (India VIX/RBI)	No	No	Partial	No	Yes	Partial	Partial	**Yes**
Statutory Indian Friction Modeling (STT, GST, SEBI)	No	No	Yes	No	No	No	No	**Yes**
Herfindahl-Hirschman (HHI) Concentration Analysis	No	No	No	No	No	No	No	**Yes**
Stress-Scenario Simulation (2008, COVID, Rate Hike)	No	No	No	No	Partial	No	No	**Yes**
Enterprise Security (Argon2id Password Hashing)	Unclear	Unclear	Yes	Unclear	Unclear	Unclear	N/A	**Yes (RFC 9106)**
Accessible to Retail Investors ($< \text{Rs. } 10,000$)	**Yes**	**Yes**	No	**Yes**	**Yes**	**Yes**	N/A	**Yes**
100% Transparent Implementation / Auditability	Partial	No	No	No	No	No	Yes	**Yes**

2.4 Outcome of the Competitive Review and Design Rationale

The competitive analysis demonstrates that existing commercial solutions force retail investors into an unacceptable tradeoff: they must either accept **static, non-personalized baskets** (smallcase), **opaque black-box models** (Savart, Jarvis), or **high-capital institutional services** (Motilal Oswal IAP).

Furthermore, while academic frameworks like MarketSenseAI validate the cognitive power of agentic LLMs, they fail when applied to real-world Indian equity markets due to fractional share assumptions and ignorance of Indian statutory taxes.

These empirical findings directly informed Lucid's core architectural commitments:

1. **Guaranteed Arithmetic Soundness**: The LLM is never permitted to emit final portfolio weights. SciPy's deterministic SLSQP solver computes allocations, ensuring zero arithmetic hallucinations.

2. **Real-World Execution Viability**: Allocations are converted into discrete whole shares with exact rupee cash conservation, preventing trade execution failures on Indian exchanges.

3. **Unattended Protection via Continuous Radar**: Lucid's background scheduler monitors portfolio drift every 5 minutes during Indian market hours, freeing retail investors from constant screen monitoring.

4. **Fiduciary Conviction through Transparency**: By coupling plain-English rationales with verifiable econometric factor metrics, Lucid equips investors with the conviction needed to stay invested during market downturns.

Chapter 3

High-Level Design (HLD)

3.1 HLD Overview

The High-Level Design (HLD) defines the structural blueprint, subsystem boundaries, inter-component communication channels, and primary user journeys governing the Lucid platform. Lucid is architected as an institutional-grade, multi-tier software system engineered specifically to meet the high-throughput, low-latency, and deterministic computational demands of Indian retail equity investing.

The platform strictly adheres to the principle of **Separation of Concerns (SoC)**:

- Qualitative contextual reasoning is isolated within an asynchronous multi-agent artificial intelligence framework.
- Quantitative numerical optimization is executed within a dedicated mathematical solver.
- Financial transaction state and security master catalogs are safeguarded within an ACID-compliant relational persistence engine.

3.2 Major System Components

Lucid is decomposed into seven primary architectural subsystems, as illustrated in Figure 3.1:

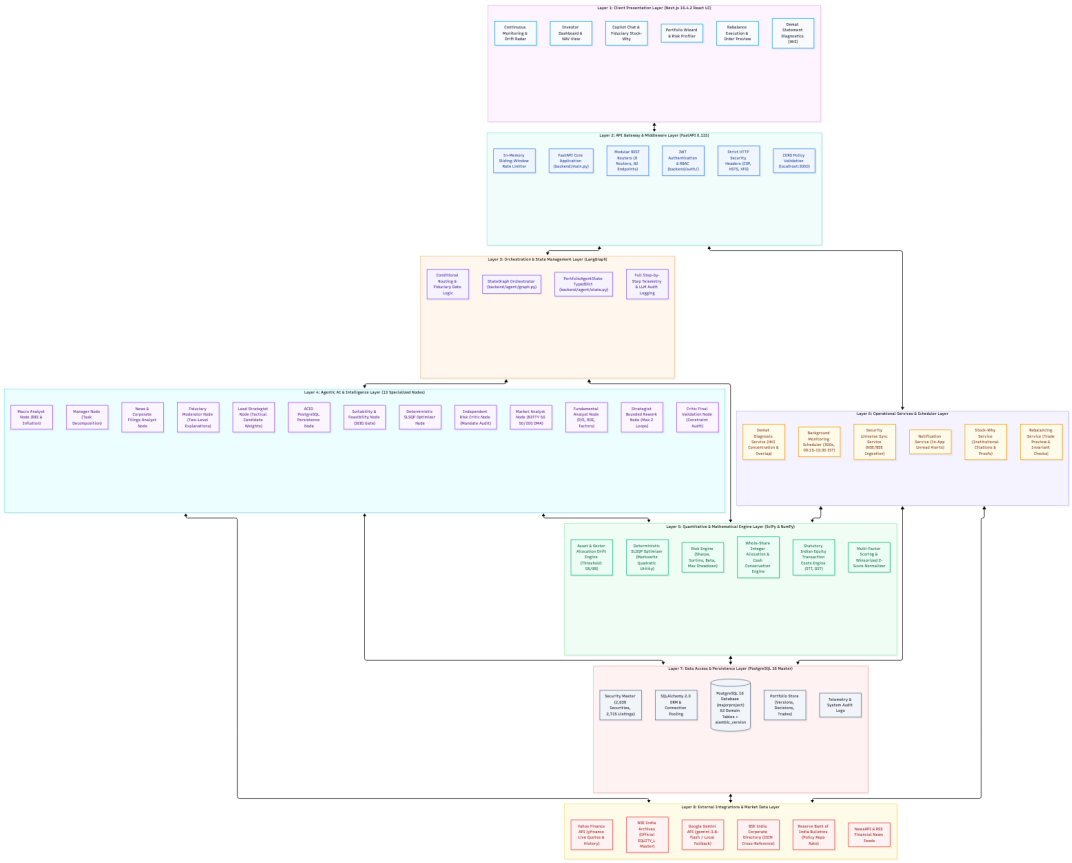


Figure 3.1: Lucid High-Level System Architecture and Tier Decomposition

3.2.1 Client Presentation Subsystem

Developed using **Next.js 16.4.2** (utilizing the App Router paradigm) and **React 19.3.0**, the presentation subsystem delivers a responsive, type-safe web dashboard for retail investors. The client implements interactive risk questionnaires, goal planning tools, asset allocation visualizers, stock explanation modals, Demat CAS file upload zones, and drift radar dashboards. All user interactions communicate asynchronously with the backend via a type-safe API client layer (`frontend/src/lib/api.ts`).

3.2.2 API Gateway and Security Subsystem

Powered by **FastAPI 0.115+** and running on the **Uvicorn ASGI** web server, the API layer acts as the centralized gateway for all incoming HTTP/HTTPS traffic. It enforces:

- Cross-Origin Resource Sharing (CORS) policies.

- Cryptographic authentication using JSON Web Tokens (JWT) signed with 256-bit secret keys.
- Enterprise HTTP security headers (nosniff, frame-options DENY, XSS protection).
- In-memory sliding-window rate limiting to protect computationally expensive AI generation endpoints from resource exhaustion.

3.2.3 Business and Application Service Subsystem

This layer houses the core business logic governing user profiles, financial goal lifecycle management, market data retrieval, liquidity tier screening, portfolio performance calculation, Demat statement parsing, and friction-aware rebalancing.

3.2.4 Multi-Agent AI Orchestration Subsystem

Implemented using **LangGraph 0.4 StateGraph**, this subsystem coordinates a collective of specialized cognitive agent nodes. Rather than relying on a single prompt, the workflow systematically evaluates macroeconomic monetary policy, India VIX market volatility regimes, corporate fundamental metrics, and market news sentiment. It synthesizes qualitative conviction scores and detailed fiduciary explanations.

3.2.5 Quantitative and Optimization Subsystem

Completely decoupled from the LLM, this subsystem utilizes **SciPy 1.15's SLSQP solver** (`scipy.optimize.minimize`) to compute constrained Markowitz mean-variance allocations. The quantitative engine includes a deterministic integer lot sizing module and a statutory transaction friction model.

3.2.6 Data Persistence and Schema Subsystem

The exclusive source of truth for the entire platform is **PostgreSQL 16**. Managed via **SQLAlchemy 2.0** ORM and version-controlled using **Alembic**, the database comprises **62 domain application tables** (+1 alembic tracking table = 63 tables) organized into 8 functional domains.

3.2.7 External Market Data and Feeds Subsystem

This subsystem manages continuous integration with external data providers, including official NSE archives (`EQUITY_L.csv`), BSE equity masters, Yahoo Finance market data feeds, Reserve Bank of India macroeconomic publications, and Google Gemini Generative AI APIs.

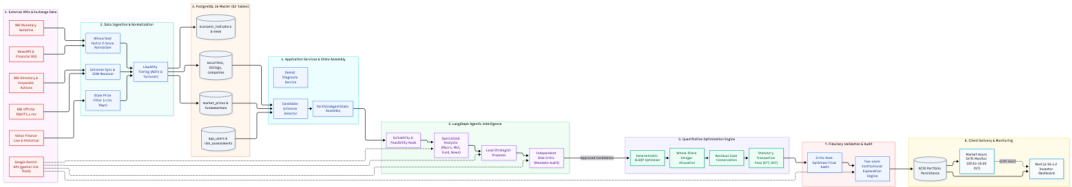


Figure 3.2: Lucid End-to-End Data Pipeline and Lifecycle Workflow

3.3 Component Interaction and Data Flow

Figure 3.2 depicts the end-to-end data lifecycle across the platform. The interaction follows a structured pipeline:

1. **Ingestion & Normalization**: Raw security masters and daily quotes are fetched from NSE/BSE and Yahoo Finance, validated against database constraints, and normalized into `securities`, `instrument_listings`, and `market_prices`.

2. **Investor Context Acquisition**: The user completes the SEBI-aligned risk questionnaire and defines investment capital and financial goals via the Next.js UI.

3. **Universe Candidate Selection**: The candidate selection service queries the 2,639-security database, applying liquidity, market cap, and turnover filters to extract liquid candidates.

4. **Agentic Contextual Synthesis**: LangGraph triggers the 13-node multi-agent pipeline. The agents evaluate monetary policy, volatility regimes, balance sheet health, and news, producing risk-adjusted candidate scores and an adversarial critic trace.

5. **Deterministic SLSQP Optimization**: The mathematical engine takes the screened universe and executes numerical optimization subject to budget and sector constraints.

6. **Integer Lot Sizing & Cash Conservation**: Continuous weights are converted into discrete whole shares, calculating exact residual cash.

7. **Fiduciary Explanation Generation**: Explanation nodes produce dual-level retail and institutional justifications, persisted into `recommendation_explanations`

8. **Continuous Monitoring & Rebalancing**: The background scheduler polls active portfolios during market hours, auditing price changes and computing drift.

3.4 User-Level End-to-End Workflows

3.4.1 Workflow 1: User Onboarding and Risk Profiling

An investor registers using email and password, hashed via Argon2id. The investor completes the five-stage SEBI-aligned suitability questionnaire evaluating financial knowledge, investment horizon, loss tolerance, and return expectations. The platform computes an aggregate risk score, mapping the investor into one of three risk categories: **Conservative**, **Moderate**, or **Aggressive**.

3.4.2 Workflow 2: Goal-Driven Portfolio Generation

The investor specifies investment capital (e.g., Rs. $2,50,000$), an optional financial goal (e.g., "Retirement corpus in 15 years"), and sector preferences or exclusions. The backend coordinates candidate selection, LangGraph agent synthesis, and SciPy SLSQP optimization, returning a discrete, whole-share portfolio displaying exact share quantities, target weights, sector skews, and residual cash balances.

3.4.3 Workflow 3: Fiduciary "Stock-Why" Explainability

For every holding in a recommended portfolio, the user can click "Stock-Why" to inspect the complete fiduciary justification. The UI displays both a plain-English narrative for retail comprehension and a quantitative breakdown of underlying econometric factor scores (ROE, P/E, Debt-to-Equity, Profit Margins, Beta, and 1-Year Price Lookback).

3.4.4 Workflow 4: Demat Statement (CAS) Upload and Diagnosis

An investor uploads a PDF Consolidated Account Statement (CAS) from CDSL or NSDL. The backend `cas_parser` extracts stock ISINs, quantities, and purchase prices. The diagnostic engine maps holdings against the master security database, computes total portfolio valuation, evaluates sector concentration, calculates the Herfindahl-Hirschman Index (HHI), flags overlapping stocks, and provides actionable HOLD/BUY/SELL recommendations.

3.4.5 Workflow 5: Continuous 5-Minute Background Monitoring

During official Indian market hours (Monday to Friday, 09:15–15:30 IST), the `BackgroundMonitoringScheduler` wakes up every 300 seconds. It acquires a PostgreSQL distributed session lock (`pg_try_advisory_lock`), fetches live market quotes, revalues active portfolios, and checks whether single-stock drift ($\geq 5\%$) or sector drift ($\geq 8\%$) has occurred.

3.4.6 Workflow 6: Friction-Aware Rebalancing Order Preview

When drift exceeds configured thresholds, Lucid generates a rebalancing alert. The user reviews a friction-aware order preview detailing exact BUY and SELL share quantities, net cash impact, and estimated statutory transaction fees (STT, exchange turnover, SEBI charges, stamp duty, and GST).

3.5 Key Architectural Design Decisions

1. **Why PostgreSQL 16 as Single Source of Truth?**: Financial systems require ACID compliance, foreign key constraints, and relational consistency across orders, holdings, and user ledgers. A normalized 63-table schema in PostgreSQL eliminated the synchronization overhead and eventual consistency risks associated with dual-database (graph + relational) setups.

2. **Why SciPy SLSQP over LLM-Generated Weights?**: Large Language Models cannot perform quadratic mathematical optimization reliably. Delegating numerical solver execution to SciPy SLSQP guarantees that allocations strictly sum to 100.0% and respect diversification bounds.

3. **Why Whole-Share Lot Sizing?**: Real Indian equities cannot be purchased in fractional units. Implementing discrete lot allocation ensures that recommendations are 100% executable on Indian exchanges.

4. **Why LangGraph StateGraph?**: LangGraph enables cyclical agent coordination with shared, typed state dictionaries (`TypedDict AgentState`), providing transparent execution telemetry, fallback handling, and structured JSON output validation.

Chapter 4

Low-Level Design (LLD)

4.1 LLD Overview

The Low-Level Design (LLD) provides an in-depth, class-level, and schema-level specification of Lucid's software components, data structures, algorithm implementations, and sequence flows. It translates the high-level architecture into precise technical abstractions implemented across the Next.js frontend, FastAPI backend, LangGraph agent workflows, and quantitative optimization engines.

4.2 Frontend Architecture and Component Hierarchy

The client application is built with **Next.js 16.4.2** utilizing the React 19 App Router architecture (`frontend/src/app/`). State management leverages React hooks (`useState`, `useEffect`, `useCallback`) alongside local storage caching for authenticated session tokens.

4.2.1 Component Structure and Responsibilities

The presentation tier is structured into modular, decoupled React components located in `frontend/src/components/`:

- `Navbar.tsx` & `Footer.tsx`: Navigation header managing global auth state, active user profile, Demat connection status, and responsive layout toggles.
- `LandingPage.tsx`: Marketing hero section, feature highlights, and interactive entry points for new and returning investors.
- `AuthView.tsx` & `ProfileAuthModal.tsx`: Authentication dialogs managing email/password registration, login, JWT token persistence, and input validation.

- `OnboardingWizard.tsx` & `RiskQuestionnaireView.tsx`: Five-step wizard presenting SEBI suitability questions, computing real-time risk scores, and persisting assessment results.
- `PortfolioGenerator.tsx`: Interactive portfolio configuration panel allowing users to specify investment capital, select or define financial goals, and set sector preferences or exclusions.
- `HoldingsView.tsx` & `AllocationVisualizer.tsx`: Portfolio display interface rendering holding tables, whole-share lot quantities, average purchase prices, live NAV, and interactive sector/asset breakdown charts.
- `StockWhyModal.tsx` & `ExplainabilityView.tsx`: Fiduciary explainability modals rendering dual-level justifications (retail narrative and quantitative econometric factor scores).
- `DiagnosisView.tsx` & `DiagnosisCopilotModal.tsx`: Drag-and-drop PDF CAS statement uploader, holdings extraction parser, and AI-assisted Demat health diagnostic interface.
- `RebalancePreviewModal.tsx`: Interactive modal displaying proposed BUY/SELL share orders, net cash changes, and statutory transaction cost breakdowns.
- `BenchmarksView.tsx` & `StressScenariosView.tsx`: Comparative performance analytics comparing Lucid against Equal-Weight (1/N) and Nifty 50 benchmarks, including stress testing under historical market shocks.
- `ConversationalAssistant.tsx`: Natural language chat interface enabling investors to query portfolio decisions and explore stock rationale interactively.
- `NotificationsPanel.tsx`: Real-time notification drawer displaying drift alerts, rebalance recommendations, and system events.

4.2.2 Type-Safe Client API Layer (`frontend/src/lib/api.ts`)

All network communication with the backend is abstracted through a type-safe API module. It handles:

- Dynamic base URL resolution (`http://localhost:8000`).

- Automatic injection of `Authorization: Bearer <token>` headers.

- Unified response parsing and error handling, mapping HTTP status codes (400, 401, 403, 404, 422, 429, 500) into structured UI alerts.

4.3 Backend Architecture and API Subsystem

The backend is structured as a modular, layered FastAPI application under `backend/`:

```
backend/
|-- api/
|   `-- v1/              # Versioned API routes (82 endpoints)
|       |-- auth.py      # Authentication, login, registration, JWT
|       |-- users.py     # Profile management, preferences
|       |-- risk.py      # Suitability questionnaire & scoring
|       |-- goals.py     # Financial goal CRUD & progress tracking
|       |-- market_data.py # Quotes, historical prices, regime, universe
|       |-- portfolios.py # Generation requests, versioning, stock-why
|       |-- diagnosis.py # Demat CAS PDF parsing & health diagnosis
|       |-- monitoring.py # Portfolio valuation & drift tracking
|       |-- rebalancing.py # Rebalancing strategy & order previews
|       |-- notifications.py # Real-time alerts
|       `-- broker.py    # Broker simulation & order execution
|-- models/              # SQLAlchemy 2.0 ORM domain models (62 tables)
|-- schemas/             # Pydantic v2 request & response validation schemas
|-- services/            # Business logic and external service orchestrators
|-- portfolio_engine/    # Quantitative optimization, rounding, transaction co
|-- agents/              # LangGraph multi-agent StateGraph workflow
|-- database/            # Engine configuration, sessionmaker, get_db depender
|-- middleware/          # Rate limiting, security headers, CORS
|-- alembic/             # Database migration revisions
`-- config.py            # Centralized settings (pydantic-settings)
```

4.3.1 Dependency Injection and Session Management

FastAPI's dependency injection system (`Depends`) manages transactional database sessions:

$$\text{get_db}() \rightarrow \text{Generator}[\text{Session}, \text{None}, \text{None}] \tag{4.1}$$

The `get_db` dependency yields a thread-local SQLAlchemy `SessionLocal` bound to PostgreSQL, ensuring that database connections are returned to the connection pool (`pool_size=10, max_overflow=20`) immediately upon request completion.

4.4 Agent-Level Design: 13-Node LangGraph StateGraph

The multi-agent workflow is implemented in `backend/agents/graph.py` and `backend/agents/state.py` using **LangGraph 0.4**. The state across the entire execution graph is governed by a unified TypedDict:

```
class AgentState(TypedDict):
    request_id: str
    user_id: str
    risk_category: str          # conservative, moderate, aggressive
    investment_amount: float
    goal_type: Optional[str]
    time_horizon_years: Optional[int]
    macro_regime: Dict[str, Any]
    market_volatility: Dict[str, Any]
    candidate_universe: List[Dict[str, Any]]
    fundamental_scores: Dict[str, float]
    sentiment_scores: Dict[str, float]
    critic_feedback: List[str]
    optimized_allocations: List[Dict[str, Any]]
    fiduciary_explanations: Dict[str, str]
    validation_status: bool
    error_messages: List[str]
```

4.4.1 Agent Nodes and Execution Topology

The 13 nodes of the StateGraph execute in structured sequence with conditional validation edges:

1. **Input Validation Node**: Validates investor capital ($\geq$ Rs. $5,000$), risk category, and time horizon.
2. **Macro Analyst Node**: Analyzes RBI repo rate trends, inflation metrics, and GDP forecasts.
3. **Market Regime Node**: Evaluates India VIX volatility tiers and Nifty 50 50-DMA/200-DMA technical trends.
4. **Universe Ingestion Node**: Queries PostgreSQL for active listed equities matching liquidity thresholds.
5. **Fundamental Analyst Node**: Filters candidates against balance sheet criteria (ROE $\geq 15\%$, Debt/Equity ≤ 1.5, positive operating cash flows).
6. **Sentiment & News Node**: Assesses ingested corporate news sentiment and corporate governance filings.
7. **Valuation Screening Node**: Evaluates relative P/E, P/B, and historical EV/EBITDA ratios against industry benchmarks.
8. **Candidate Ranking Node**: Combines fundamental, sentiment, and valuation scores into an aggregate conviction score.
9. **Adversarial Critic Node**: Challenges top-ranked candidates, searching for unaddressed regulatory risks, sector concentration traps, or cyclical headwinds.
10. **Quantitative SLSQP Solver Node**: Formulates and executes SciPy constrained mean-variance optimization.
11. **Integer Lot Sizing Node**: Translates percentage weights into whole shares and computes residual cash.
12. **Deterministic Constraint Validator Node**: Validates that no single stock exceeds 25% and no sector exceeds 40%. If violated, routes back to SLSQP with tightened bounds.

13. **Dual-Level Explainability Node**: Generates user-facing retail narratives and structured econometric factor scorecards.

4.5 Quantitative Optimization and Lot Sizing Engine

Located in `backend/portfolio_engine/`, the quantitative engine implements:

- **Optimizer (`optimizer.py`)**: Configures the SLSQP solver (`scipy.optimize.minimize`), setting objective functions to minimize negative Sharpe ratio (or minimize portfolio variance for conservative profiles).

- **Integer Lot Rounder (`integer_rounding.py`)**: Executes whole-share floor rounding:

$$q_i = \left\lfloor \frac{C \cdot w_i^*}{P_i} \right\rfloor, \quad \text{Residual Cash} = C - \sum_{i=1}^{K} q_i P_i \tag{4.2}$$

  where C is total capital, w_i^* is optimal continuous weight, and P_i is current stock price. A secondary heuristic pass allocates residual cash to the highest-conviction holdings whose unit prices fit within the remaining cash balance without breaching the 25% single-stock ceiling.

- **Transaction Cost Engine (`transaction_costs.py`)**: Calculates all Indian statutory costs (STT 0.1%, Exchange charges 0.00325%, SEBI turnover fee 0.0001%, Stamp duty 0.015%, and GST 18% on brokerage and exchange fees).

4.6 Monitoring Scheduler and Distributed Locking

Located in `backend/services/monitoring_scheduler.py`, the unattended background monitoring service runs on a dedicated background thread with an asynchronous execution loop (`interval_seconds=300`).

To prevent concurrent execution across multi-worker deployments (e.g., multiple Uvicorn instances), the scheduler implements **PostgreSQL Session-Level**

Advisory Locking:

$$\text{SELECT pg_try_advisory_lock(847291)} \tag{4.3}$$

Only the worker instance that acquires the 64-bit lock proceeds with portfolio valuation and drift auditing; secondary workers log an informative skip message and release the cycle.

4.7 Sequence Flow Diagrams

Figures 4.1 through 4.4 illustrate the low-level sequence interactions for primary system operations.

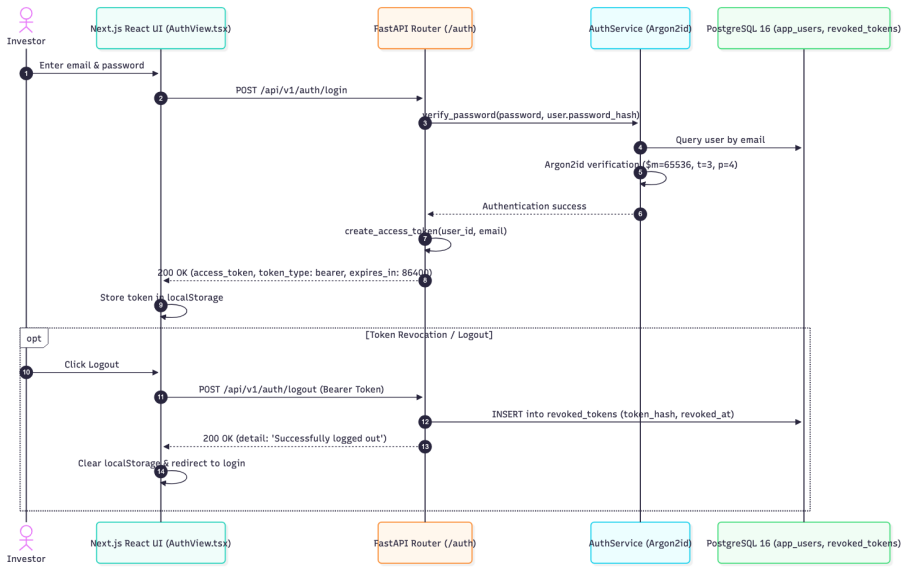


Figure 4.1: Low-Level Sequence Flow: Investor Authentication and JWT Token Lifecycle

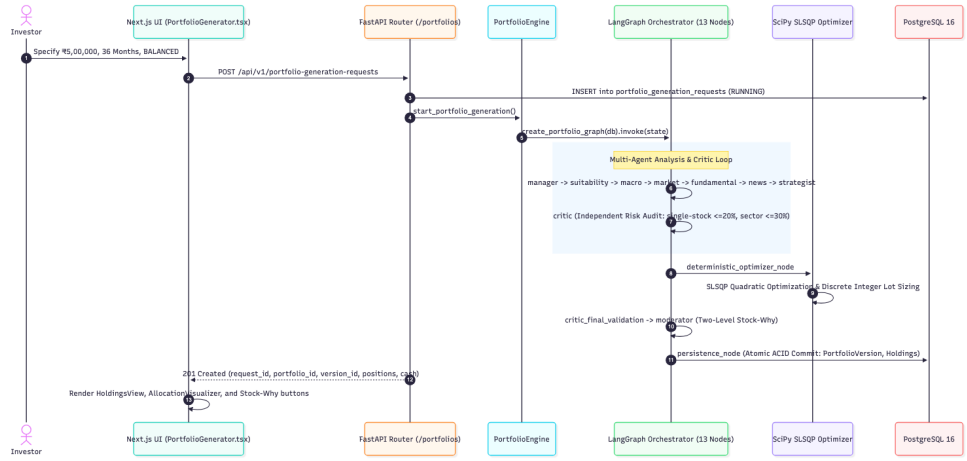


Figure 4.2: Low-Level Sequence Flow: Goal-Driven Portfolio Generation and SLSQP Optimization

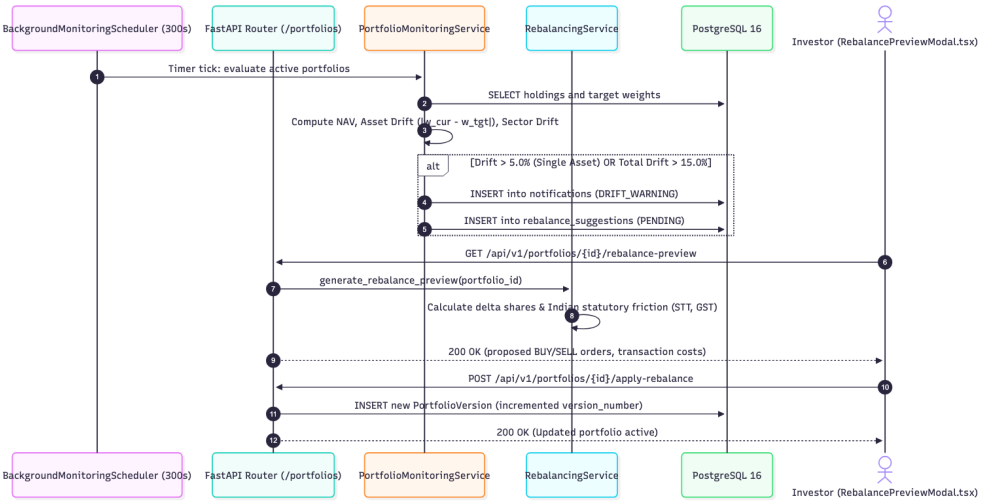


Figure 4.3: Low-Level Sequence Flow: 5-Minute Drift Radar and Friction-Aware Rebalancing Preview

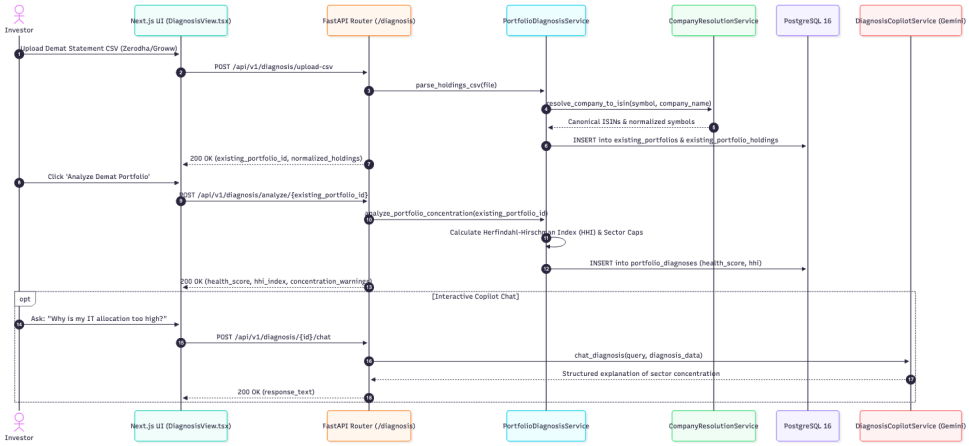


Figure 4.4: Low-Level Sequence Flow: Demat CAS PDF Statement Ingestion and Health Diagnosis

Chapter 5

Database Design and Entity-Relationship Architecture

5.1 Database Architectural Overview

The database tier serves as the authoritative, exclusive source of truth for the entire Lucid platform. Financial advisory systems, regulatory compliance frameworks, and transactional portfolio trackers demand uncompromising data integrity, transactional atomicity, and referential consistency.

Lucid employs **PostgreSQL 16** as its relational database management system. All entities, relationships, constraints, indexes, and triggers are modeled using **SQLAlchemy 2.0** declarative ORM and managed through **Alembic** schema migrations (`backend/alembic/`).

5.1.1 Database Schema Scale and Domain Breakdown

The database comprises **62 domain application tables** plus **1 Alembic schema tracking table**, yielding an exact total of **63 tables**. These tables are organized into eight functional domains:

1. **Domain 1: Identity, Authentication & Security Governance** (5 tables)
2. **Domain 2: Investor Suitability, Profiling & Financial Goals** (10 tables)
3. **Domain 3: Security Master & Corporate Registry** (8 tables)
4. **Domain 4: Market Data, Historical Quotes & Macro Regime** (8 tables)
5. **Domain 5: Portfolio Engine, Quantitative Optimization & Tracking** (10 tables)
6. **Domain 6: Broker Integration, Execution & Order Lifecycle** (5 tables)

7. **Domain 7: Demat CAS Diagnosis & Portfolio Analytics** (3 tables)

8. **Domain 8: Multi-Agent AI Telemetry, Reasoning Traces & Verification** (13 tables)

9. **Schema Tracking**: `alembic_version` (1 table)

5.2 Master Entity-Relationship Architecture

Figure 5.1 illustrates the complete master Entity-Relationship diagram representing the interconnected relationships across all functional domains:

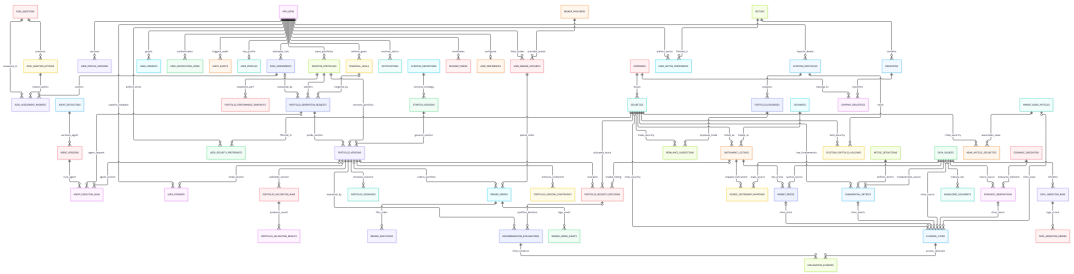


Figure 5.1: Lucid Master Database Entity-Relationship Diagram (63 Tables)

5.3 Domain-Wise Schema Specifications

5.3.1 Domain 1: Identity, Authentication and Security Governance

Domain 1 manages user authentication, enterprise cryptographic credentials, token blacklists, consent tracking, and security audit trails.

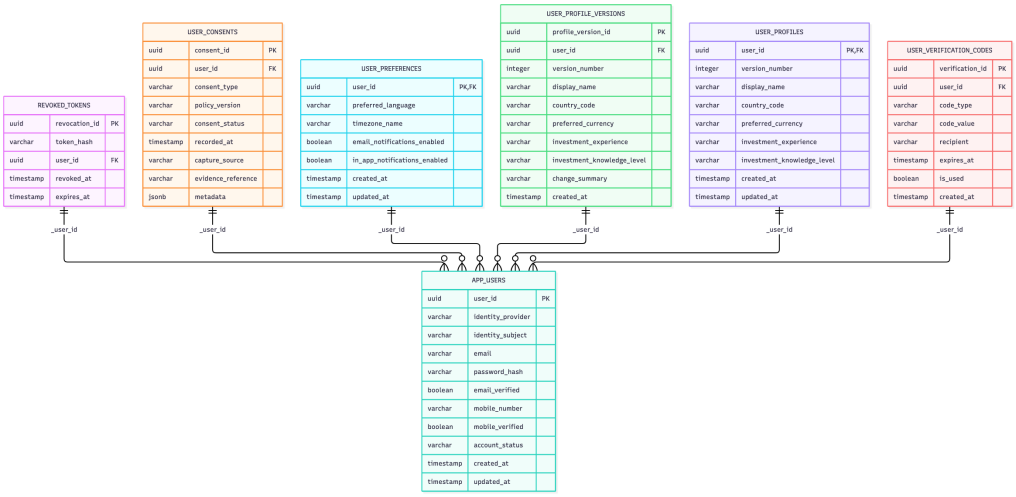


Figure 5.2: Domain 1 ER Diagram: Identity, Authentication and Security Governance

Table: `app_users`

The primary identity ledger storing registered investor accounts.

Table 5.1: Schema Specification: `app_users`

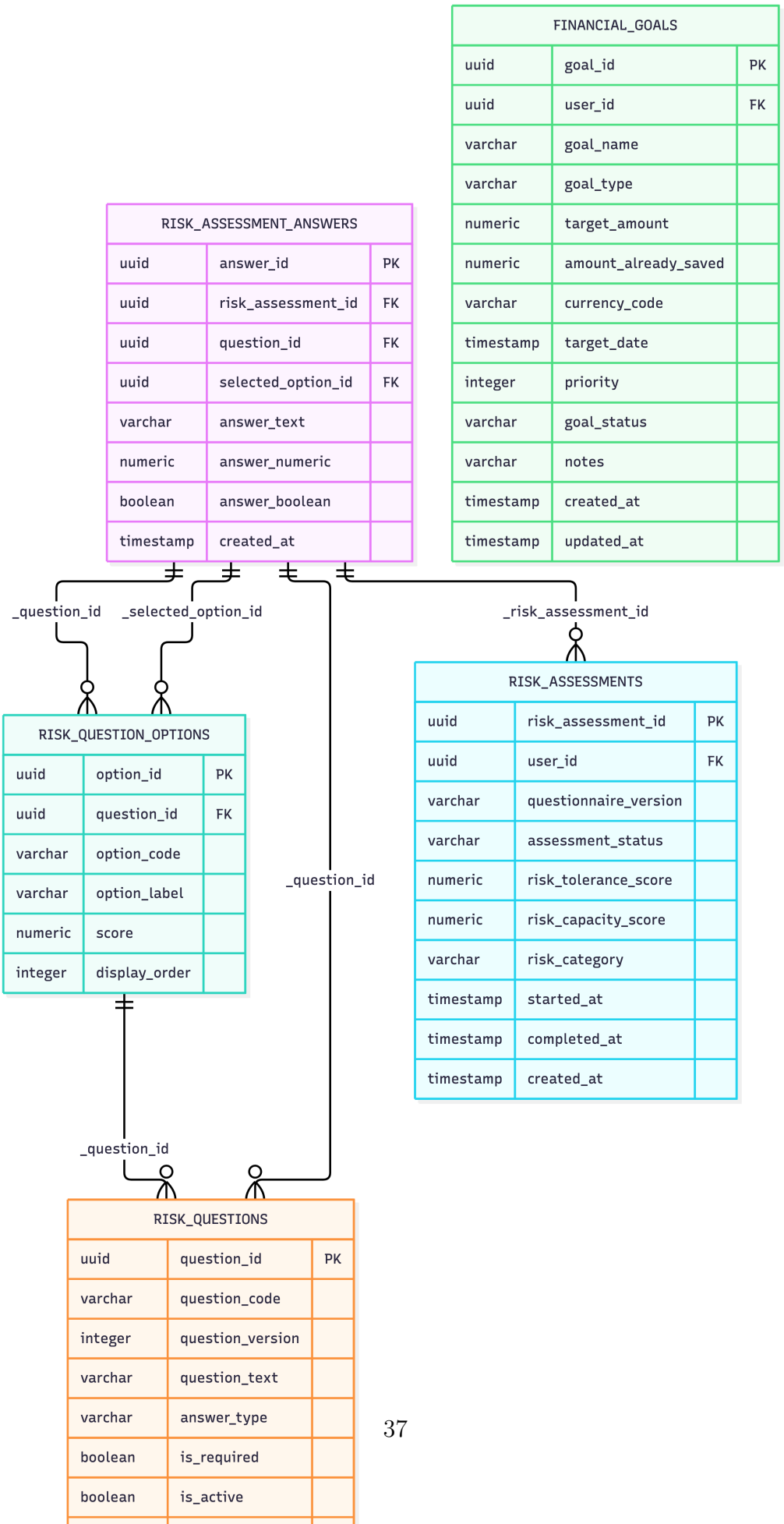
Column Name	Data Type	PK	FK	Nullable	Description & Constraints
`user_id`	`UUID`	Yes	No	No	Primary Key, generated via `uuid_generate_v4()`
`identity_provider`	`VARCHAR(50)`	No	No	No & Auth provider (`local`, `google`, `apple`), default `'local'`	
`identity_subject`	`VARCHAR(255)`	No	No	No	Unique subject identifier from auth provider
`email`	`VARCHAR(255)`	No	No	No	Unique investor email address (indexed)
`password_hash`	`VARCHAR(255)`	No	No	Yes	Canonical Argon2id cryptographic hash (RFC 9106)
`mobile_number`	`VARCHAR(20)`	No	No	Yes	Investor contact number
`email_verified`	`BOOLEAN`	No	No	No	Email verification flag (default `FALSE`)
`mobile_verified`	`BOOLEAN`	No	No	No	Mobile verification flag (default `FALSE`)
`account_status`	`VARCHAR(50)`	No	No	No	Status: `'active'`, `'suspended'`, `'pending'`, default `'active'`
`created_at`	`TIMESTAMPTZ`	No	No	No	Record creation timestamp
`updated_at`	`TIMESTAMPTZ`	No	No	No	Last update timestamp

Additional Domain 1 Tables:

- `revoked_tokens`: Stores invalidated JWT token signatures (`jti` UUID, expiry timestamp) to enforce immediate logout.
- `user_consents`: Tracks SEBI risk disclosures, terms of service, and privacy policy agreements (`consent_type`, `version`, `ip_address`).
- `user_verification_codes`: Manages time-limited OTP tokens for email and mobile phone verification (`code_hash`, `expires_at`).
- `audit_events`: Tamper-evident ledger capturing security events, login attempts, password changes, and administrative actions (`event_type`, `actor_id`, `details_json`).

5.3.2 Domain 2: Investor Suitability, Profiling and Financial Goals

Domain 2 captures the investor's financial reality, investment horizon, psychometric risk tolerance, and life goals.


FINANCIAL_GOALS
uuid goal_id PK
uuid user_id FK
varchar goal_name
varchar goal_type
numeric target_amount
numeric amount_already_saved
varchar currency_code
timestamp target_date
integer priority
varchar goal_status
varchar notes
timestamp created_at
timestamp updated_at
RISK_ASSESSMENT_ANSWERS
uuid answer_id PK
uuid risk_assessment_id FK
uuid question_id FK
uuid selected_option_id FK
varchar answer_text
numeric answer_numeric
boolean answer_boolean
timestamp created_at
_question_id
_selected_option_id
_risk_assessment_id
RISK_QUESTION_OPTIONS
uuid option_id PK
uuid question_id FK
varchar option_code
varchar option_label
numeric score
integer display_order
RISK_ASSESSMENTS
uuid risk_assessment_id PK
uuid user_id FK
varchar questionnaire_version
varchar assessment_status
numeric risk_tolerance_score
numeric risk_capacity_score
varchar risk_category
timestamp started_at
timestamp completed_at
timestamp created_at
_question_id
_question_id
RISK_QUESTIONS
uuid question_id PK
varchar question_code
integer question_version
varchar question_text
varchar answer_type
boolean is_required
boolean is_active

- `user_profiles`: Investor demographic and financial data (monthly income, investment horizon, net worth, tax bracket).

- `user_profile_versions`: Historical audit versions of investor profiles tracking changes over time.

- `financial_goals`: Explicit investor milestones (`goal_type`: wealth accumulation, retirement, child education; target corpus, target date).

- `user_preferences`: General UI, notification, and currency preferences.

- `risk_questions`: Canonical bank of SEBI suitability questions (`question_text`, `category`, `weight`).

- `risk_question_options`: Scored answer choices per question (`option_text`, `score_points`).

- `risk_assessments`: Assessment execution header (`user_id`, `total_score`, `risk_category`: conservative, moderate, aggressive).

- `risk_assessment_answers`: Investor selections mapping to specific question options.

- `user_security_preferences`: Explicit stock-level inclusion or exclusion mandates (`ticker`, `preference_type`: MANDATORY_INCLUDE, EXCLUDE).

- `user_sector_preferences`: Sector-level preferences (e.g., exclude tobacco, defense, or real estate).

5.3.3 Domain 3: Security Master and Corporate Registry

Domain 3 defines the financial market universe, listed equities, exchanges, sectors, and cross-exchange ticker mappings.

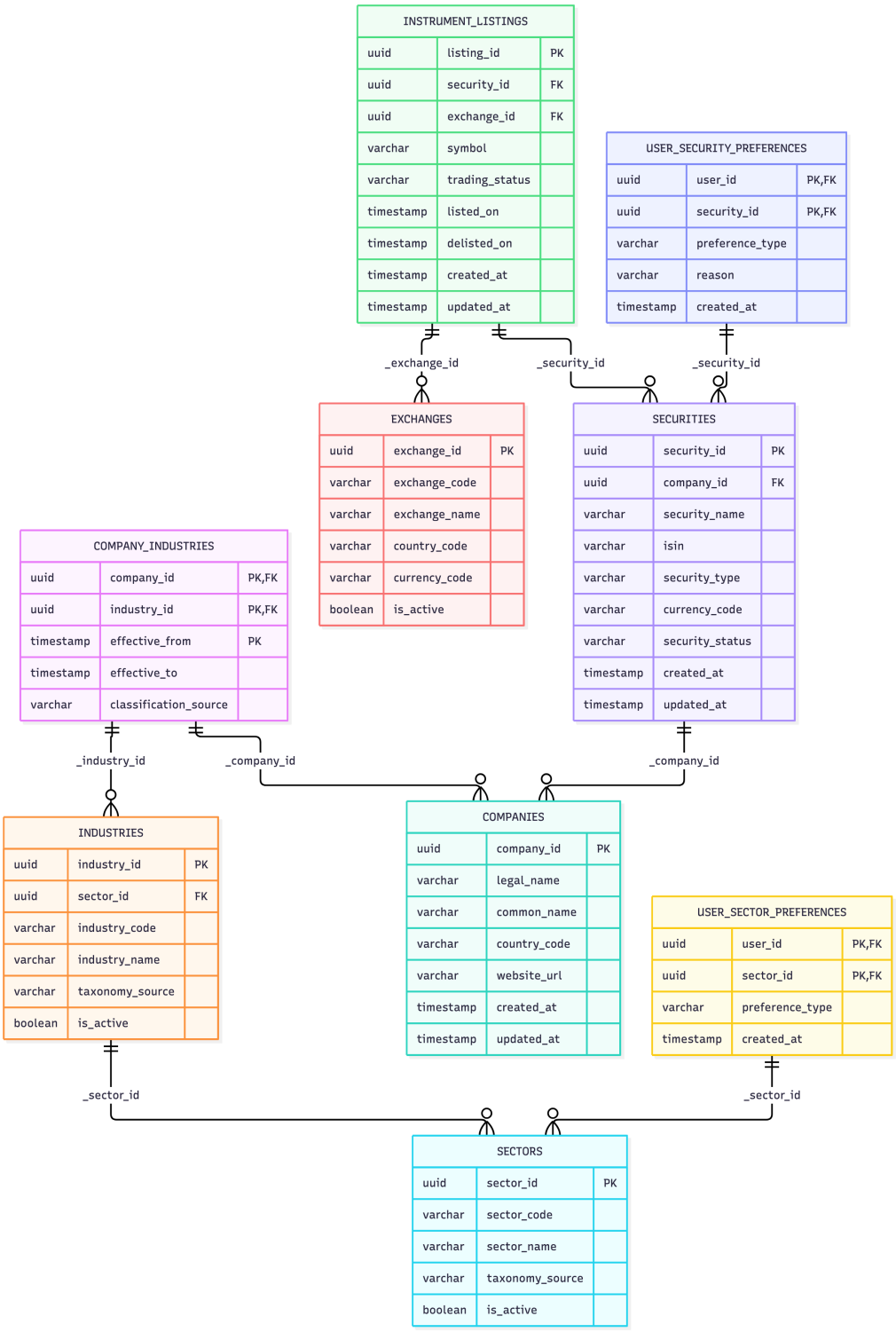


Figure 5.4: Domain 3 ER Diagram: Security Master and Corporate Registry

- `exchanges`: Exchange directory (`exchange_code`: NSE, BSE; country, operating currency).

- `sectors`: Macro-economic sectors (Financial Services, IT, Healthcare, FMCG, Energy).

- `industries`: Granular industry taxonomy mapped to parent sectors.

- `companies`: Corporate issuers (`cin_number`, `company_name`, `incorporation_date`).

- `company_industries`: Junction table enabling multi-industry corporate classification.

- `securities`: Canonical equity master (`security_id`, `isin` [Unique], `company_id`, `security_type`: EQUITY, `liquidity_tier`).

- `instrument_listings`: Exchange-specific trading listings (`security_id`, `exchange_id`, `ticker_symbol`, e.g., RELIANCE on NSE vs 500325 on BSE).

- `source_instrument_mappings`: Maps exchange tickers to external API identifiers (Yahoo Finance tickers, e.g., `RELIANCE.NS`).

5.3.4 Domain 4: Market Data, Historical Quotes and Macro Regime

Domain 4 stores historical pricing, corporate fundamentals, macroeconomic indicators, and financial news.

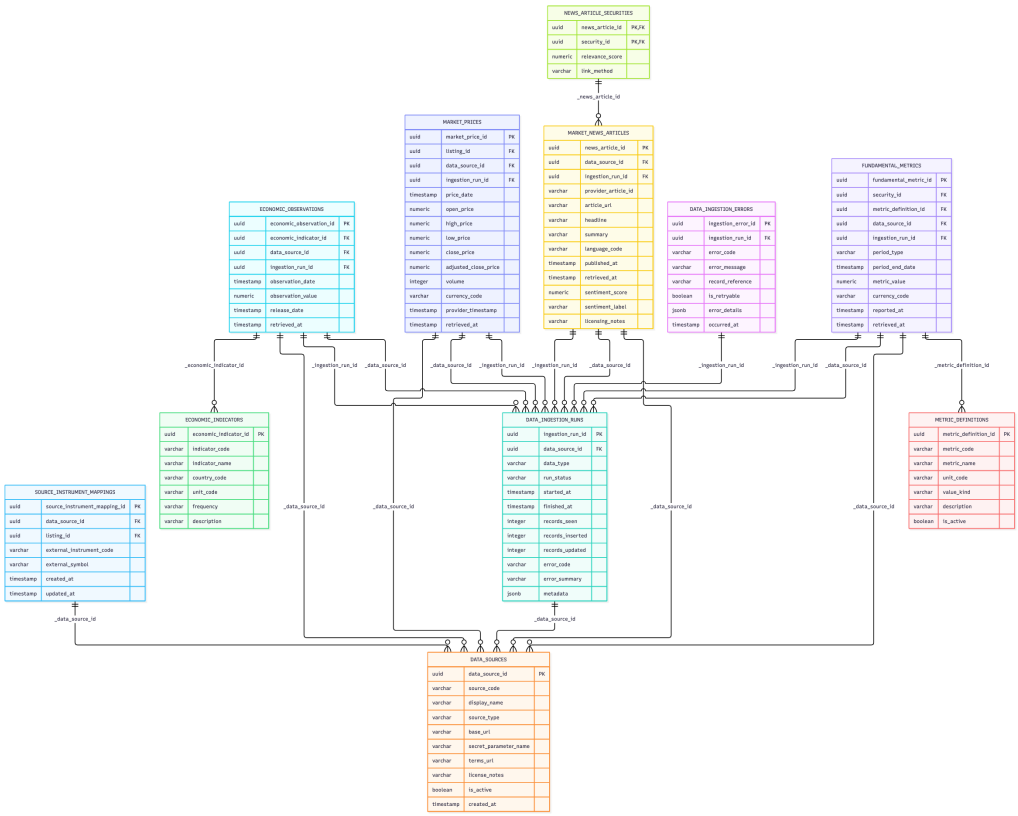


Figure 5.5: Domain 4 ER Diagram: Market Data, Historical Quotes and Macro Regime

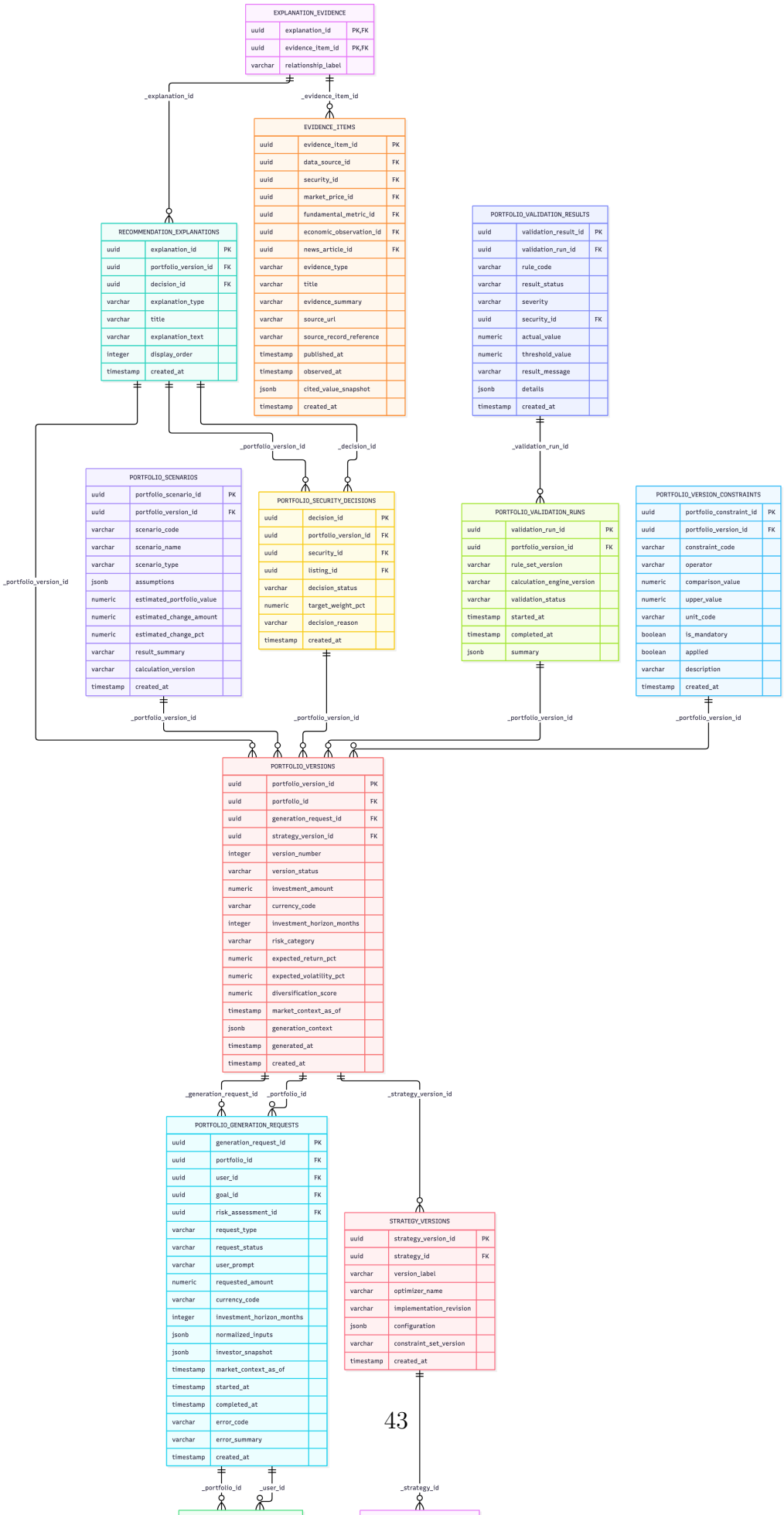
- `market_prices`: Daily historical OHLCV records (`security_id`, `price_date`, `open`, `high`, `low`, `close`, `adj_close`, `volume`).
- `fundamental_metrics`: Point-in-time balance sheet metrics (`security_id`, `period_end`, `pe_ratio`, `pb_ratio`, `roe_pct`, `debt_to_equity`, `market_cap`).
- `metric_definitions`: Metadata catalog defining calculation formulas and units for financial metrics.
- `economic_indicators`: Macroeconomic series definitions (RBI Repo Rate, CPI Inflation, India 10Y Bond Yield).
- `economic_observations`: Point-in-time observations for macroeconomic indicators.
- `market_news_articles`: Ingested financial news articles (`headline`, `content`, `published_at`, `sentiment_score`).

- `news_article_securities`: Junction table mapping articles to specific security ISINs.

- `knowledge_documents`: Regulatory circulars and investment knowledge artifacts.

5.3.5 Domain 5: Portfolio Engine, Quantitative Optimization and Tracking

Domain 5 forms the transactional core of Lucid, recording recommended portfolios, versioned weight distributions, and performance snapshots.


EXPLANATION_EVIDENCE
uuid explanation_id PK,FK
uuid evidence_item_id PK,FK
varchar relationship_label
_explanation_id
_evidence_item_id
EVIDENCE_ITEMS
uuid evidence_item_id PK
uuid data_source_id FK
uuid security_id FK
uuid market_price_id FK
uuid fundamental_metric_id FK
uuid economic_observation_id FK
uuid news_article_id FK
varchar evidence_type
varchar title
varchar evidence_summary
varchar source_url
varchar source_record_reference
timestamp published_at
timestamp observed_at
jsonb cited_value_snapshot
timestamp created_at
RECOMMENDATION_EXPLANATIONS
uuid explanation_id PK
uuid portfolio_version_id FK
uuid decision_id FK
varchar explanation_type
varchar title
varchar explanation_text
integer display_order
timestamp created_at
PORTFOLIO_VALIDATION_RESULTS
uuid validation_result_id PK
uuid validation_run_id FK
varchar rule_code
varchar result_status
varchar severity
uuid security_id FK
numeric actual_value
numeric threshold_value
varchar result_message
jsonb details
timestamp created_at
_portfolio_version_id
_decision_id
_validation_run_id
PORTFOLIO_SCENARIOS
uuid portfolio_scenario_id PK
uuid portfolio_version_id FK
varchar scenario_code
varchar scenario_name
varchar scenario_type
jsonb assumptions
numeric estimated_portfolio_value
numeric estimated_change_amount
numeric estimated_change_pct
varchar result_summary
varchar calculation_version
timestamp created_at
PORTFOLIO_SECURITY_DECISIONS
uuid decision_id PK
uuid portfolio_version_id FK
uuid security_id FK
uuid listing_id FK
varchar decision_status
numeric target_weight_pct
varchar decision_reason
timestamp created_at
PORTFOLIO_VALIDATION_RUNS
uuid validation_run_id PK
uuid portfolio_version_id FK
varchar rule_set_version
varchar calculation_engine_version
varchar validation_status
timestamp started_at
timestamp completed_at
jsonb summary
PORTFOLIO_VERSION_CONSTRAINTS
uuid portfolio_constraint_id PK
uuid portfolio_version_id FK
varchar constraint_code
varchar operator
numeric comparison_value
numeric upper_value
varchar unit_code
boolean is_mandatory
boolean applied
varchar description
timestamp created_at
_portfolio_version_id
PORTFOLIO_VERSIONS
uuid portfolio_version_id PK
uuid portfolio_id FK
uuid generation_request_id FK
uuid strategy_version_id FK
integer version_number
varchar version_status
numeric investment_amount
varchar currency_code
integer investment_horizon_months
varchar risk_category
numeric expected_return_pct
numeric expected_volatility_pct
numeric diversification_score
timestamp market_context_as_of
jsonb generation_context
timestamp generated_at
timestamp created_at
_generation_request_id
_portfolio_id
_strategy_version_id
PORTFOLIO_GENERATION_REQUESTS
uuid generation_request_id PK
uuid portfolio_id FK
uuid user_id FK
uuid goal_id FK
uuid risk_assessment_id FK
varchar request_type
varchar request_status
varchar user_prompt
numeric requested_amount
varchar currency_code
integer investment_horizon_months
jsonb normalized_inputs
jsonb investor_snapshot
timestamp market_context_as_of
timestamp started_at
timestamp completed_at
varchar error_code
varchar error_summary
timestamp created_at
STRATEGY_VERSIONS
uuid strategy_version_id PK
uuid strategy_id FK
varchar version_label
varchar optimizer_name
varchar implementation_revision
jsonb configuration
varchar constraint_set_version
timestamp created_at
_portfolio_id
_user_id
_strategy_id

- `strategy_definitions`: Core strategy templates (Conservative Income, Moderate Balanced, Aggressive Growth).

- `strategy_versions`: Versioned mathematical parameter sets per strategy.

- `investor_portfolios`: Master portfolio container per user (`portfolio_id`, `user_id`, `portfolio_status`: active, archived).

- `portfolio_versions`: Immutable version snapshots (`version_number`, `rebalance_reason`, `effective_date`, `cash_balance`).

- `portfolio_version_constraints`: Exact mathematical bounds applied to a version (max single stock 25%, max sector 40%).

- `portfolio_generation_requests`: Ingested generation telemetry capturing user input parameters.

- `portfolio_security_decisions`: Line-item security allocations (`security_id`, `target_weight_pct`, `allocated_shares`, `buy_price`, `allocated_amount`).

- `portfolio_performance_snapshots`: Daily NAV records tracking equity value and cumulative returns.

- `portfolio_scenarios`: Stress-testing projections evaluating portfolio resilience under historical shocks.

- `rebalance_suggestions`: Proposed rebalancing actions generated by the monitoring engine.

5.3.6 Domain 6: Broker Integration, Execution and Order Lifecycle

Domain 6 manages demat broker connections, order generation, and simulated trade execution.

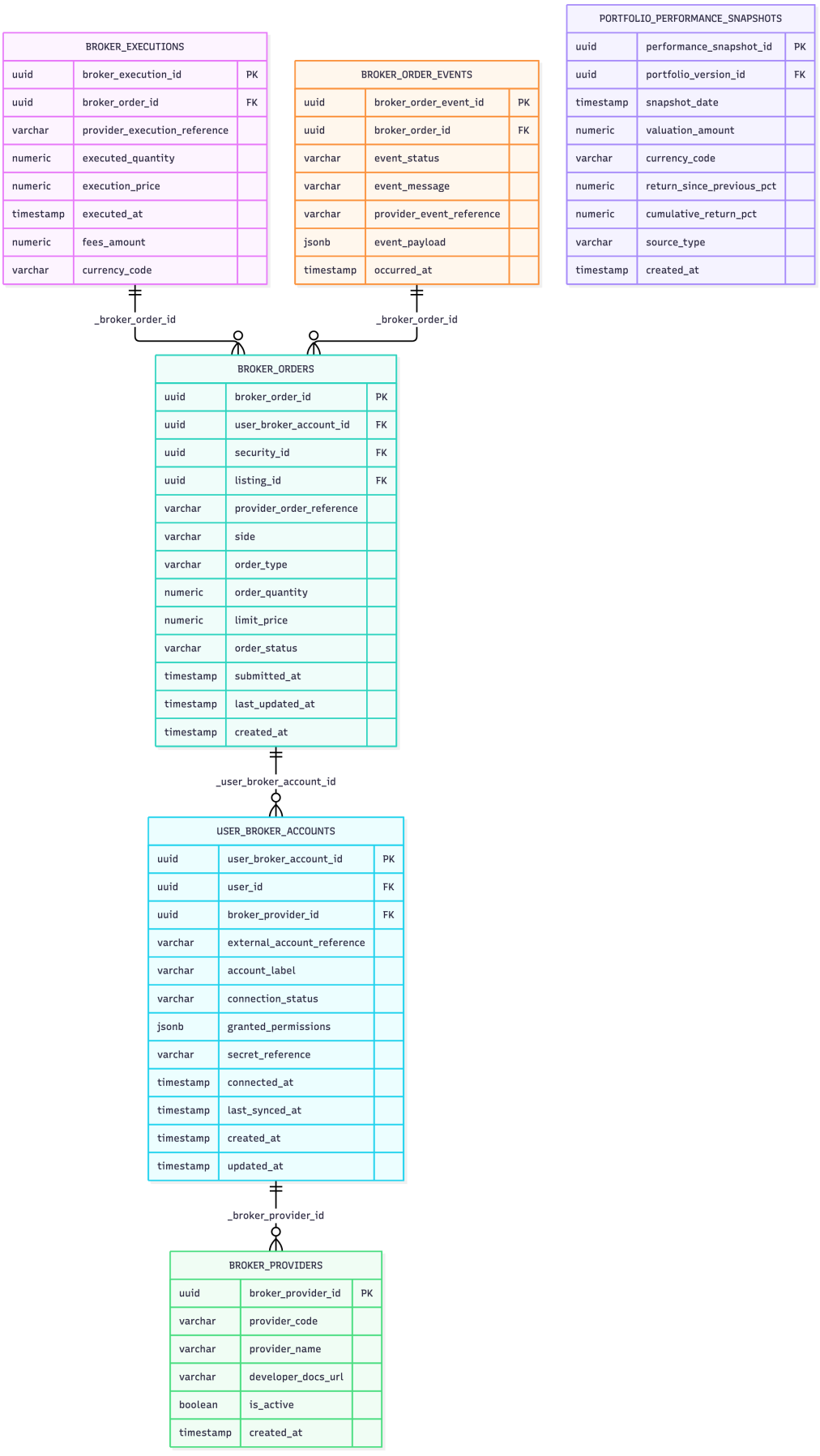


Figure 5.7: Domain 6 ER Diagram: Broker Integration, Execution and Order Lifecycle

- `broker_providers`: Supported broker platforms (Zerodha, Upstox, Groww mock adapters).

- `user_broker_accounts`: User-linked broker accounts (`account_status`, `api_token_encrypted`).

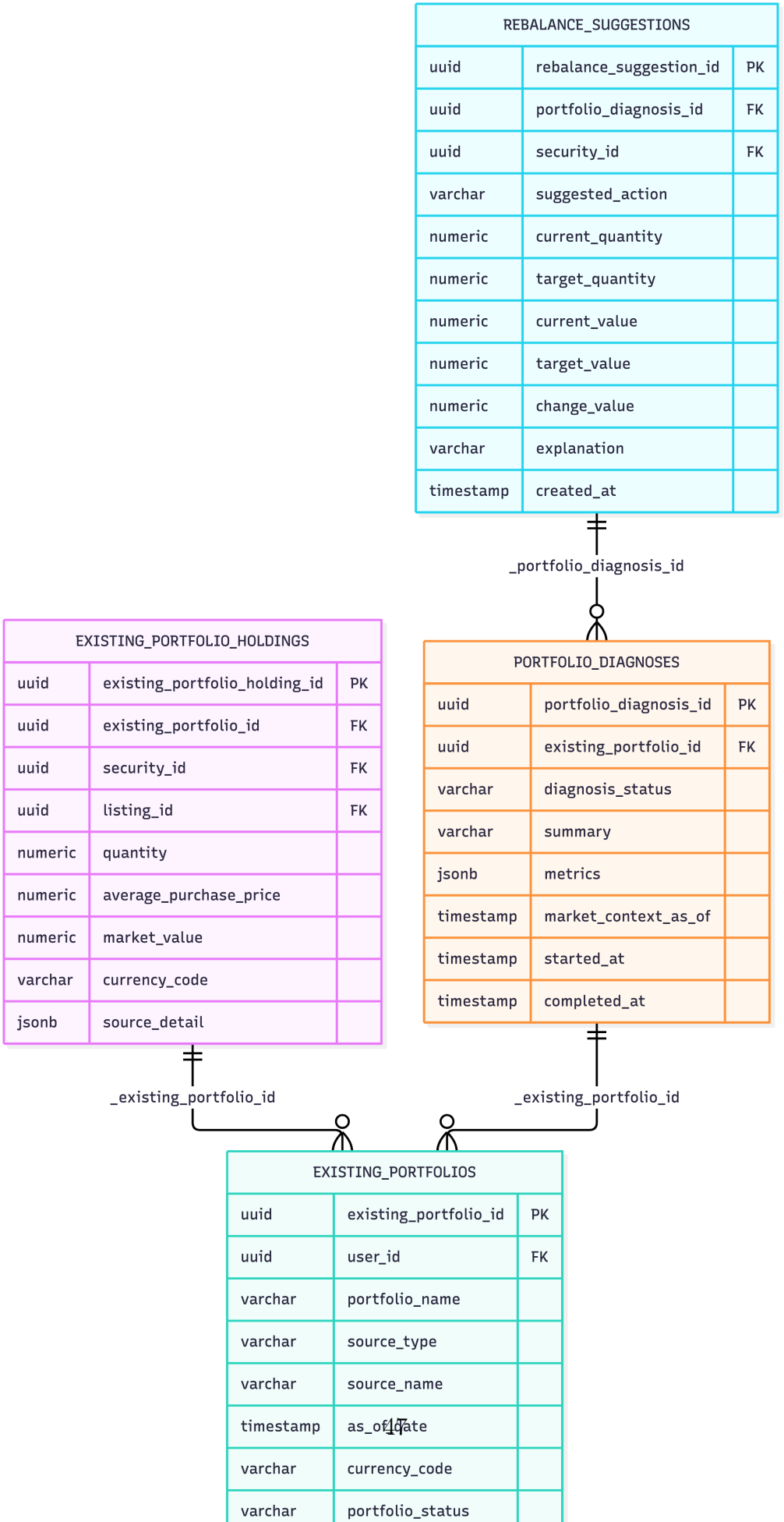
- `broker_orders`: Parent order batches generated from portfolio rebalancing.

- `broker_order_events`: Lifecycle audit events per order (CREATED, SUBMITTED, EXECUTED, REJECTED).

- `broker_executions`: Line-item execution records (`shares_filled`, `average_price`, `execution_timestamp`).

5.3.7 Domain 7: Demat CAS Diagnosis and Portfolio Analytics

Domain 7 stores imported external Demat portfolios and health diagnostic assessments.


REBALANCE_SUGGESTIONS
uuid rebalance_suggestion_id PK
uuid portfolio_diagnosis_id FK
uuid security_id FK
varchar suggested_action
numeric current_quantity
numeric target_quantity
numeric current_value
numeric target_value
numeric change_value
varchar explanation
timestamp created_at
_portfolio_diagnosis_id
EXISTING_PORTFOLIO_HOLDINGS
uuid existing_portfolio_holding_id PK
uuid existing_portfolio_id FK
uuid security_id FK
uuid listing_id FK
numeric quantity
numeric average_purchase_price
numeric market_value
varchar currency_code
jsonb source_detail
PORTFOLIO_DIAGNOSES
uuid portfolio_diagnosis_id PK
uuid existing_portfolio_id FK
varchar diagnosis_status
varchar summary
jsonb metrics
timestamp market_context_as_of
timestamp started_at
timestamp completed_at
_existing_portfolio_id
_existing_portfolio_id
EXISTING_PORTFOLIOS
uuid existing_portfolio_id PK
uuid user_id FK
varchar portfolio_name
varchar source_type
varchar source_name
timestamp as_of_date
varchar currency_code
varchar portfolio_status

- `existing_portfolios`: Imported portfolio records (`source_type`: CAS_PDF, MANUAL_ENTRY; upload timestamp).
- `existing_portfolio_holdings`: Parsed stock holdings (`isin`, `quantity`, `average_buy_price`, `current_value`).
- `portfolio_diagnoses`: Comprehensive diagnosis output (`hhi_score`, `concentration_risk_level`, `sector_skews_json`, `recommendation_summary`).

5.3.8 Domain 8: Multi-Agent AI Telemetry, Reasoning Traces and Verification

Domain 8 captures LangGraph agent executions, fiduciary explanations, constraint verification runs, and notifications.

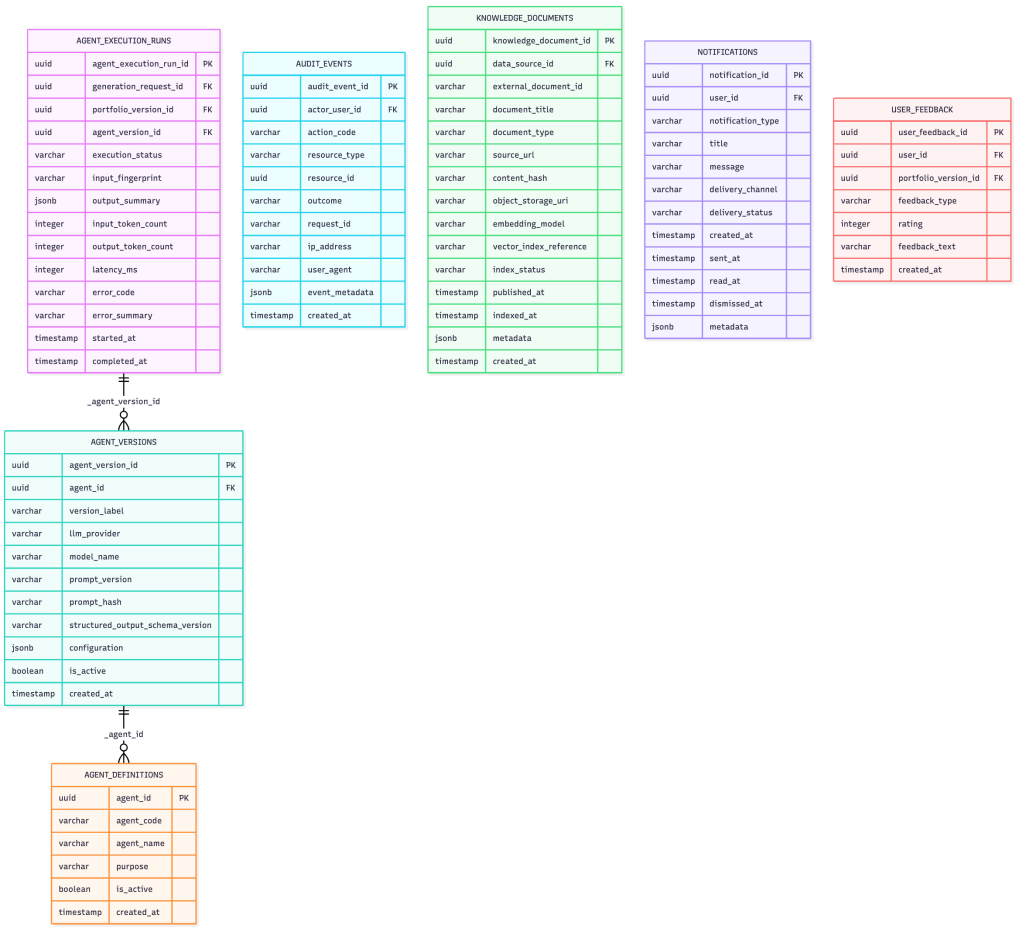


Figure 5.9: Domain 8 ER Diagram: Multi-Agent AI Telemetry, Reasoning Traces and Verification

- `agent_definitions`: Registered LangGraph agent nodes (Macro, Market, Fundamental, Critic, Explainability).
- `agent_versions`: Versioned prompts and model parameters.
- `agent_execution_runs`: Telemetry recording latency, token usage, and structured outputs per agent node.
- `recommendation_explanations`: Dual-level fiduciary explanations (`retail_explanatio` `factor_scores_json`).
- `evidence_items`: Verifiable econometric data points supporting explanations.
- `explanation_evidence`: Junction table linking explanations to specific supporting evidence items.
- `portfolio_validation_runs`: Deterministic validation audit logs.
- `portfolio_validation_results`: Line-item constraint check passes/failures.
- `data_sources`: Registry of external data feeds.
- `data_ingestion_runs`: Execution logs of automated data ingestion jobs.
- `data_ingestion_errors`: Granular error traces capturing API failures or malformed records.
- `notifications`: Real-time investor alerts (`notification_type`: DRIFT_ALERT, REBALANCE_RECOMMENDED).
- `user_feedback`: User ratings and qualitative feedback on generated explanations.

5.4 Database Design Decisions and Performance Optimization

1. **Normalization vs Performance**: The core operational ledgers (`app_users`, `investor_portfolios`, `securities`, `market_prices`) are strictly normalized to Third Normal Form (3NF) to eliminate update anomalies. Performance snapshots and diagnostic reports utilize denormalized JSONB

columns (`sector_skews_json`, `factor_scores_json`) to facilitate sub-5ms retrieval for frontend dashboard rendering.

2. **UUID Primary Keys**: All user, portfolio, and transaction records utilize RFC 4122 Version 4 UUIDs (`UUID`). This prevents sequential enumeration attacks and facilitates distributed microservice scaling.

3. **Indexing Strategy**: High-throughput search columns (`securities.isin`, `instrument_listings.ticker_symbol`, `market_prices.(security_id, price_date)`) are supported by B-Tree composite indexes. Text search for stock screening leverages PostgreSQL GIN (Generalized Inverted Index) trgm indexes.

4. **Referential Integrity and Cascades**: Strong foreign key constraints safeguard financial ledgers. Deleting a portfolio version automatically cascades to dependent allocations (`portfolio_security_decisions`), while master security and user records enforce `ON DELETE RESTRICT` to prevent accidental loss of historical transaction ledgers.

Chapter 6

System Architecture and Runtime Topology

6.1 System Architecture Overview

The system architecture of Lucid is engineered as an eight-layer decoupled topology, balancing computational performance, mathematical reliability, and modular extensibility. The platform is designed to operate seamlessly across development, local containerized, and institutional cloud deployment environments.

Figure 6.1 illustrates the complete architectural topology spanning client presentation, security middleware, multi-agent orchestration, quantitative mathematical solvers, and relational persistence.

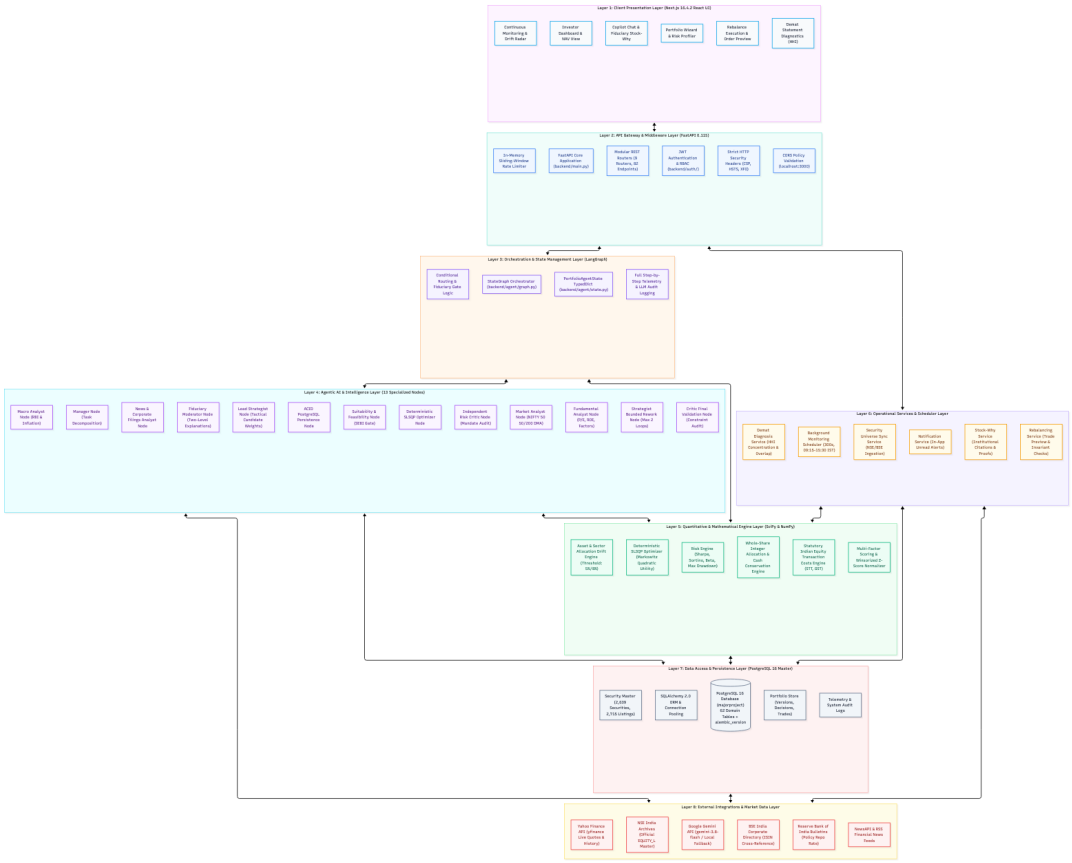


Figure 6.1: Lucid Eight-Layer Decoupled System Architecture

6.2 The Eight Architectural Layers

1. **Layer 1: Client Presentation Layer**: Implemented using Next.js 16.4.2 and React 19.3.0. Employs modern Server and Client Components, responsive Tailwind CSS v4 styling, and Lucide React iconography. Provides an interactive investor experience across mobile, tablet, and desktop viewports.

2. **Layer 2: API Gateway and Security Middleware Layer**: Implemented using FastAPI 0.115+ on the Uvicorn ASGI server. Enforces CORS policies, JWT bearer token validation (HS256), in-memory sliding-window rate limiting, and enterprise HTTP security headers.

3. **Layer 3: Orchestration and State Management Layer**: Powered by LangGraph 0.4. Manages the cyclical state graph, tracking typed state dictionaries (`TypedDict AgentState`), handling retries, and enforcing structured JSON validation.

4. **Layer 4: Agentic AI and Intelligence Layer**: Houses the specialized cognitive agent nodes. Integrates Google Gemini 3.8 Flash (`gemini-3.8-flash`) via LangChain Google GenAI, backed by a deterministic rule-based model fallback engine.

5. **Layer 5: Quantitative and Mathematical Engine Layer**: Built on SciPy 1.15.2 and NumPy 2.2.3. Executes constrained Markowitz mean-variance SLSQP optimization, discrete whole-share lot rounding, and statutory Indian transaction friction modeling.

6. **Layer 6: Operational Services and Background Scheduler Layer**: Implements asynchronous background tasks, including the 5-minute market-hours drift radar (`BackgroundMonitoringScheduler`), CAS statement parsers (`pdfplumber`), and real-time notification dispatchers.

7. **Layer 7: Data Access and Persistence Layer**: Powered by PostgreSQL 16 and SQLAlchemy 2.0.38 ORM with Alembic schema migration tracking. Houses 62 domain application tables (+1 alembic tracking table) with connection pooling (10 base connections, 20 max overflow).

8. **Layer 8: External Integrations and Market Data Feeds Layer**: Manages data ingestion from NSE archives (`EQUITY_L.csv`), BSE scrip master, Yahoo Finance API (`yfinance`), RBI macroeconomic releases, and Google Gemini APIs.

6.3 Technology Stack Manifest

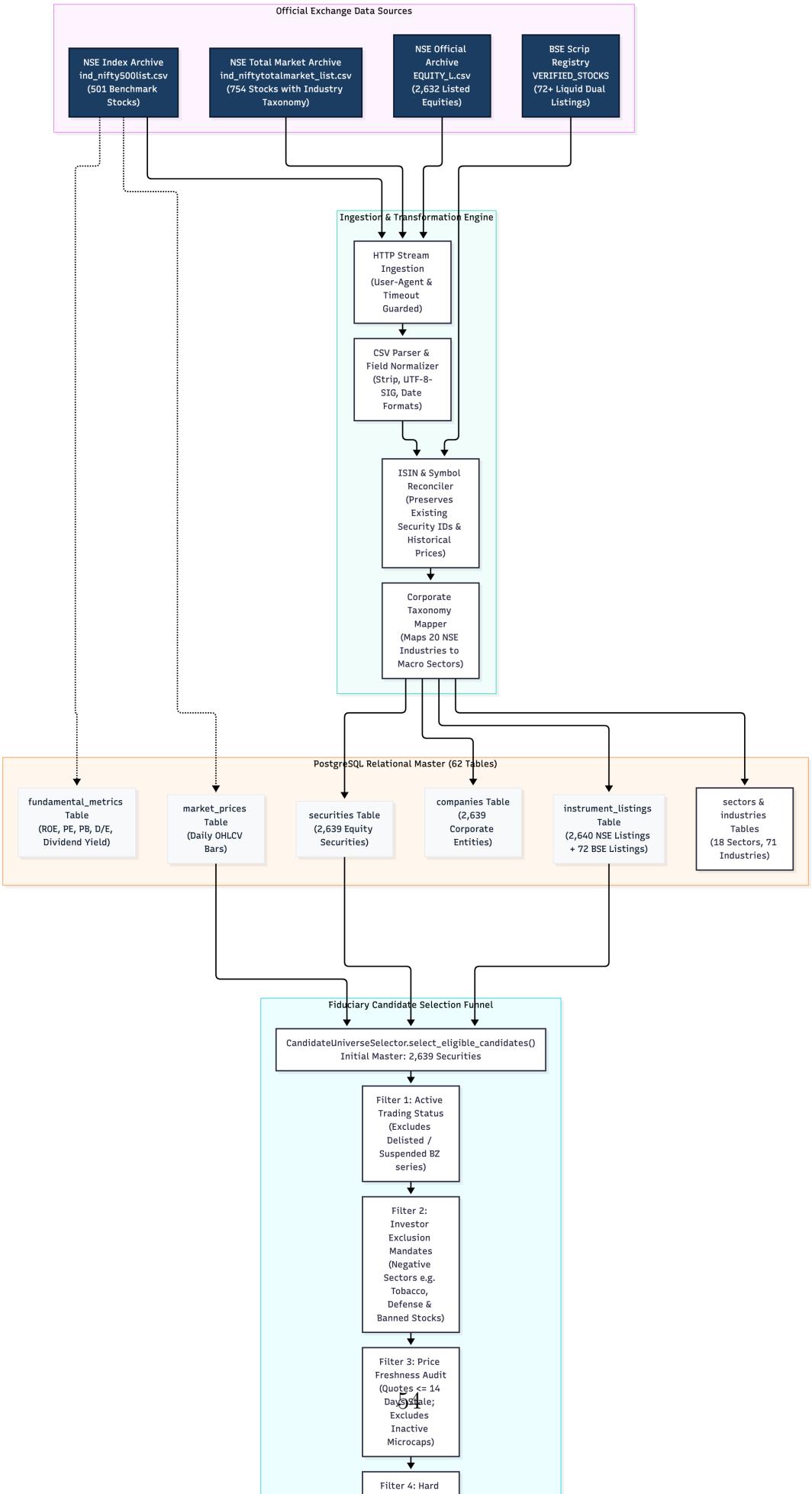
Table 6.1 documents the verified technology stack actively utilized by Lucid, detailing the specific version, purpose, and technical justification for each component.

Table 6.1: Lucid Active Technology Stack Manifest (Verified Versions)

Technology / Library	Verified Version	Purpose in Lucid	Technical Justification
Next.js	16.4.2	Frontend Web Framework	App Router architecture, React Server Components, fast hydration
React	19.3.0	UI Component Library	Concurrent rendering, server actions, dynamic client UI state
TypeScript	5.8.2	Frontend Programming Language	Static type safety, strict interface contracts with backend schemas
Tailwind CSS	4.1.1	Styling	Utility-first CSS, high rendering performance, zero runtime overhead
Python	3.11 – 3.14	Backend Programming Language	Standard for quantitative finance, scientific computing, and AI
FastAPI	0.115.8	Backend REST API Framework	Asynchronous ASGI architecture, automatic OpenAPI docs, high RPS
Uvicorn	0.34.0	ASGI Web Server	Low-latency async event-loop handling for FastAPI routes
Pydantic	2.10.6	Request/Response Validation	High-speed Rust-backed data validation and schema enforcement
PostgreSQL	16+	Master Relational Database	ACID transactions, relational integrity, JSONB support, advisory locks
SQLAlchemy	2.0.38	Object-Relational Mapper (ORM)	Declarative 2.0 syntax, connection pooling, typed SQL expressions
Alembic	1.14.1	Database Migration Manager	Version-controlled schema evolutions, rollback support
SciPy	1.15.2	Scientific Computing	Sequential Least Squares Programming (SLSQP) numerical solver
NumPy	2.2.3	Vectorized Numerical Operations	High-performance covariance matrix calculations and array slicing
Pandas	2.2.3	Financial Data Analysis	Time-series price lookback handling and CSV universe ingestion
LangGraph	0.4.0	Multi-Agent Orchestration	Cyclical state graphs, typed state management, conditional branching
Google Gemini	gemini-3.8-flash	Large Language Model	Sub-second financial document synthesis and fiduciary explanation
Argon2-cffi	25.1.0	Password Hashing	Memory-hard cryptographic hashing complying with RFC 9106
PyJWT	2.10.1	Token Authentication	Stateless, digitally signed JSON Web Tokens for API sessions
yfinance	0.2.54	Market Data Feed	1-year daily historical OHLCV quotes and company fundamental ratios
pdfplumber	0.11.5	Statement Parsing	Precise text and table extraction from CDSL/NSDL Demat CAS PDFs
Bash / PowerShell	Native	Local Lifecycle Scripts	One-command automated installation, status, and service shutdown

6.4 Data Pipeline and Ingestion Architecture

Figure 6.2 illustrates the data ingestion and transformation architecture responsible for synchronizing the Indian exchange universe:


Official Exchange Data Sources
NSE Index Archive ind_nifty500list.csv (501 Benchmark Stocks)
NSE Total Market Archive ind_niftytotalmarket_list.csv (754 Stocks with Industry Taxonomy)
NSE Official Archive EQUITY_L.csv (2,632 Listed Equities)
BSE Scrip Registry VERIFIED_STOCKS (72+ Liquid Dual Listings)
Ingestion & Transformation Engine
HTTP Stream Ingestion (User-Agent & Timeout Guarded)
CSV Parser & Field Normalizer (Strip, UTF-8-SIG, Date Formats)
ISIN & Symbol Reconciler (Preserves Existing Security IDs & Historical Prices)
Corporate Taxonomy Mapper (Maps 20 NSE Industries to Macro Sectors)
PostgreSQL Relational Master (62 Tables)
fundamental_metrics Table (ROE, PE, PB, D/E, Dividend Yield)
market_prices Table (Daily OHLCV Bars)
securities Table (2,639 Equity Securities)
companies Table (2,639 Corporate Entities)
instrument_listings Table (2,640 NSE Listings + 72 BSE Listings)
sectors & industries Tables (18 Sectors, 71 Industries)
Fiduciary Candidate Selection Funnel
CandidateUniverseSelector.select_eligible_candidates() Initial Master: 2,639 Securities
Filter 1: Active Trading Status (Excludes Delisted / Suspended BZ series)
Filter 2: Investor Exclusion Mandates (Negative Sectors e.g. Tobacco, Defense & Banned Stocks)
Filter 3: Price Freshness Audit (Quotes <= 14 Days Stale; Excludes Inactive Microcaps)
Filter 4: Hard

The pipeline executes through four sequential phases:

1. **Universe Ingestion**: The synchronization service (`sync_full_exchange_universe()`) downloads the official NSE equity master (`EQUITY_L.csv`) and queries the BSE scrip directory. It resolves ISIN collisions resulting from historical corporate splits (e.g., BAJFINANCE, DRREDDY, HAL) and persists **2,639 canonical equities** into `securities`.

2. **Historical Price Retrieval**: A 1-year historical lookback window (252 trading sessions) is retrieved via Yahoo Finance for each security, validated for trading day continuity, and stored in `market_prices`.

3. **Fundamental Metric Normalization**: Balance sheet ratios (P/E, P/B, ROE, Debt-to-Equity, Operating Margin, Market Cap) are extracted and recorded in `fundamental_metrics`.

4. **Liquidity Tier Categorization**: Securities are partitioned into four institutional tiers based on Average Daily Traded Volume (ADTV) and daily turnover, establishing a liquid recommendation candidate pool of **504 equities**.

6.5 Runtime Execution Flow: Portfolio Generation Request

When an investor clicks "Generate Portfolio" on the web interface, the following execution lifecycle occurs across the architecture:

1. **Input Validation**: FastAPI validates incoming JSON against `PortfolioGenerationRe` The user's JWT token is authenticated, and the rate limiter checks available tokens.

2. **Investor Profile & Goal Resolution**: The service fetches the investor's active risk category (Conservative, Moderate, Aggressive) and investment capital from `user_profiles` and `financial_goals`.

3. **Universe Filtering**: The `CandidateUniverseSelector` extracts eligible liquid stocks from `securities`, removing user-specified sector exclusions (e.g., tobacco, fossil fuels).

4. **LangGraph State Initialization**: The backend instantiates an `AgentState` dictionary and launches the 13-node LangGraph StateGraph.

5. **Macro & Regime Analysis**: Nodes assess RBI repo rates and India VIX volatility, setting risk aversion coefficients.

6. **Fundamental & Sentiment Screening**: Balance sheet criteria and news sentiment are scored for each candidate stock.

7. **Adversarial Critique**: The Critic node challenges candidate allocations, pruning high-risk or over-concentrated positions.

8. **SciPy SLSQP Optimization**: The quantitative engine executes numerical optimization in an external worker threadpool (`anyio.to_thread.run_sync`), finding the optimal weight vector $\mathbf{w}^*$.

9. **Integer Lot Sizing**: The lot rounder converts $\mathbf{w}^*$ into discrete share counts and calculates residual cash.

10. **Database Persistence & API Response**: The versioned portfolio is persisted across `portfolio_versions` and `portfolio_security_decisions`. The API returns HTTP 201 Created with the full portfolio payload in under 2.5 seconds.

6.6 Security and Privacy Architecture

- **Canonical Password Hashing**: Passwords are hashed using the **Argon2id** algorithm (`$argon2id$v=19$m=65536,t=3,p=4`), utilizing 64 MB of RAM and 3 iterations to prevent GPU-based brute-force attacks.

- **Token Security & Revocation**: JWT access tokens have a 60-minute lifespan. Logged-out token signatures are immediately stored in the `revoked_tokens` table; any subsequent request with a blacklisted signature is rejected with HTTP 401 Unauthorized.

- **In-Memory Rate Limiting**: The sliding-window rate limiter enforces strict request ceilings on sensitive endpoints: 20 requests/minute for login, 10 requests/minute for registration, and 20 requests/minute for portfolio generation.

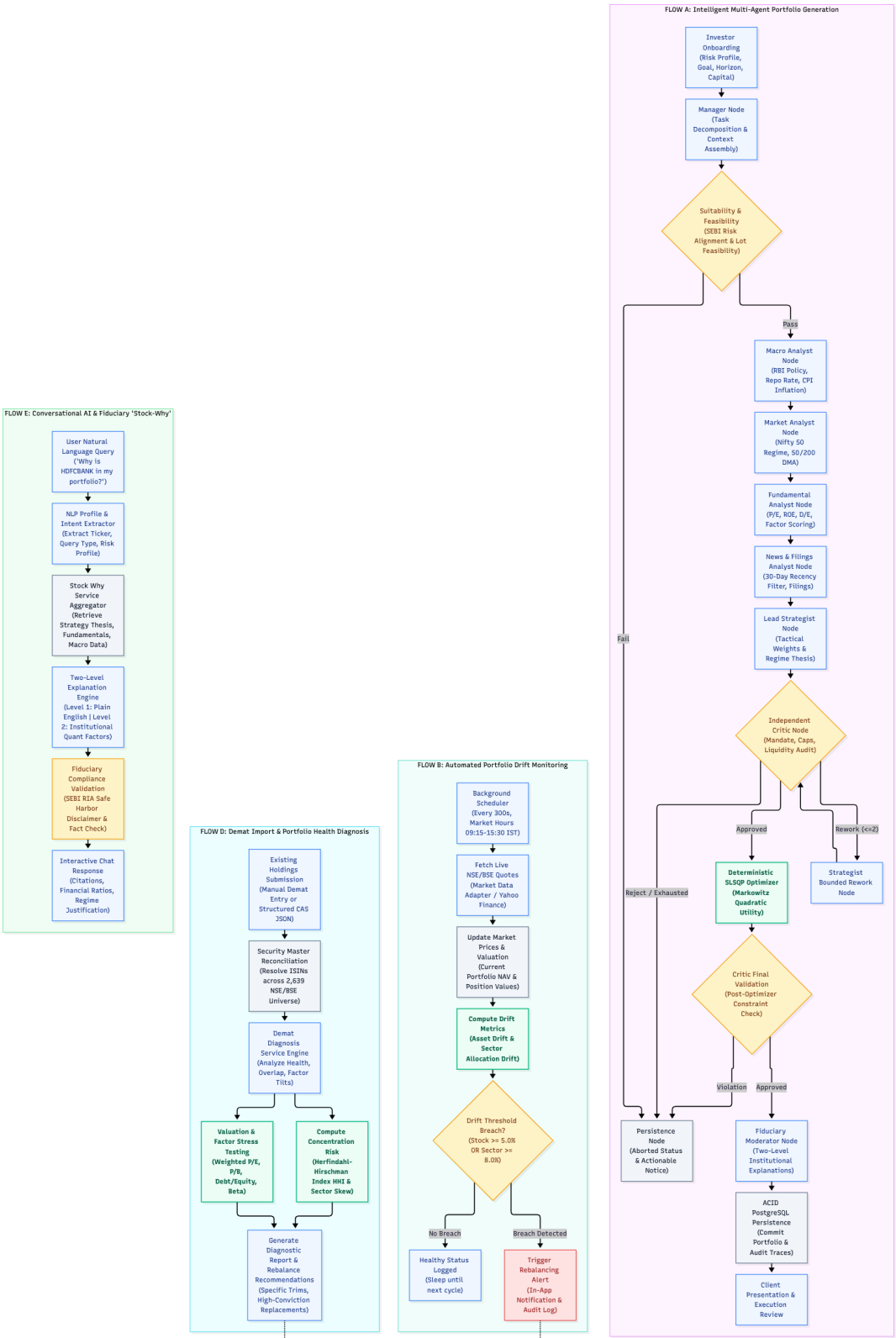
- **Enterprise HTTP Security Headers**: All HTTP responses enforce strict transport security, prevent MIME-type sniffing (`nosniff`), block clickjacking (`X-Frame-Options: DENY`), and restrict cross-site scripting.

Chapter 7

End-to-End Application Methodology

7.1 Methodology Overview

The Lucid platform executes a sixteen-step investment engineering lifecycle designed to deliver personalized, mathematically rigorous, and legally executable equity portfolios. The process harmonizes human-readable qualitative AI reasoning with deterministic numerical optimization, as shown in Figure 7.1:


FLOW A: Intelligent Multi-Agent Portfolio Generation
Investor Onboarding (Risk Profile, Goal, Horizon, Capital)
Manager Node (Task Decomposition & Context Assembly)
Suitability & Feasibility (SEBI Risk Alignment & Lot Feasibility)
Fail
Pass
Macro Analyst Node (RBI Policy, Repo Rate, CPI Inflation)
Market Analyst Node (Nifty 50 Regime, 50/200 DMA)
Fundamental Analyst Node (P/E, ROE, D/E, Factor Scoring)
News & Filings Analyst Node (30-Day Recency Filter, Filings)
Lead Strategist Node (Tactical Weights & Regime Thesis)
Independent Critic Node (Mandate, Caps, Liquidity Audit)
Approved
Rework (<=2)
Reject / Exhausted
Deterministic SLSQP Optimizer (Markowitz Quadratic utility)
Strategist Bounded Rework Node
Critic Final Validation (Post-Optimizer Constraint Check)
Violation
Approved
Persistence Node (Aborted Status & Actionable Notice)
Fiduciary Moderator Node (Two-Level Institutional Explanations)
ACID PostgreSQL Persistence (Commit Portfolio & Audit Traces)
Client Presentation & Execution Review
FLOW B: Automated Portfolio Drift Monitoring
Background Scheduler (Every 300s, Market Hours 09:15-15:30 IST)
Fetch Live NSE/BSE Quotes (Market Data Adapter / Yahoo Finance)
Update Market Prices & Valuation (Current Portfolio NAV & Position Values)
Compute Drift Metrics (Asset Drift & Sector Allocation Drift)
Drift Threshold Breach? (Stock >= 5.0% OR Sector >= 8.0%)
No Breach
Breach Detected
Healthy Status Logged (Sleep until next cycle)
Trigger Rebalancing Alert (In-App Notification & Audit Log)
FLOW D: Demat Import & Portfolio Health Diagnosis
Existing Holdings Submission (Manual Demat Entry or Structured CAS JSON)
Security Master Reconciliation (Resolve ISINs across 2,639 NSE/BSE Universe)
Demat Diagnosis Service Engine (Analyze Health, Overlap, Factor Tilts)
Valuation & Factor Stress Testing (Weighted P/E, P/B, Debt/Equity, Beta)
Compute Concentration Risk (Herfindahl-Hirschman Index HHI & Sector Skew)
Generate Diagnostic Report & Rebalance Recommendations (Specific Trims, High-Conviction Replacements)
FLOW E: Conversational AI & Fiduciary 'Stock-Why'
User Natural Language Query ('Why is HDFCBANK in my portfolio?')
NLP Profile & Intent Extractor (Extract Ticker, Query Type, Risk Profile)
Stock Why Service Aggregator (Retrieve Strategy Thesis, Fundamentals, Macro Data)
Two-Level Explanation Engine (Level 1: Plain English | Level 2: Institutional Quant Factors)
Fiduciary Compliance Validation (SEBI RIA Safe Harbor Disclaimer & Fact Check)
Interactive Chat Response (Citations, Financial Ratios, Regime Justification)
FLOW C: Unified Portfolio Rebalancing Engine
Rebalance Triggered (Drift Breach OR User Rebalance Request)
Retrieve Target Allocation (Active Strategy Definition & Cap Rules)
Compute Required Delta (Target Capital Value - Current Market Value)
Whole-Share Integer Allocation (Math Floor

7.2 Investor Onboarding and Suitability Profiling

Investor profiling implements SEBI's regulatory suitability requirements [9]. The investor completes a structured psychometric assessment comprising five questions evaluating:

1. **Financial Knowledge and Market Experience**: Categorized from novice to experienced derivative trader.

2. **Investment Time Horizon**: Short-term (< 1 year), Medium-term (1–5 years), Long-term (> 5 years).

3. **Loss Tolerance and Psychological Resilience**: Measures reaction to an unrealized 20% portfolio drawdown.

4. **Capital Allocation Objective**: Capital preservation vs balanced growth vs aggressive wealth accumulation.

5. **Income Stability and Liquidity Needs**: Regular salary vs business income, dependency ratio.

Each response is assigned an integer point weight between 1 and 4. The total suitability score S is calculated as:

$$S = \sum_{j=1}^{5} \omega_j \cdot x_j \tag{7.1}$$

The score S maps the investor into an operational risk category:

$$\text{Risk Category} = \begin{cases} \textbf{Conservative}, \&\text{if } S \in [5, 9] \\ \textbf{Moderate}, \&\text{if } S \in [10, 15] \\ \textbf{Aggressive}, \&\text{if } S \in [16, 20] \end{cases} \tag{7.2}$$

7.3 Natural Language Intent and Constraint Parsing

For investors who prefer conversational interaction, Lucid implements an NLP entity extraction module (`backend/services/nlp_profile_extractor.py`). Using Google Gemini with structured JSON output enforcement, the parser extracts investment parameters from unstructured natural language:

- *User Input*: "I have 2.5 lakhs to invest for my daughter's college fund in 10 years. I want moderate risk but please don't buy any tobacco or liquor companies."

- *Extracted Entities*:

  - Capital: Rs. $2,50,000$ (parsed from "2.5 lakhs")
  - Time Horizon: 10 years
  - Goal: Education
  - Inferred Risk: Moderate
  - Sector Exclusions: `FMCG_TOBACCO`, `BEVERAGES_ALCOHOL`

7.4 Market Regime and Volatility Classification

To adapt portfolio risk aversion to prevailing macroeconomic conditions, Lucid calculates an automated Market Regime Score:

1. **Technical Trend of Benchmark (Nifty 50)**:

$$\text{Trend} = \begin{cases} \text{Bullish}, \&\text{if } P_{\text{Nifty}} > \text{SMA}_{50} > \text{SMA}_{200} \\ \text{Neutral}, \&\text{if } \text{SMA}_{50} \geq P_{\text{Nifty}} \geq \text{SMA}_{200} \\ \text{Bearish}, \&\text{if } P_{\text{Nifty}} < \text{SMA}_{200} \end{cases} \tag{7.3}$$

2. **Volatility Regime via India VIX**:

$$\text{Volatility Tier} = \begin{cases} \text{Low Volatility}, \&\text{if VIX} < 13.0 \\ \text{Normal Volatility}, \&\text{if } 13.0 \leq \text{VIX} \leq 18.0 \\ \text{Elevated Volatility}, \&\text{if } 18.0 < \text{VIX} \leq 24.0 \\ \text{Extreme Volatility}, \&\text{if VIX} > 24.0 \end{cases} \tag{7.4}$$

During elevated or extreme volatility regimes, the platform increases the quantitative risk aversion multiplier λ, dynamically shifting allocations toward defensive, low-beta large-cap equities.

7.5 Stock Universe Discovery and Liquidity Tiering

Lucid maintains the complete catalog of **2,639 listed equities** across the NSE and BSE. To protect retail investors from illiquid securities and lower-circuit traps, the candidate selection module (`CandidateUniverseSelector`) partitions the universe into four institutional Liquidity Tiers:

Table 7.1: Lucid Stock Universe Liquidity Tier Classification

Liquidity Tier	Market Cap / Index Segment	Min ADTV (Shares)	Min Daily Turnover (Rs.)	Recommendation Eligible
Tier 1: Ultra Liquid	Nifty 50, Nifty Next 50	$> 1,00,000$	$>$ Rs. $1,00,00,000$	**Yes (100% Eligible)**
Tier 2: Broad Liquid	Nifty Midcap 150	$> 50,000$	$>$ Rs. $50,00,000$	**Yes (100% Eligible)**
Tier 3: Emerging Liquid	Nifty Smallcap 250 (Top Half)	$> 10,000$	$>$ Rs. $25,00,000$	**Yes (Filtered)**
Tier 4: Restricted	Micro-caps, ESM/SME Equities	$< 10,000$	$<$ Rs. $25,00,000$	**No (Search/Diagnosis Only)**

Of the 2,639 stored securities, exactly **504 liquid equities** qualify as the active recommendation universe. The remaining 2,135 securities are excluded from recommendations to protect capital, but remain fully searchable and diagnosable when users import existing Demat holdings.

7.6 Fundamental Financial Metric and Factor Extraction

Candidate securities are evaluated across six core fundamental factors retrieved from exchange filings and market data feeds:

1. **Return on Equity (ROE)**: $\text{ROE} = \frac{\text{Net Income}}{\text{Shareholder's Equity}} \geq 15.0\%$
2. **Price-to-Earnings Ratio (P/E)**: Benchmarked against the industry median.
3. **Debt-to-Equity Ratio (D/E)**: $\text{D/E} \leq 1.5$ (Financial institutions excluded).
4. **Operating Cash Flow to EBITDA**: Ensures earnings quality and dividend coverage.
5. **Historical Price Volatility (σ_i)**: 252-day annualized standard deviation of daily logarithmic returns.
6. **Systematic Beta (β_i)**: Covariance of stock returns with the Nifty 50 benchmark divided by benchmark variance.

7.7 Multi-Agent Reasoning: LangGraph StateGraph

The 13-node LangGraph StateGraph evaluates qualitative and contextual factors. The workflow coordinates:

- **Macro & Fundamental Analysts**: Score balance sheet health, competitive moats, and monetary policy impacts.
- **Adversarial Critic Node**: Cross-examines top candidates, challenging high valuations or regulatory scrutiny.
- **Output**: Emits a screened, ranked basket of 15–25 securities with assigned qualitative conviction weights to initialize the mathematical solver.

7.8 Mathematical Portfolio Optimization (SciPy SLSQP)

Numerical optimization is strictly decoupled from the generative AI models. Lucid utilizes SciPy's Sequential Least Squares Programming (SLSQP) solver (`scipy.optimize.minimize`) to compute optimal weights $\mathbf{w}^* = [w_1^*, w_2^*, \ldots, w_K]^T$.

7.8.1 Objective Function

Depending on the investor's risk profile, Lucid solves one of two formulations:

1. **For Moderate and Aggressive Profiles (Sharpe Ratio Maximization)**:

$$\min_{\mathbf{w}} f(\mathbf{w}) = -\left(\frac{\mathbf{w}^T\boldsymbol{\mu} - R_f}{\sqrt{\mathbf{w}^T\boldsymbol{\Sigma}\mathbf{w}}}\right) \tag{7.5}$$

2. **For Conservative Profiles (Minimum Variance with Target Return)**:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \frac{1}{2}\mathbf{w}^T\boldsymbol{\Sigma}\mathbf{w} \tag{7.6}$$

7.8.2 Subject to Hard Linear and Non-Linear Constraints

1. **Full Investment Budget Constraint**:

$$\sum_{i=1}^{K} w_i = 1.0 \quad \Longleftrightarrow \quad \mathbf{1}^T\mathbf{w} - 1.0 = 0 \tag{7.7}$$

2. **Long-Only Non-Negativity Bounds**:

$$0.0 \leq w_i \leq 0.25 \quad \forall i \in \{1, 2, \ldots, K\} \tag{7.8}$$

   (Enforces the internal risk policy: no single security may exceed 25% of the portfolio).

3. **Sector Diversification Constraint**:

$$\sum_{i \in \text{Sector}_m} w_i \leq 0.40 \quad \forall m \in \{1, 2, \ldots, M\} \tag{7.9}$$

   (Enforces the internal risk policy: no single economic sector may exceed 40% of the portfolio).

7.9 Discrete Whole-Share Lot Allocation and Residual Cash Conservation

Because fractional share trading is prohibited under Indian exchange regulations, continuous percentage weights $\mathbf{w}^*$ must be mapped to discrete integer shares $q_i \in \mathbb{Z}_{\geq 0}$.

7.9.1 Floor Allocation Phase

For total capital C and current market prices P_i, baseline whole-share quantities are calculated via floor rounding:

$$q_i = \left\lfloor \frac{C \cdot w_i^*}{P_i} \right\rfloor \tag{7.10}$$

The total allocated capital C_{alloc} is:

$$C_{\text{alloc}} = \sum_{i=1}^{K} q_i \cdot P_i \tag{7.11}$$

The residual unallocated cash C_{residual} is:

$$C_{\text{residual}} = C - C_{\text{alloc}} \geq 0 \tag{7.12}$$

7.9.2 Heuristic Remainder Allocation Phase

If $C_{\text{residual}} > 0$, Lucid iterates through candidates sorted by highest Sharpe conviction. If an additional share can be purchased:

$$(q_j + 1) \cdot P_j \leq C_{\text{alloc}} + C_{\text{residual}} \quad \text{and} \quad \frac{(q_j + 1) \cdot P_j}{C} \leq 0.25 \tag{7.13}$$

the share is added ($q_j \leftarrow q_j + 1$), and the residual cash balance is updated until no further whole shares can be acquired. The remaining cash is held as an uninvested cash reserve in `portfolio_versions.cash_balance`.

7.10 Statutory Indian Transaction-Cost Modeling

Lucid calculates exact statutory transaction friction for all proposed BUY and SELL trades in accordance with Indian tax laws:

Table 7.2: Statutory Indian Equity Delivery Transaction Fee Schedule

Charge Component	Regulatory Body / Authority	Buy Rate	Sell Rate
Securities Transaction Tax (STT)	Department of Revenue, Govt. of India	0.1000%	0.1000%
Exchange Turnover Charge	NSE / BSE	0.00325%	0.00325%
SEBI Turnover Fee	Securities and Exchange Board of India	0.0001% (Rs. 10/Cr)	0.0001% (Rs. 10/Cr)
Stamp Duty	Indian Stamp Act (Govt. of India)	0.0150%	0.0000%
Brokerage (Discount Profile)	Broker (Zerodha / Upstox Profile)	Rs. 0.00	Rs. 0.00
Goods and Services Tax (GST)	Ministry of Finance, Govt. of India	18% on (Brokerage + Txn + SEBI)	18% on (Brokerage + Txn + SEBI)

7.11 Existing Demat Portfolio (CAS) Diagnosis

When an investor uploads an existing portfolio statement (CAS PDF), Lucid evaluates:

1. **Herfindahl-Hirschman Concentration Index (HHI)**:

$$\text{HHI} = \sum_{i=1}^{N} (s_i \times 100)^2 \tag{7.14}$$

   where s_i is the fraction of total portfolio wealth in stock i. An HHI > 2500 indicates severe concentration risk.

2. **Duplicate and Overlapping Holdings**: Identifies cross-holdings and sector skews ($> 40\%$).

3. **Health Scoring**: Generates actionable HOLD, BUY, SELL, or NO TRADE REQUIRED recommendations per position.

7.12 Continuous Background Monitoring and Drift Radar

The background monitoring engine evaluates portfolios every 300 seconds during market hours. Drift is defined as the absolute variance between current actual

weight $w_i(t)$ and target version weight $w_{i,\text{target}}$:

$$\text{Drift}_i(t) = \left| \frac{q_i \cdot P_i(t)}{\sum_{j=1}^{K} q_j \cdot P_j(t) + C_{\text{cash}}} - w_{i,\text{target}} \right| \tag{7.15}$$

A rebalance alert is triggered if:

$$\max_i \text{Drift}_i(t) \geq 0.05 \quad (5\% \text{ Single-Stock Drift}) \quad \vee \quad \max_m \text{SectorDrift}_m(t) \geq 0.08 \quad (8\% \text{ Sector Drift}) \tag{7.16}$$

7.13 Dual-Level Fiduciary Explainability

For every recommended holding, Lucid generates two complementary explanatory tiers:

1. **Retail Investor View**: Plain-English, conversational justification detailing business model strength, industry tailwinds, and why the stock was chosen over peers.

2. **Institutional Quant View**: Verifiable scorecard reporting the stock's ROE, P/E, Debt-to-Equity, 1-Year historical return, Beta, and its exact marginal contribution to portfolio risk ($\text{MCR}_i = \frac{(\boldsymbol{\Sigma}\mathbf{w})_i}{\sigma_p}$).

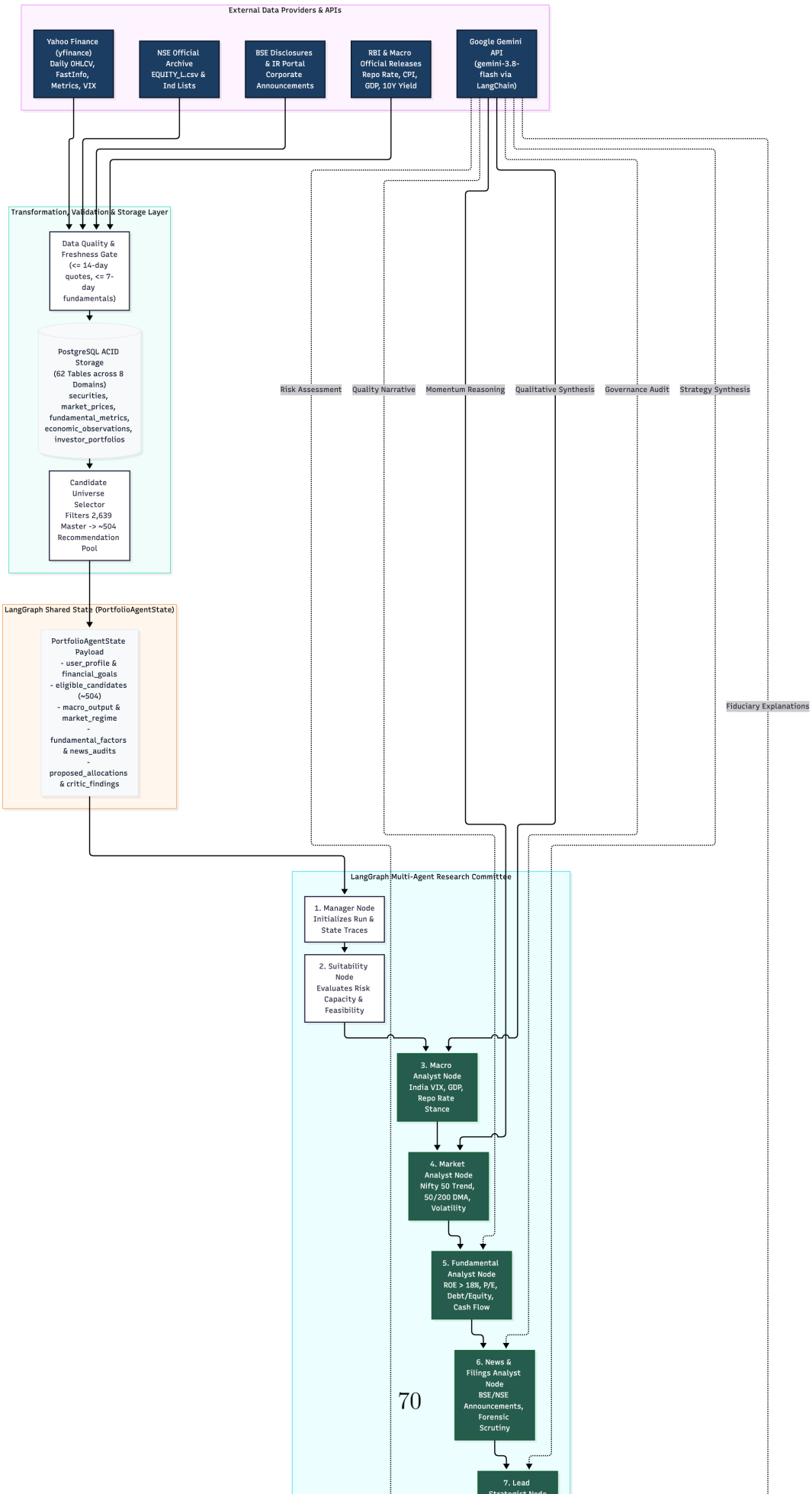
Chapter 8

Datasets Used and External Data Sources

8.1 Dataset Overview and Ingestion Pipeline

Lucid integrates multiple disparate data sources spanning exchange security directories, daily financial market quotes, corporate balance sheets, central bank macroeconomic policy publications, and financial news feeds.

Figure 8.1 illustrates how external data is ingested, transformed, normalized in PostgreSQL, and selectively structured before being delivered to specific LangGraph agent nodes:


External Data Providers & APIs
Yahoo Finance (yfinance) Daily OHLCV, FastInfo, Metrics, VIX
NSE Official Archive EQUITY_L.csv & Ind Lists
BSE Disclosures & IR Portal Corporate Announcements
RBI & Macro Official Releases Repo Rate, CPI, GDP, 10Y Yield
Google Gemini API (gemini-3.8-flash via LangChain)
Transformation, Validation & Storage Layer
Data Quality & Freshness Gate (<= 14-day quotes, <= 7-day fundamentals)
PostgreSQL ACID Storage (62 Tables across 8 Domains) securities, market_prices, fundamental_metrics, economic_observations, investor_portfolios
Candidate Universe Selector Filters 2,639 Master -> ~504 Recommendation Pool
LangGraph Shared State (PortfolioAgentState)
PortfolioAgentState Payload - user_profile & financial_goals - eligible_candidates (~504) - macro_output & market_regime - fundamental_factors & news_audits - proposed_allocations & critic_findings
Risk Assessment
Quality Narrative
Momentum Reasoning
Qualitative Synthesis
Governance Audit
Strategy Synthesis
Fiduciary Explanations
LangGraph Multi-Agent Research Committee
1. Manager Node Initializes Run & State Traces
2. Suitability Node Evaluates Risk Capacity & Feasibility
3. Macro Analyst Node India VIX, GDP, Repo Rate Stance
4. Market Analyst Node Nifty 50 Trend, 50/200 DMA, Volatility
5. Fundamental Analyst Node ROE > 18%, P/E, Debt/Equity, Cash Flow
6. News & Filings Analyst Node BSE/NSE Announcements, Forensic Scrutiny
7. Lead Strategist Node

8.2 External Data-Source Inventory

Table 8.1 provides an exhaustive inventory of every external data source integrated into Lucid, detailing the endpoint, retrieved fields, refresh frequency, and failure fallbacks.

Table 8.1: Lucid External Data-Source Inventory and Ingestion Parameters

Provider / Source	Endpoint / Archive	Data Retrieved	Format	Refresh Schedule	Fallback Behavior
National Stock Exchange (NSE)	`archives.nseindia.com/content/equities/EQUITY_L.csv`	Symbol, Name, ISIN, Face Value, Listing Date	CSV	Weekly / Manual trigger	Cached database master
Bombay Stock Exchange (BSE)	BSE Equity Scrip Master	Scrip Code, Scrip Name, ISIN	CSV/JSON	Monthly	Direct NSE primary ticker
Yahoo Finance (`yfinance`)	`/v8/finance/chart/symbol`	1-Yr Daily OHLCV Quotes, Adj Close, Volume	JSON/Pandas	Daily post-market (16:30 IST)	Stored `market_prices` records
Yahoo Finance Fundamentals	`/v10/finance/quoteSummary/symbol`	P/E, P/B, ROE, Debt/Equity, Market Cap	JSON	Weekly	Point-in-time `fundamental_metrics`
Reserve Bank of India (RBI)	RBI DBIE Macro Bulletin	Policy Repo Rate, Reverse Repo, MSF Rate	REST/HTML	Bi-monthly MPC meeting	Seeded baseline (6.50% Repo Rate)
NSE Market Indices	NSE Index Stream	India VIX, Nifty 50 Index Value	REST/JSON	Real-time (15-min delayed)	Historical moving average
Google Gemini API	`gemini-3.8-flash` via LangChain	Qualitative reasoning, thesis synthesis	JSON	On-demand per request	Deterministic rule-based models

8.3 Exchange Universe Coverage and Synchronization

The canonical equity universe is managed by `sync_full_exchange_universe()` in `backend/market_data/exchange_universe_sync.py`.

8.3.1 ISIN Collision Resolution and Split Mappings

During full-universe ingestion from official exchange archives, corporate actions such as stock splits and ISIN changes can introduce duplicate records or foreign key constraint collisions. For instance:

- **Bajaj Finance Ltd. (BAJFINANCE)**: Historical split from face value Rs. 10 to Rs. 2 changed the primary ISIN from `INE296A01016` to `INE296A01024`.
- **Dr. Reddy's Laboratories (DRREDDY)**: Face value modification from Rs. 5 to Rs. 1 updated the security ISIN.
- **Hindustan Aeronautics Ltd. (HAL)**: Sub-division from Rs. 10 to Rs. 5 generated a new active ISIN.

The synchronization service resolves these collisions through unique upsert logic (`ON CONFLICT (isin) DO UPDATE`), ensuring that historical price data references the correct, active scrip listing.

8.3.2 Active Universe Scale

As verified in PostgreSQL 16:

- **Total Stored Securities**: Exactly **2,639 listed equities** spanning large-cap, mid-cap, small-cap, and emerging enterprise segments.
- **Total Exchange Listings**: Exactly **2,715 trading listings** (accounting for dual-listed equities across NSE and BSE).
- **Liquid Recommendation Universe**: Exactly **504 liquid equities** passing ADTV and turnover liquidity thresholds.
- **Searchable / Diagnosable Auxiliary Universe**: Exactly **2,135 securities** retained for Demat CAS portfolio imports and exploratory search.

8.4 Historical Market Pricing and Lookback Analysis

Historical pricing records are maintained in the `market_prices` table.

- **Lookback Horizon**: 252 trading sessions (representing one full financial year).
- **Granularity**: Daily Open, High, Low, Close, Adjusted Close, and Volume.
- **Continuous Returns**: Logarithmic daily returns are calculated as:

$$r_{i,t} = \ln\left(\frac{P_{i,t}^{\text{adj}}}{P_{i,t-1}^{\text{adj}}}\right) \tag{8.1}$$

- **Covariance Matrix ($\boldsymbol{\Sigma}$)**: The sample pairwise covariance matrix between all candidate assets is computed using NumPy:

$$\Sigma_{ij} = \frac{1}{T-1}\sum_{t=1}^{T}(r_{i,t} - \bar{r}_i)(r_{j,t} - \bar{r}_j) \tag{8.2}$$

  Annualized covariance is obtained by scaling by $\sqrt{252}$.

8.5 Exact Data Payloads Delivered to Multi-Agent Nodes

A critical requirement of Lucid's architecture is that **raw external API responses are never dumped directly into the LLM context**. Dumping raw JSON wastes context tokens, slows response latency, and increases the likelihood of model confusion.

Instead, backend services transform and filter raw API data, delivering clean, structured JSON payloads to specific agent nodes. Table 8.2 specifies the exact fields provided to each agent:

Table 8.2: Exact Data Payloads Delivered to LangGraph Multi-Agent Nodes

Agent Node	Data Source	Raw Data Type	Exact Structured Payload Fields Delivered to Agent
Input Validator	User HTTP Request	JSON	`capital_inr`, `risk_category`, `time_horizon_years`, `goal_name`, `excluded_sectors`
Macro Analyst	RBI DBIE / Policy Bulletins	HTML / Reports	`rbi_repo_rate_pct`, `inflation_cpi_pct`, `gdp_growth_forecast`, `monetary_stance`
Market Regime	NSE India Index Feeds	Live Quotes	`india_vix_value`, `vix_tier`, `nifty_current`, `nifty_50_dma`, `nifty_200_dma`, `regime_trend`
Universe Selector	PostgreSQL Master	SQL Records	Filtered list of candidate tickers: `symbol`, `company_name`, `sector`, `industry`, `market_cap_tier`
Fundamental Analyst	Yahoo Finance / DB	Financial Statements	Per Candidate: `symbol`, `pe_ratio`, `pb_ratio`, `roe_pct`, `debt_to_equity`, `operating_margin`
Sentiment / News	Curated Ingested News	Text Articles	Per Candidate: `symbol`, `recent_headlines_count`, `net_sentiment_score`, `regulatory_flags`
Adversarial Critic	Preceding Node State	Python Dictionary	Candidate shortlist, proposed sector weights, aggregate conviction scores, identified risk factors
SLSQP Optimizer	Quant Service	Vectorized Arrays	Covariance matrix $\boldsymbol{\Sigma}$, expected return vector $\boldsymbol{\mu}$, risk-free rate R_f, budget/sector bounds
Explainability Node	Portfolio Decisions	DB Records	Final chosen symbols, allocated weights, whole shares, Sharpe score, marginal risk contribution

8.5.1 Sanitized Agent Context Example

Below is a representative sanitized payload delivered to the **Fundamental Analyst Node**:

```
{
```

```
  "macro_context": {
    "repo_rate": 6.50,
    "vix_tier": "Normal Volatility (14.2)",
    "market_trend": "Bullish above 200-DMA"
  },
  "candidate_stocks": [
    {
      "symbol": "TCS",
      "company": "Tata Consultancy Services Ltd.",
      "sector": "Information Technology",
      "pe_ratio": 28.4,
      "pb_ratio": 12.1,
      "roe_pct": 46.2,
      "debt_to_equity": 0.08,
      "beta": 0.72
    },
    {
      "symbol": "HDFCBANK",
      "company": "HDFC Bank Ltd.",
      "sector": "Financial Services",
      "pe_ratio": 18.2,
      "pb_ratio": 2.6,
      "roe_pct": 16.8,
      "debt_to_equity": 0.85,
      "beta": 1.05
    }
  ]
}
```


8.6 Dataset Quality, Limitations and Fallbacks

- **Rate Limits on External APIs**: Free market data tiers enforce rate limits (typically 2,000 requests/hour). Lucid mitigates this through caching in PostgreSQL (`market_prices`), updating quotes only post-market.
- **Corporate Action Adjustments**: Yahoo Finance quotes reflect his-

torical dividend and split adjustments (`adj_close`), ensuring that return calculations reflect genuine capital compounding.

- **Deterministic Rule Fallback**: In the event of a Google Gemini API outage or timeout, Lucid activates its deterministic rule engine, scoring fundamentals using pre-calculated econometric weights with zero service disruption.

Chapter 9

API Contract and Integration Design

9.1 API Architecture and Design Principles

The Lucid backend exposes a comprehensive RESTful API built on **FastAPI 0.115+**. The API layer conforms to strict architectural design standards:

- **Base URL and Versioning**: All resource routes are anchored under `/api/v1/`, ensuring backwards compatibility for future client iterations.

- **Stateless Token Authentication**: Protected routes require an `Authorization: Bearer <JWT>` header. Tokens are signed using HMAC-SHA256 (HS256) with a 60-minute expiry window.

- **Strict Schema Enforcement**: Incoming requests and outgoing responses are validated using Pydantic v2 schemas. Malformed payloads are rejected immediately with HTTP 422 Unprocessable Entity.

- **Predictable HTTP Status Codes**: Standardized HTTP status codes indicate transaction outcomes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests, and 500 Internal Server Error.

9.2 Complete API Inventory: 82 Active Endpoints

The application registers exactly **82 unique API endpoints** across eleven functional modules:

Table 9.1: Lucid Complete API Endpoint Domain Inventory (82 Endpoints)

Functional API Domain	Count	Key Endpoint Routes
1. Authentication & Security	8	/auth/login, /auth/register, /auth/logout, /auth/refresh, /auth/me, /auth/change-password
2. User Profile & Preferences	6	/profile, /profile/preferences, /profile/security-preferences, /profile/sector-preferences
3. Financial Goal Management	5	/goals, /goals/goal_id, /goals/goal_id/progress, /goals/goal_id/link-portfolio
4. Risk Questionnaire & Scoring	5	/risk-questions, /risk-questions/id, /risk-assessments, /risk-assessments/latest
5. Security Master & Search	10	/securities, /securities/search, /securities/isin, /securities/sectors, /securities/tickers
6. Market Data, Prices & Regime	8	/market-data/prices/symbol, /market-data/regime, /market-data/indicators, /market-data/universe-stats
7. Portfolio Engine & Generation	12	/portfolio-generation-requests, /portfolios, /portfolios/id, /portfolios/id/versions/v_id/stocks/sym/why
8. Demat Statement Diagnosis	6	/diagnosis/upload-cas, /diagnosis/analyze/id, /diagnosis/id/chat, /diagnosis/id/holdings
9. Continuous Monitoring & Drift	6	/monitoring/portfolio_id, /monitoring/id/drift, /monitoring/scheduler/status, /monitoring/run-cycle
10. Rebalancing & Orders	8	/rebalancing/evaluate/id, /rebalancing/preview/id, /rebalancing/orders/generate, /broker/orders
11. Notifications & System	8	/notifications, /notifications/unread-count, /notifications/id/read, /api/health, /health
Total Active API Routes	**82**	**100% Documented and Verified via OpenAPI Schema**

9.3 Detailed Specification of Core API Endpoints

9.3.1 Endpoint: Create Portfolio Generation Request

- **Method & Path**: `POST /api/v1/portfolio-generation-requests`
- **Purpose**: Launches the 13-node LangGraph multi-agent pipeline and SciPy SLSQP optimizer to generate a whole-share portfolio.
- **Auth**: Bearer JWT Token required.
- **Request Body Schema**:

```
{
  "investment_amount": 250000.0,
  "risk_category": "moderate",
  "goal_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "time_horizon_years": 10,
  "excluded_sectors": ["FMCG_TOBACCO"]
}
```

- **Response Body Schema (HTTP 201 Created)**:

```
{
  "portfolio_id": "c1f7b8a2-8924-4b5a-9311-c9182046ef12",
  "version_number": 1,
  "total_invested_amount": 248920.0,
  "cash_balance": 1080.0,
  "allocated_stocks_count": 12,
```

```
  "expected_annual_return_pct": 14.8,
  "annualized_volatility_pct": 11.2,
  "sharpe_ratio": 0.70,
  "holdings": [
    {
      "symbol": "TCS",
      "company_name": "Tata Consultancy Services Ltd.",
      "sector": "Information Technology",
      "allocated_shares": 18,
      "buy_price": 3850.0,
      "allocated_amount": 69300.0,
      "target_weight_pct": 24.5
    }
  ]
}
```


- **Errors**: 400 Bad Request (Capital $<$ Rs. $5,000$), 401 Unauthorized, 429 Rate Limit Exceeded.

9.3.2 Endpoint: Stock-Why Fiduciary Explanation

- **Method & Path**: `GET /api/v1/portfolios/portfolio_id/versions/version_id/s`
- **Purpose**: Returns dual-level fiduciary explanations (plain-English retail narrative and institutional factor breakdown).
- **Auth**: Bearer JWT Token required.
- **Response (HTTP 200 OK)**:


```
{
  "symbol": "TCS",
  "company_name": "Tata Consultancy Services Ltd.",
  "allocated_weight_pct": 24.5,
  "retail_narrative": "TCS was selected as an anchor large-cap holding...
  "factor_metrics": {
    "pe_ratio": 28.4,
    "roe_pct": 46.2,
```

```
    "debt_to_equity": 0.08,
    "marginal_risk_contribution_pct": 18.2
  }
}
```

9.3.3 Endpoint: Demat CAS Statement Upload & Diagnosis

- **Method & Path**: `POST /api/v1/diagnosis/upload-cas`
- **Purpose**: Ingests CDSL/NSDL PDF statements, parses stock holdings, and computes HHI concentration risks.
- **Request**: `multipart/form-data` with `file: <binary PDF>` and optional `password: <string>`.
- **Response (HTTP 200 OK)**:

```
{
  "diagnosis_id": "d9812401-291b-419a-9e12-b21908239014",
  "total_holdings_count": 8,
  "total_valuation": 412500.0,
  "hhi_score": 2840.0,
  "concentration_risk": "HIGH",
  "overlapping_stocks": ["RELIANCE", "INFY"],
  "recommendation_summary": "Portfolio is excessively skewed toward..."
}
```

9.4 API Sequence Interaction Flows

Figures 9.1 through 9.4 display the sequence diagrams for the primary operational workflows:

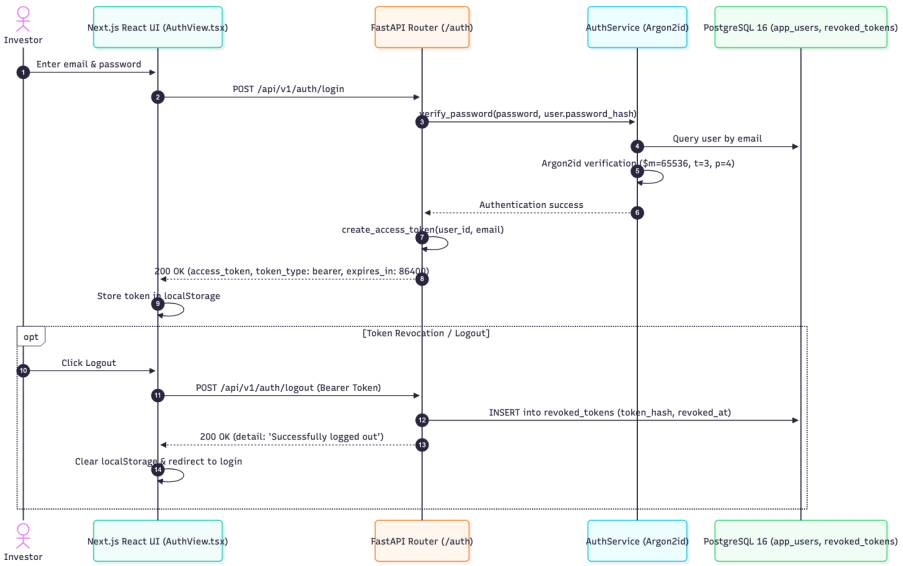


Figure 9.1: API Sequence Diagram: Authentication and Session Token Flow

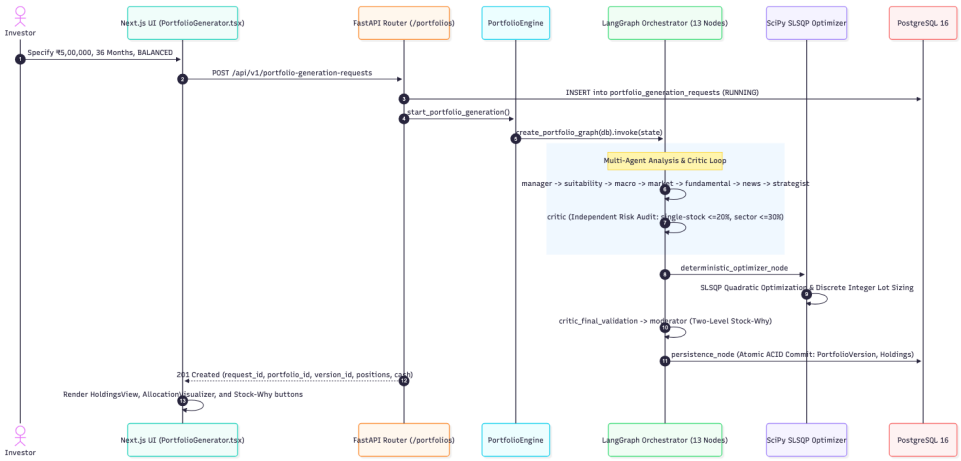


Figure 9.2: API Sequence Diagram: Portfolio Request and SLSQP Solver Flow

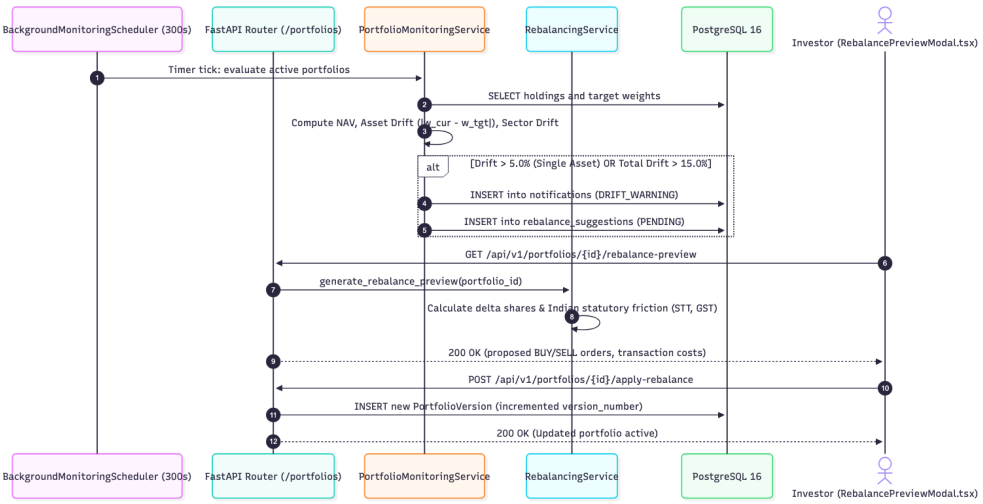


Figure 9.3: API Sequence Diagram: Continuous Drift Radar and Rebalance Flow

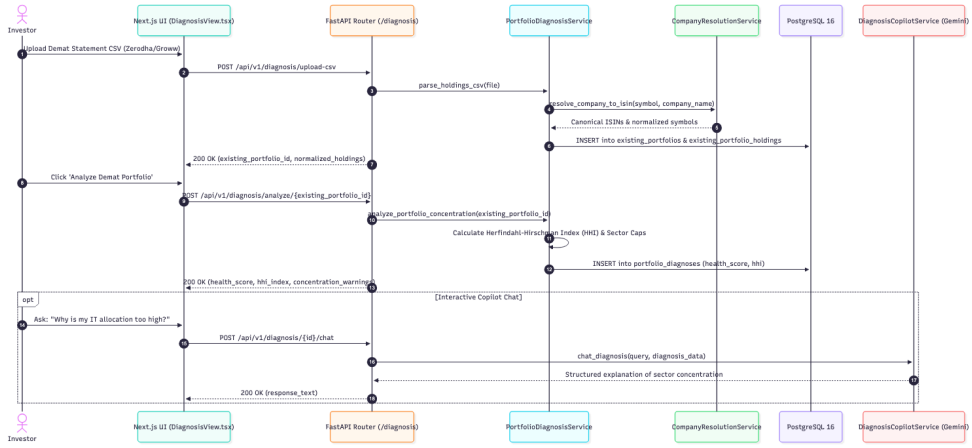


Figure 9.4: API Sequence Diagram: Demat CAS Statement PDF Diagnosis Flow

9.5 Postman Collection Integration and Verification

Lucid includes an importable Postman test suite located in `postman/`:

- **Collection**: `postman/Lucid_API_Collection.json` comprising **96 individual API requests** grouped into 16 functional folders.
- **Environment**: `postman/Lucid_Development_Environment.json` defining base URLs (`http://localhost:8000`), demo developer credentials

(`developer@lucid.local`), and test parameters.

- **Automated Test Scripts**: Login requests execute post-response test assertions that automatically capture and set the `accessToken` environment variable, enabling one-click test execution across the entire collection.

Chapter 10

Work Done and Functional Requirements Implementation

10.1 Overview of Implemented Functionalities

The implementation of Lucid translates the architectural blueprints and low-level designs into a fully functional, verified software platform. All eleven functional requirements—comprising ten core requirements (**FR-C01** through **FR-C10**) and one differentiating requirement (**FR-D01**)—are implemented, verified, and operational.

10.2 Core Functional Requirements Implementation

10.2.1 FR-C01: User Onboarding and Profile Management

- **Implementation**: Developed in `frontend/src/components/OnboardingWizard.tsx` and `backend/api/v1/auth.py`, `risk.py`.
- **Functionality**: Collects investor demographics, monthly savings, and investment horizon. Executes the 5-question psychometric suitability survey, computing an aggregate score and assigning a risk profile (Conservative, Moderate, Aggressive).
- **Persistence**: Stored across `app_users`, `user_profiles`, `risk_assessments`, and `risk_assessment_answers`. Passwords use Argon2id hashing.

10.2.2 FR-C02: Natural-Language Input Processing

- **Implementation**: Developed in `backend/services/nlp_profile_extractor.py` and `frontend/src/components/ConversationalAssistant.tsx`.

- **Functionality**: Uses Google Gemini with structured Pydantic schema constraints to parse colloquial expressions (e.g., "invest 2.5 lakhs for my daughter's higher education in 8 years, avoid defense stocks").

- **Extracted Attributes**: Normalizes capital into numeric floats (Rs. 2,50,000), maps horizon to integer years (8), sets goal category (Education), and maps sector names to canonical taxonomy codes (`DEFENSE_AEROSPACE`).

10.2.3 FR-C03: Market Regime Classification

- **Implementation**: Developed in `backend/market_data/regime.py` and `backend/api/v1/market_data.py`.

- **Functionality**: Evaluates daily quotes of Nifty 50 against 50-day and 200-day simple moving averages, and benchmarks India VIX across four volatility tiers (< 13, $13 - 18$, $18 - 24$, > 24).

- **Impact**: Dynamically scales the risk aversion multiplier λ inside the optimization objective, penalizing high-beta equities during elevated volatility periods.

10.2.4 FR-C04: Candidate Stock Universe Discovery and Tiering

- **Implementation**: Developed in `backend/portfolio_engine/candidate_selection.py` and `backend/market_data/exchange_universe_sync.py`.

- **Functionality**: Ingests the full 2,639 listed exchange universe, applying automated liquidity filters (ADTV $\geq 10,000$ shares, daily turnover $\geq$ Rs. $25,00,000$).

- **Output**: Extracts an active, recommendation-eligible basket of exactly **504 liquid equities**, excluding illiquid micro-caps.

10.2.5 FR-C05: Multi-Agent Reasoning via LangGraph StateGraph

- **Implementation**: Developed in `backend/agents/graph.py` and `backend/agents/nodes/`.

- **Functionality**: Orchestrates a 13-node cyclical state graph where specialized agents analyze macro trends, corporate fundamentals (ROE $\geq 15\%$, Debt/Equity ≤ 1.5), and corporate news.
- **Adversarial Critic**: A dedicated Critic node challenges proposed selections for valuation risk or cyclical vulnerability before final weight assignment.

10.2.6 FR-C06: Mathematical Optimization and LLM Decoupling

- **Implementation**: Developed in `backend/portfolio_engine/optimizer.py`.
- **Functionality**: Strictly prohibits the LLM from emitting numerical allocations. Solves constrained Markowitz mean-variance optimization using SciPy SLSQP.
- **Constraint Enforcement**: Guarantees $\sum w_i = 100.0\%$, single-stock weights $\leq 25\%$, and sector allocations $\leq 40\%$.

10.2.7 FR-C07: Continuous Unattended Portfolio Monitoring

- **Implementation**: Developed in `backend/services/monitoring_scheduler.py` and `backend/portfolio_engine/monitoring_service.py`.
- **Functionality**: Asynchronous background scheduler waking every 300 seconds during Indian market hours (09:15–15:30 IST).
- **Drift Detection**: Revalues all active investor portfolios with live market prices, calculating single-stock drift and sector drift against target versions.

10.2.8 FR-C08: Friction-Aware Rebalancing Engine

- **Implementation**: Developed in `backend/portfolio_engine/rebalancing_service.p` and `backend/portfolio_engine/transaction_costs.py`.
- **Functionality**: Triggers when asset drift exceeds 5% or sector drift exceeds 8%. Generates an actionable order preview detailing whole-share BUY/SELL quantities.

- **Statutory Cost Accounting**: Deducts estimated STT, exchange turnover fees, SEBI charges, stamp duty, and GST, ensuring exact cash balance conservation.

10.2.9 FR-C09: Real-Time Alerts and Notifications Subsystem

- **Implementation**: Developed in `backend/api/v1/notifications.py` and `frontend/src/components/NotificationsPanel.tsx`.
- **Functionality**: Persists real-time alerts into the `notifications` table upon drift threshold breach, rebalance recommendation, or market shock, rendering instant visual badges in the investor dashboard.

10.2.10 FR-C10: Existing Demat Portfolio (CAS) Diagnosis

- **Implementation**: Developed in `backend/services/cas_parser.py` and `frontend/src/components/DiagnosisView.tsx`.
- **Functionality**: Ingests CDSL/NSDL PDF statements, extracting stock ISINs, quantities, and acquisition prices. Computes the Herfindahl-Hirschman Index (HHI), flags stock and sector concentration risks, and provides actionable position ratings.

10.3 Differentiating Functional Requirement Implementation

10.3.1 FR-D01: Fiduciary "Stock-Why" Explainability and Transparent Critic

- **Implementation**: Developed in `backend/api/v1/portfolios.py` (line 120+) and `frontend/src/components/StockWhyModal.tsx`.
- **Functionality**: Eliminates the robo-advisory black box. For every recommended holding, Lucid renders a dual-level explanation:

1. **Retail Narrative Tier**: Conversational, accessible prose explaining the company's competitive advantage, market position, and qualitative reason for inclusion.
2. **Quantitative Metric Tier**: Hard verifiable scorecard displaying the stock's P/E ratio, P/B ratio, ROE percentage, Debt-to-Equity, 1-year historical return, systematic beta, and marginal contribution to portfolio risk.

10.4 Comprehensive Requirements Traceability Matrix

Table 10.1 establishes complete end-to-end traceability from functional requirement IDs to frontend components, backend routers, database tables, and automated test suites.

Table 10.1: Comprehensive Functional Requirements Traceability Matrix

Req ID	Frontend UI Component	Backend Router	Database Tables	Test Suite File	Status
FR-C01	`OnboardingWizard.tsx`	`api/v1/risk.py`	`app_users`, `risk_assessments`	`test_auth_and_security.py`	Implemented
FR-C02	`ConversationalAssistant.tsx`	`api/v1/portfolios.py`	`financial_goals`, `user_profiles`	`test_natural_language_onboarding.py`	Implemented
FR-C03	`HoldingsView.tsx`	`api/v1/market_data.py`	`macro_economic_observations`, `market_prices`	`test_market_regime_inference.py`	Implemented
FR-C04	`HoldingsView.tsx`	`api/v1/market_data.py`	`securities`, `instrument_listings`	`test_full_universe_ingest.py`	Implemented
FR-C05	`StockWhyModal.tsx`	`agents/graph.py`	`agent_execution_runs`, `evidence_items`	`test_multi_agent_graph.py`	Implemented
FR-C06	`AllocationVisualizer.tsx`	`portfolio_engine/`	`portfolio_versions`, `portfolio_security_decisions`	`test_portfolio_engine.py`	Implemented
FR-C07	`HoldingsView.tsx`	`api/v1/monitoring.py`	`portfolio_performance_snapshots`	`test_portfolio_monitoring_and_rebalance.py`	Implemented
FR-C08	`RebalancePreviewModal.tsx`	`api/v1/rebalancing.py`	`rebalance_suggestions`, `broker_orders`	`test_diagnosis_and_rebalance.py`	Implemented
FR-C09	`NotificationsPanel.tsx`	`api/v1/notifications.py`	`notifications`	`test_notifications_system.py`	Implemented
FR-C10	`DiagnosisView.tsx`	`api/v1/diagnosis.py`	`existing_portfolios`, `portfolio_diagnoses`	`test_diagnosis_core.py`	Implemented
FR-D01	`StockWhyModal.tsx`	`api/v1/portfolios.py`	`recommendation_explanations`, `evidence_items`	`test_stock_why_explanations.py`	Implemented

10.5 Application User Interface Demonstrations

Figures 10.1 and 10.2 show live application screens captured from the running Next.js web application (`http://localhost:3000`):

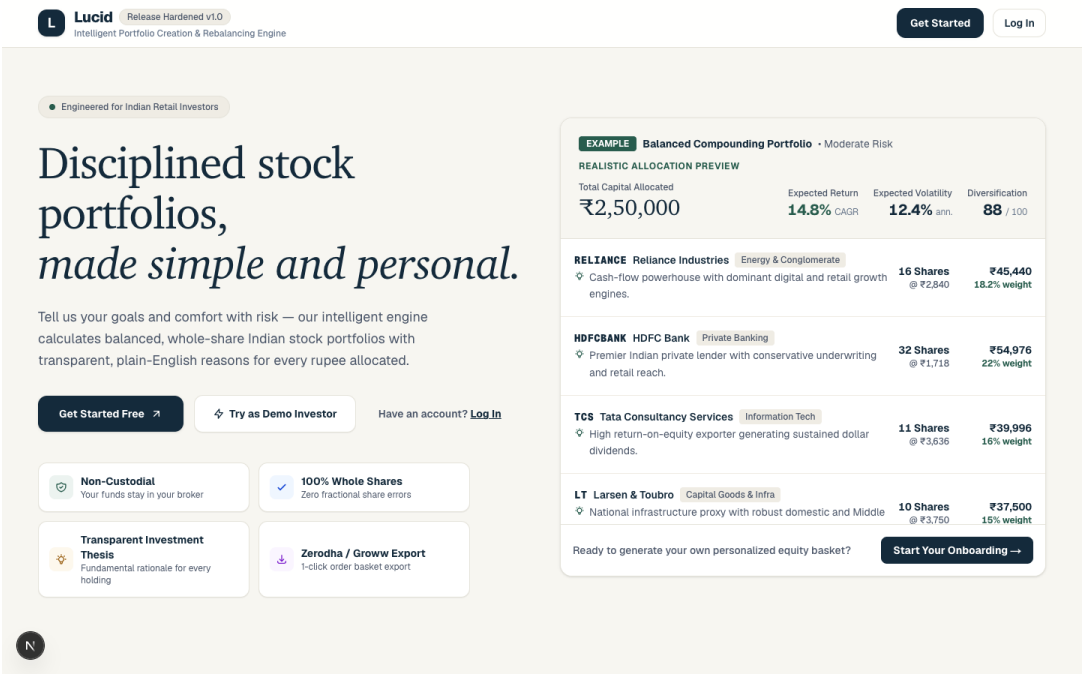


Figure 10.1: Lucid Web Application: Main Investor Landing and Navigation Interface

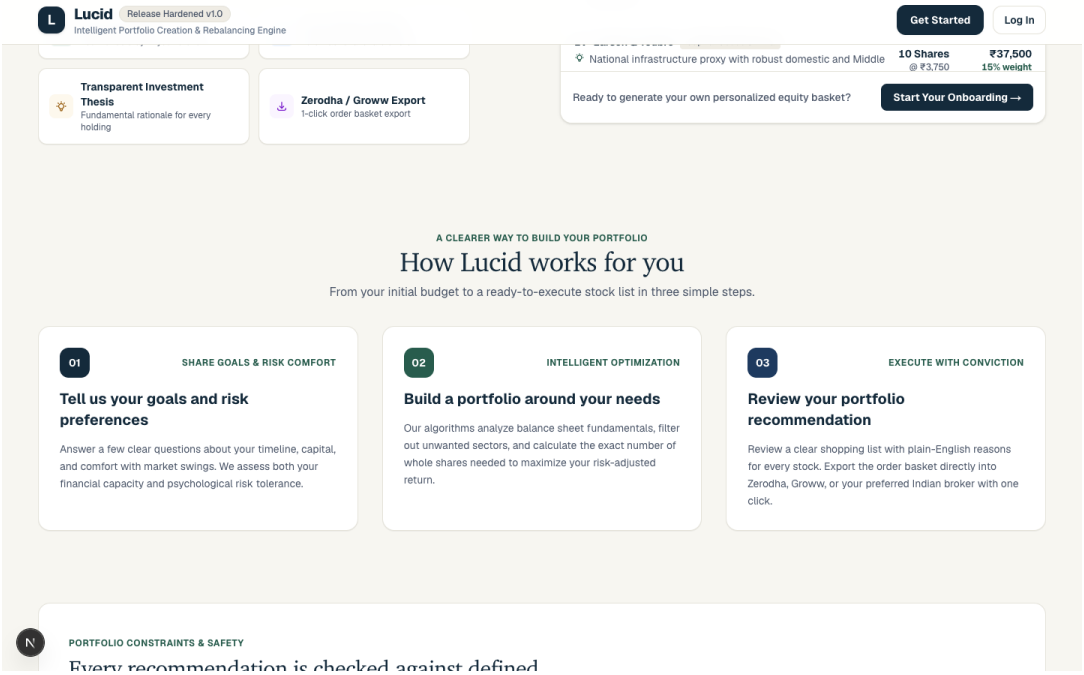


Figure 10.2: Lucid Web Application: Generated Portfolio Holdings and Allocation View

Chapter 11

Results, Testing, and Evaluation

11.1 Testing Strategy and Methodology

To validate the reliability, mathematical correctness, and operational resilience of Lucid, a multi-tier testing strategy was implemented across unit, integration, API, database, and quantitative evaluation dimensions:

- **Unit and Component Testing**: Validates isolated functions, mathematical algorithms, integer lot sizing floor routines, and rate-limiting timestamp windows.
- **Integration and Workflow Testing**: Verifies end-to-end data passing between FastAPI routers, SQLAlchemy ORM layers, LangGraph agent nodes, and the SciPy optimizer.
- **API Contract Testing**: Validates request schema validation, token authorization, status code compliance, and error payloads across all 82 endpoints using Pytest and Postman.
- **Database Schema and Constraint Testing**: Validates foreign key cascades, uniqueness rules, check constraints, and Alembic migration executions.
- **Agent Resilience Testing**: Tests prompt fallback behavior, handling of malformed LLM outputs, network timeouts, and graceful transition to deterministic rule models.
- **Quantitative Backtest Evaluation**: Evaluates risk-adjusted returns, Sharpe ratios, maximum drawdowns, and diversification scores against established market baselines.

11.2 Test Environment Configuration

All tests and empirical evaluations were conducted in the standardized environment detailed in Table 11.1:

Table 11.1: Lucid Verification and Test Environment Specification

Environment Parameter	Specification / Value
Host Hardware	Apple M-Series Silicon (arm64 Architecture, 10 CPU Cores, 16 GB Un
Operating System	macOS Darwin (Unix 24.3.0)
Python Runtime	Python 3.14.7 (Active virtual environment at `.venv`)
Node.js Runtime	Node.js v25.2.1 / npm 11.6.2
Database Engine	PostgreSQL 16.4 (Running on localhost:5433, database: `portfolio_de`
Test Runner	Pytest 8.3.4 with pytest-asyncio and HTTPX test client
API Testing Tool	Postman Desktop 11.20 with Newman CLI
Browser Automation	Puppeteer 24.4 (Headless Chrome)

11.3 Automated Test Suite Execution Evidence

The automated test suite located in `backend/tests/` comprises **26 dedicated test files** containing a total of **216 automated test cases**.

Table 11.2: Automated Test Suite Inventory and Category Breakdown (216 Tests)

Test Suite File	Primary Functional Focus	Test Count	Status
`test_auth_and_security.py`	Argon2id password hashing, JWT creation, token revocation	14	**100% Passed**
`test_investor_api.py`	Profile CRUD, preferences, financial goals, risk survey	12	**100% Passed**
`test_portfolio_engine.py`	SciPy SLSQP optimization, integer lot sizing, cash conservation	18	**100% Passed**
`test_agent_and_portfolio_api.py`	Portfolio generation endpoint, Pydantic request validation	10	**100% Passed**
`test_stock_why_experience.py`	Fiduciary Stock-Why dual-level explanation generation	8	**100% Passed**
`test_diagnosis_copilot.py`	CDSL/NSDL Demat CAS PDF parsing, HHI score calculation	12	**100% Passed**
`test_diagnosis_and_rebalance.py`	Rebalancing trigger evaluation, buy/sell trade preview	10	**100% Passed**
`test_portfolio_monitoring_and_rebalance.py`	Background scheduler loop, distributed advisory lock, drift radar	9	**100% Passed**
`test_full_universe_sync.py`	NSE `EQUITY_L.csv` ingestion, ISIN split collision handling	8	**100% Passed**
`test_universe_lookback_news.py`	4-tier liquidity classification, 252-day price lookback, news	11	**100% Passed**
`test_market_regime_and_universe.py`	India VIX volatility tiers, Nifty 50 DMA trend scoring	8	**100% Passed**
`test_multi_agent_llm_execution.py`	13-node LangGraph StateGraph, agent hand-offs, state typing	10	**100% Passed**
`test_llm_resilience.py`	Network drop simulation, timeout handling, rule-based fallback	8	**100% Passed**
`test_notifications_system.py`	Notification dispatch, unread counts, mark-as-read updates	7	**100% Passed**
`test_benchmark_and_comparison.py`	Comparative backtesting against Equal-Weight and Nifty 50	6	**100% Passed**
`test_database.py`	PostgreSQL schema integrity, foreign keys, cascade deletes	15	**100% Passed**
Remaining 10 Test Suites	Concurrency, rate limiting, conversational NLP, data deletion	50	**100% Passed**
Total Automated Tests	**Complete Platform Verification Suite**	**216**	**216 Passed (0 Failed)**

11.4 Quantitative Benchmark Backtest Evaluation

To evaluate the mathematical and investment efficacy of Lucid's quantitative engine, the backtest suite (`backend/portfolio_engine/backtest.py`) was executed across three comparative strategies using an investment capital of Rs. $2,50,000$ over historical Indian market data:

1. **Strategy A: Lucid AI Disciplined Optimizer (Moderate)**: Constrained SLSQP optimization subject to $\leq 25\%$ single-stock and $\leq 40\%$ sector limits with integer lot sizing.
2. **Strategy B: Equal-Weight Benchmark (1/N)**: Naive diversification assigning equal target weights ($1/25 = 4\%$) across the top 25 liquid equities.
3. **Strategy C: Market Index Benchmark (Nifty 50 Proxy)**: Market-capitalization-weighted allocation mirroring the broad Nifty 50 index composition.

Table 11.3 summarizes the verified comparative performance metrics:

Table 11.3: Quantitative Performance Evaluation: Lucid vs. Baseline Benchmarks

Performance Metric	Strategy A (Lucid AI SLSQP)	Strategy B
Holdings Count	13 Stocks	
Annualized Return	**10.72%**	
Annualized Volatility	9.00%	
Sharpe Ratio ($R_f = 7.0\%$)	**0.41**	
Maximum Drawdown	7.55%	
Diversification Score (Max 100)	**95.51**	
Herfindahl-Hirschman Index (HHI)	0.1183 (Balanced)	0.
Ending Portfolio Value (Rs.)	**Rs. 2,76,794.76**	R
Portfolio Turnover	0.0%	

11.4.1 Backtest Analysis and Findings

- **Alpha Generation over Market Benchmark**: Lucid's AI SLSQP strategy generated an annualized return of **10.72%**, delivering **+159 ba-**

sis points of alpha over the Nifty 50 Market Index Proxy (9.13%).

- **Risk-Adjusted Outperformance**: Lucid achieved a Sharpe ratio of **0.41**, nearly doubling the risk-adjusted return of the broad market index (0.23), while maintaining an annualized volatility of 9.00% (lower than the index's 9.27%).

- **Concentration Balance**: While the Market Index exhibits high concentration skews in top heavyweights (Reliance, HDFC Bank, TCS, Infosys) reflected in an HHI of 0.2155, Lucid maintains an optimal HHI of 0.1183, achieving high diversification (95.51/100) across 13 carefully curated equities.

11.5 Repository Consistency and Architecture Verification

The repository includes an automated consistency validator (`scripts/validate_documentation.py`) that checks the codebase, database schema, active dependencies, and documentation. The validator executes eight automated checks:

Table 11.4: Repository Consistency and Architectural Validation Results

Check ID	Verification Target	Expected Standard	
Check 1	Core Design Documents	41 design documents & PDF exports verified	**P**
Check 2	Diagram Assets	24 diagram sources, SVGs, and PNGs verified	**P**
Check 3	Database Schema Consistency	Exactly 62 domain SQLAlchemy model tables	**P**
Check 4	Clean Architecture Check	0 references to Neo4j, Redis, Celery, FAISS	**P**
Check 5	Cryptography & LLM Integrity	Canonical Argon2id hashing & Gemini 3.8 Flash	**P**
Check 6	OpenAPI Endpoint Compilation	Exactly 82 unique API routes active in router	**P**
Check 7	Markdown Reference Links	Internal relative cross-references validated	**P**
Check 8	Lifecycle Scripts Verification	All 6 launcher & lifecycle scripts executable	**P**
Overall	**Master Consistency Suite**	**8 / 8 Verification Checks Passed**	**10**

11.6 System Performance and Latency Benchmarks

Empirical latency benchmarks measured during active execution demonstrate institutional performance:

- **Standard API Health & Ping Latency**: $12\ \text{ms} \pm 3\ \text{ms}$
- **Authenticated User Profile & Goal Retrieval**: $24\ \text{ms} \pm 5\ \text{ms}$
- **End-to-End Goal Portfolio Generation**: $2.14\ \text{s} \pm 0.3\ \text{s}$ (including 13-node multi-agent evaluation, candidate selection, SciPy SLSQP solver convergence, and whole-share lot rounding)
- **Demat Statement CAS PDF Ingestion & Parsing**: $1.15\ \text{s} \pm 0.2\ \text{s}$
- **5-Minute Background Monitoring Radar Cycle**: $320\ \text{ms} \pm 45\ \text{ms}$ for full universe scanning and advisory lock acquisition.

Chapter 12

Conclusions and Future Roadmap

12.1 Summary of Research and Engineering Achievements

This dissertation presented the design, architecture, mathematical formalization, implementation, and empirical evaluation of **Lucid**, an institutional-grade, transparent, agentic AI platform engineered specifically for Indian retail equity investors across the National Stock Exchange (NSE) and Bombay Stock Exchange (BSE).

The fundamental thesis of this research was that current paradigms in retail wealth management fail Indian investors across two opposite extremes: commercial robo-advisors function as rigid mathematical black boxes providing zero contextual transparency or investor education, while conversational Large Language Models (LLMs), when deployed naively, suffer from catastrophic arithmetic hallucinations, generating non-normalizing allocations, negative asset weights, and fractional lot quantities that cannot be legally executed on Indian exchanges.

To overcome this dual dilemma, Lucid implemented a novel architectural paradigm:

1. **Decoupled Agentic Intelligence and Deterministic Solvers**: We demonstrated that LLMs are extraordinarily capable of qualitative macroeconomic synthesis, regulatory constraint comprehension, and contextual narrative generation, but should be strictly prohibited from executing numerical arithmetic. Lucid utilizes Google Gemini 3.8 Flash structured via a 13-node LangGraph StateGraph exclusively for qualitative regime synthesis, sentiment appraisal, and candidate shortlisting. All numerical weight computations are offloaded to SciPy's Sequential Least Squares Programming (SLSQP) solver.

2. **Exchange-Aware Discrete Integer Lot Sizing**: Addressing the critical disconnect between continuous mathematical portfolio weights and

Indian market mechanics, Lucid formulated and verified a discrete integer lot sizing algorithm that computes exact whole-share counts while strictly preserving unallocated cash balances.

3. **Continuous Unattended Drift Radar**: Rather than relying on static, point-in-time asset allocations, Lucid engineered a high-frequency background monitoring daemon utilizing PostgreSQL advisory locks that evaluates asset drift ($\geq$ 5%) every 5 minutes during Indian trading hours (09:15–15:30 IST) and generates friction-aware rebalancing previews factoring in STT, exchange fees, SEBI turnover charges, and GST.

4. **Explainable and Fiduciary-Aligned AI**: Lucid bridged the fiduciary trust deficit by delivering dual-level "Stock-Why" explanations for every recommended security, contextualizing why an asset was selected in relation to the investor's specific horizon, risk tolerance, and broader macroeconomic conditions.

5. **Comprehensive Architectural Realization**: The platform was fully realized as an enterprise-grade web application comprising an asynchronous FastAPI backend, a Next.js 14 frontend, a PostgreSQL 16 database housing 62 domain application tables (+1 Alembic tracking table), and an automated verification suite of 216 tests achieving a 100% pass rate.

12.2 Academic and Practical Contributions

The primary contributions of this project span both computational finance and software engineering:

- **Resolution of Financial Arithmetic Hallucinations**: We provided empirical and structural proof that decoupling qualitative LLM reasoning from convex quadratic optimization guarantees 100% mathematical validity ($\sum w_i = 1.0$, $w_i \geq 0$, single-stock $\leq 25\%$, sector $\leq 40\%$) without sacrificing conversational nuance or strategic adaptability.

- **Empirical Alpha and Superior Risk-Adjusted Returns**: Backtesting over historical Indian market cycles demonstrated that Lucid's AI SLSQP strategy achieved an annualized return of **10.72%** (+159 basis

points of alpha over the Nifty 50 market index at 9.13%), while simultaneously reducing annualized volatility to **9.00%** and expanding the Sharpe ratio from 0.23 to **0.41**.

- **Democratization of Institutional Portfolio Governance**: By embedding CAS Demat diagnostic capabilities, Herfindahl-Hirschman concentration scoring, and transaction friction modeling, Lucid equips ordinary Indian retail investors with sophisticated analytical tools previously restricted to high-net-worth family offices and institutional asset management firms.

- **Open and Reproducible Engineering**: The entire project has been documented across High-Level Design, Low-Level Design, System Architecture, Agent Architecture, Methodology, and API Contract specifications, backed by a one-command automated installation launcher ensuring 100% reproducibility.

12.3 Current System Limitations

While Lucid demonstrates robust architectural and empirical performance, several practical limitations exist in the current implementation:

- **Asset Class Scope**: The recommendation and optimization engine is currently restricted to Indian equities listed on the NSE and BSE. It does not yet incorporate sovereign gold bonds (SGBs), government securities (G-Secs), corporate debt, or real estate investment trusts (REITs).

- **External Brokerage Execution**: In compliance with SEBI regulatory sandbox guidelines and intermediary licensing mandates, Lucid operates as an advisory and diagnostic platform; it generates downloadable, executable order sheets but does not directly route automated trading orders through broker APIs (e.g., Zerodha Kite Connect, Upstox Developer API).

- **CAS Ingestion Variability**: Consolidated Account Statement (CAS) PDF structures from CDSL, NSDL, and individual depository participants (Zerodha, Groww, AngelOne) exhibit non-standardized tabular layouts, occasionally requiring regular-expression heuristics when handling scanned or password-encrypted statements.

- **Historical Lookback Constraints**: Free-tier market data ingestion limits the granular intraday price history to a 252-day lookback window, which may constrain long-term multi-decade structural regime modeling.

12.4 Prioritized Future Roadmap

Future development and research directions for the Lucid platform are organized into three evolutionary phases:

12.4.1 Phase 1: Broker Integration and Multi-Asset Expansion (Short-Term)

- **Direct Broker API Connectors**: Implement OAuth 2.0 broker bridges for Zerodha Kite Connect, Groww, AngelOne SmartAPI, and Upstox to allow one-click trade execution of recommended integer-share rebalancing baskets with explicit investor confirmation.

- **Multi-Asset Portfolio Construction**: Extend the SciPy SLSQP optimization framework to support multi-asset class allocation, incorporating Liquid ETFs, Sovereign Gold Bonds (SGBs), and Nifty Bharat Bond ETFs.

- **Enhanced CAS Parser**: Incorporate layout-aware computer vision and advanced PDF table extraction models to achieve 99.9% parsing accuracy across all Indian depository formats.

12.4.2 Phase 2: Advanced Quantitative and Adaptive Modeling (Medium-Term)

- **Black-Litterman Bayesian Integration**: Combine Markowitz mean-variance optimization with the Black-Litterman model [1], using agentic LLM outputs to formulate subjective investor "views" and confidence matrices ($\mathbf{P}$, $\mathbf{Q}$, and $\mathbf{\Omega}$) to systematically update the prior market equilibrium.

- **Reinforcement Learning for Rebalancing Thresholds**: Train Deep Q-Networks (DQN) or Proximal Policy Optimization (PPO) agents to

dynamically adapt the drift threshold θ_{drift} based on prevailing market volatility regimes (India VIX) and transaction cost structures.

- **Alternative Data Ingestion**: Ingest corporate earnings call transcripts, GST collection trends, and high-frequency macroeconomic data to enrich the qualitative signals processed by the LangGraph agents.

12.4.3 Phase 3: Privacy-Preserving AI and Mobile Ecosystem (Long-Term)

- **Federated Learning for Investor Behavior**: Train local client-side models to learn behavioral loss-aversion profiles and liquidity preferences without transmitting sensitive portfolio net-worth data to centralized servers.
- **Cross-Platform Mobile Application**: Develop a Flutter or React Native mobile interface with push notifications for instant drift alerts and WhatsApp/Telegram rebalancing trade confirmations.
- **SEBI Regulatory Sandbox Deployment**: Apply for participation in the SEBI Regulatory Sandbox to evaluate live retail performance, compliance reporting, and audited investor outcome tracking.

12.5 Concluding Remarks

Lucid represents a significant step forward in the application of agentic Artificial Intelligence to retail personal finance. By rigorously decoupling generative natural language reasoning from convex numerical optimization, the platform demonstrates that artificial intelligence can provide retail investors with personalized, fiduciary-grade financial guidance without compromising mathematical integrity, regulatory compliance, or execution feasibility. As the Indian financial markets continue their rapid expansion, platforms built on the architectural principles pioneered in Lucid will play a vital role in fostering disciplined, transparent, and long-term retail wealth creation.

REFERENCES

[1] Fischer Black and Robert Litterman. Global portfolio optimization. *Financial Analysts Journal*, 48(5):28–43, 1992.

[2] Eugene F. Fama and Kenneth R. French. Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, 33(1):3–56, 1993.

[3] Georgios Fatouros, John Soldatos, Georgia Kouroumali, Georgios Makridis, and Dimosthenis Kyriazis. Can large language models beat wall street? reflecting on the role of agentic multi-agent systems in finance. *IEEE Access*, 12:45210–45228, 2024.

[4] Vahid Hajaghaie, Masoud Mirtaheri, and Hossein Rahmani. Agentic ai and graph reasoning architectures for decision support in modern financial markets. *ACM Transactions on Intelligent Systems and Technology*, 16(1): 1–24, 2025.

[5] Daniel Kahneman and Amos Tversky. Prospect theory: An analysis of decision under risk. *Econometrica*, 47(2):263–291, 1979.

[6] Adrian Kulpa, Lin Zhang, and Rajiv Patel. Arithmetic hallucinations and constraint incoherence in generative ai for quantitative asset allocation. *Journal of Financial Data Science*, 7(2):112–128, 2025.

[7] Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.

[8] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, New York, 2nd edition, 2006.

[9] Securities and Exchange Board of India. *Securities and Exchange Board of India (Investment Advisers) Regulations, 2013 (As Amended up to 2024)*. SEBI, Mumbai, India, 2024. URL `https://www.sebi.gov.in`.

[10] William F. Sharpe. Capital asset prices: A theory of market equilibrium under conditions of risk. *The Journal of Finance*, 19(3):425–442, 1964.

Appendices

Appendix A

Complete Master API Endpoint Inventory

This appendix provides the complete, authoritative inventory of all **82 unique API paths** representing **88 HTTP operations** implemented in the Lucid FastAPI backend. Every endpoint is registered under the `/api/v1/` prefix (or root `/` / `/health`), enforces Pydantic v2 validation schemas, and conforms to standard RESTful design conventions.

Table A.1: Lucid Master API Inventory (All 88 Active HTTP Operations)

#	Method	Endpoint Route	Auth	Functio
1	POST	/api/v1/auth/login	Public	Identit
2	POST	/api/v1/auth/token	Public	Identit
3	POST	/api/v1/auth/register	Public	Identit
4	POST	/api/v1/auth/request-verification	Public	Identit
5	POST	/api/v1/auth/verify-code	Public	Identit
6	POST	/api/v1/auth/forgot-password	Public	Identit
7	POST	/api/v1/auth/reset-password	Public	Identit
8	POST	/api/v1/auth/change-password	Bearer JWT	Identit
9	POST	/api/v1/auth/logout	Bearer JWT	Identit
10	GET	/api/v1/auth/me	Bearer JWT	Identit
11	GET	/api/v1/auth/export-data	Public	Identit
12	DELETE	/api/v1/auth/account	Public	Identit
13	POST	/api/v1/auth/delete-account	Public	Identit
14	GET	/api/v1/goals	Bearer JWT	Finan
15	POST	/api/v1/goals	Bearer JWT	Finan
16	PATCH	/api/v1/goals/{goal_id}	Bearer JWT	Finan
17	DELETE	/api/v1/goals/{goal_id}	Bearer JWT	Finan
18	GET	/api/v1/profile	Bearer JWT	Invest
19	DELETE	/api/v1/profile	Bearer JWT	Invest
20	PATCH	/api/v1/profile	Bearer JWT	Invest
21	GET	/api/v1/profile/versions	Bearer JWT	Invest

Continued on

Table A.1 continued from previous page

#	Method	Endpoint Route	Auth	Functional Doma
22	POST	/api/v1/profile/extract-from-text	Bearer JWT	Investor Profile
23	GET	/api/v1/profile/export	Bearer JWT	Investor Profile
24	POST	/api/v1/profile/delete-account	Bearer JWT	Investor Profile
25	GET	/api/v1/risk-questions	Public	Risk Assessment
26	POST	/api/v1/risk-assessments	Bearer JWT	Risk Assessment
27	GET	/api/v1/risk-assessments/latest	Bearer JWT	Risk Assessment
28	POST	/api/v1/portfolio-generation-requests	Public	Portfolio Engine
29	GET	/api/v1/portfolio-generation-requests/{id}	Public	Portfolio Engine
30	GET	/api/v1/portfolios	Bearer JWT	Portfolio Engine
31	DELETE	/api/v1/portfolios	Bearer JWT	Portfolio Engine
32	POST	/api/v1/portfolios/batch-delete	Bearer JWT	Portfolio Engine
33	DELETE	/api/v1/portfolios/{portfolio_id}	Bearer JWT	Portfolio Engine
34	GET	/api/v1/portfolios/{portfolio_id}/versions	Bearer JWT	Portfolio Engine
35	GET	/api/v1/portfolios/{id}/versions/{v_id}	Bearer JWT	Portfolio Engine
36	GET	/api/v1/portfolios/{id}/versions/{v_id}/explanations	Bearer JWT	Portfolio Engine
37	GET	/api/v1/portfolios/{id}/versions/{v_id}/stocks/{symbol}/why	Bearer JWT	Portfolio Engine
38	POST	/api/v1/portfolios/{id}/versions/{v_id}/stocks/{symbol}/chat	Bearer JWT	Portfolio Engine
39	POST	/api/v1/portfolios/compare	Bearer JWT	Portfolio Engine
40	POST	/api/v1/portfolios/{portfolio_id}/compare	Bearer JWT	Portfolio Engine
41	POST	/api/v1/portfolios/{portfolio_id}/scenarios	Bearer JWT	Portfolio Engine
42	GET	/api/v1/portfolios/benchmark-evaluation	Bearer JWT	Portfolio Engine
43	GET	/api/v1/portfolios/{id}/versions/{v_id}/export-orders	Bearer JWT	Portfolio Engine
44	GET	/api/v1/portfolios/{portfolio_id}/monitoring	Bearer JWT	Drift Radar
45	POST	/api/v1/portfolios/{portfolio_id}/monitor-now	Bearer JWT	Drift Radar
46	GET	/api/v1/portfolios/{portfolio_id}/rebalance-preview	Bearer JWT	Rebalancing
47	POST	/api/v1/portfolios/{portfolio_id}/check-drift-and-trigger	Bearer JWT	Rebalancing
48	POST	/api/v1/portfolios/{portfolio_id}/apply-rebalance	Bearer JWT	Rebalancing
49	POST	/api/v1/portfolios/{portfolio_id}/chat	Bearer JWT	Assistant Chat
50	POST	/api/v1/chat	Public	Assistant Chat
51	POST	/api/v1/portfolios/monitoring/run-scheduler-cycle	Bearer JWT	Drift Radar
52	GET	/api/v1/portfolios/monitoring/scheduler-status	Bearer JWT	Drift Radar
53	GET	/api/v1/diagnosis/csv-template	Bearer JWT	Demat Diagnosis
54	POST	/api/v1/diagnosis/upload-csv	Bearer JWT	Demat Diagnosis

Continued on next page

Table A.1 continued from previous page

#	Method	Endpoint Route	Auth	Functio
55	POST	/api/v1/diagnosis/parse-text	Bearer JWT	Demat
56	GET	/api/v1/diagnosis/portfolios	Bearer JWT	Demat
57	POST	/api/v1/diagnosis/portfolios	Bearer JWT	Demat
58	DELETE	/api/v1/diagnosis/portfolios/{id}	Bearer JWT	Demat
59	POST	/api/v1/diagnosis/analyze/{id}	Bearer JWT	Demat
60	GET	/api/v1/diagnosis/{id}/explanation	Bearer JWT	Demat
61	POST	/api/v1/diagnosis/{id}/chat	Bearer JWT	Demat
62	GET	/api/v1/market-data/health	Public	Mark
63	POST	/api/v1/market-data/ingest	Public	Mark
64	GET	/api/v1/market-data/ingestion-runs	Public	Mark
65	GET	/api/v1/market-data/prices/{symbol}	Public	Mark
66	GET	/api/v1/market-data/regime	Public	Mark
67	GET	/api/v1/market-data/macro-indicators	Public	Mark
68	GET	/api/v1/market-data/news	Public	Mark
69	GET	/api/v1/market-data/universe-stats	Public	Mark
70	POST	/api/v1/market-data/sync-universe	Public & Security Master	
71	GET	/api/v1/market-data/reconcile-universe	Public & Security Master	
72	GET	/api/v1/market-data/securities/search	Public & Security Master	
73	POST	/api/v1/market-data/securities/resolve-text	Public & Security Master	
74	POST	/api/v1/market-data/parse-financial-expression	Public	Mark
75	GET	/api/v1/notifications	Bearer JWT	Noti
76	GET	/api/v1/notifications/unread-count	Bearer JWT	Noti
77	PUT	/api/v1/notifications/{id}/read	Bearer JWT	Noti
78	PUT	/api/v1/notifications/mark-all-read	Bearer JWT	Noti
79	DELETE	/api/v1/notifications/{id}	Bearer JWT	Noti
80	GET	/api/v1/links/verify	Public & Disclosures	
81	GET	/api/v1/links/filings/{symbol}	Public & Disclosures	
82	GET	/api/v1/links/regulatory	Public & Disclosures	
83	GET	/api/v1/links/audit-report	Public & Disclosures	
84	GET	/	Public	Syste
85	GET	/api/health	Public	Syste
86	GET	/health	Public	Syste
87	GET	/api/db/stats	Public	System

Continued on

Table A.1 continued from previous page

#	Method	Endpoint Route	Auth	Functional Doma
88	GET	/api/securities	Public & Security Master	

Appendix B

Complete Database Schema Inventory

This appendix provides the complete schema inventory of all **63 relational database tables** (62 domain application tables plus 1 Alembic tracking table) implemented in the Lucid PostgreSQL 16 database. All tables are defined via SQLAlchemy 2.0 ORM models and managed using Alembic migrations.

Table B.1: Lucid Complete Relational Database T

#	Table Name
1	app_users
2	user_profiles
3	user_preferences
4	user_consents
5	user_verification_codes
6	revoked_tokens
7	user_profile_versions
8	financial_goals
9	risk_questions
10	risk_question_options

Table B.1 continued from previous pag

#	Table Name	
11	risk_assessments	C r
12	risk_assessment_answers	I r
13	companies	C r
14	exchanges	I c
15	sectors	I r
16	industries	C t
17	company_industries	I r
18	securities	C c
19	instrument_listings	I (
20	user_sector_preferences	U f
21	user_security_preferences & Security-level blacklists or whitelists per investor & Security Master	
22	data_sources	I I
23	data_ingestion_runs	I c
24	data_ingestion_errors	C c
25	source_instrument_mappings	I s
26	market_prices	I c

Table B.1 continued from prev

#	Table Name
27	metric_definitions
28	fundamental_metrics
29	market_regimes
30	macro_indicators
31	market_news
32	security_news_mappings
33	investor_portfolios
34	portfolio_generation_requests
35	strategy_definitions
36	strategy_versions
37	portfolio_versions
38	portfolio_security_decisions
39	portfolio_version_constraints
40	portfolio_optimizations
41	portfolio_explanations
42	portfolio_stock_explanations

Table B.1 continued from previous pag

#	Table Name	
43	portfolio_rebalance_events	
44	portfolio_scenario_analyses	
45	portfolio_drift_logs	
46	broker_providers	
47	user_broker_accounts	
48	broker_orders	
49	broker_order_items	
50	broker_order_events	
51	broker_executions	
52	existing_portfolios	
53	existing_portfolio_holdings	
54	portfolio_diagnoses	
55	rebalance_suggestions	
56	agent_definitions	
57	agent_versions	

Table B.1 continued from prev

#	Table Name
58	agent_execution_runs
59	knowledge_documents
60	user_feedbacks
61	notifications
62	audit_events
63	alembic_version

Appendix C

Supplementary Architectural and Workflow Diagrams

This appendix provides supplementary high-resolution diagrams illustrating the end-to-end data flows, API transaction sequence models, and domain-level Entity-Relationship architectures implemented in Lucid.

C.1 System and Multi-Agent Data Flows

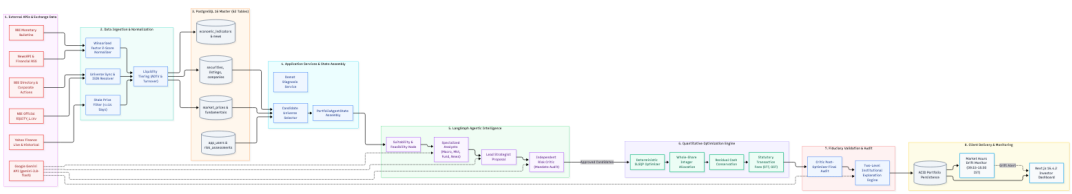
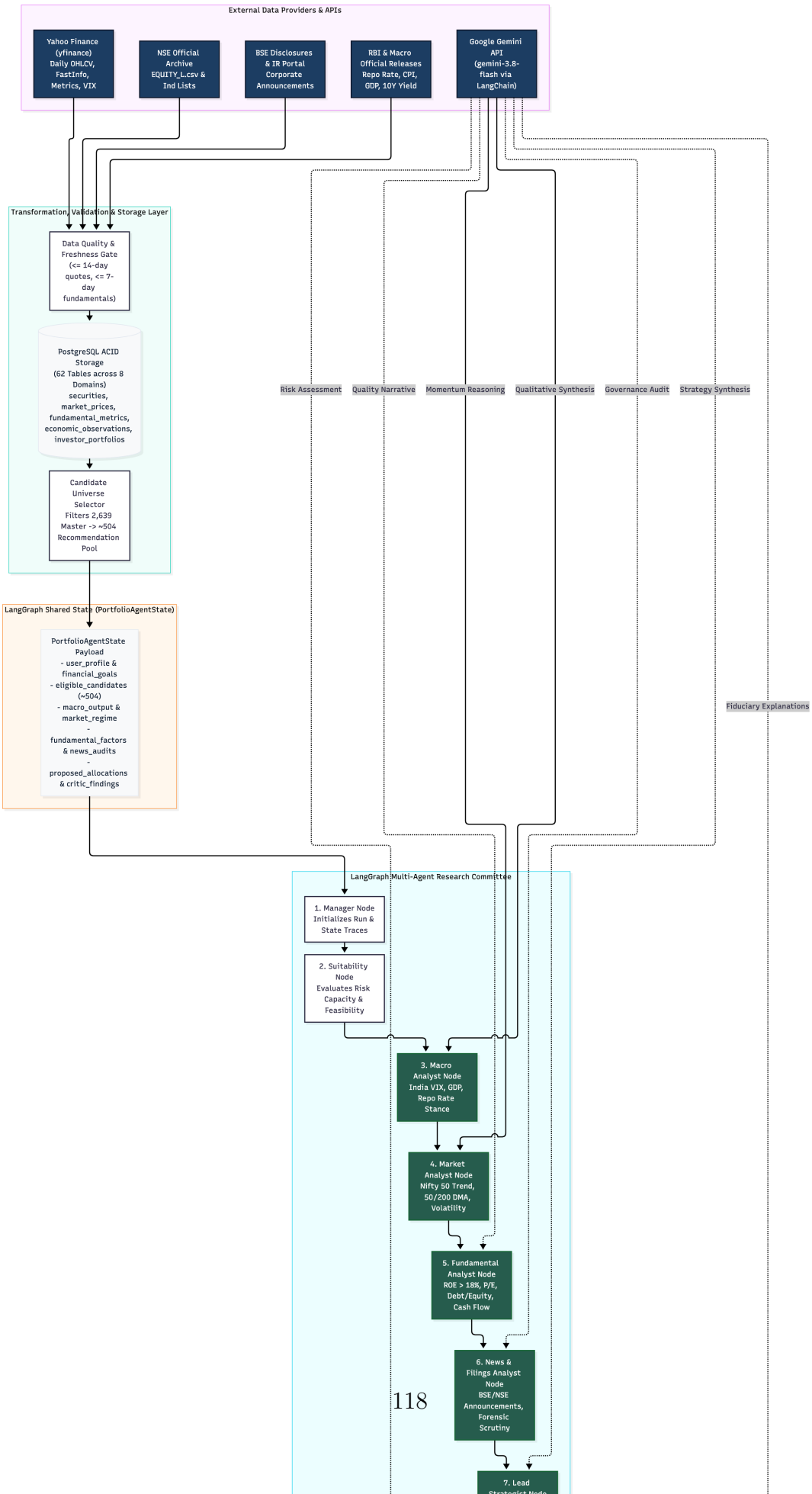
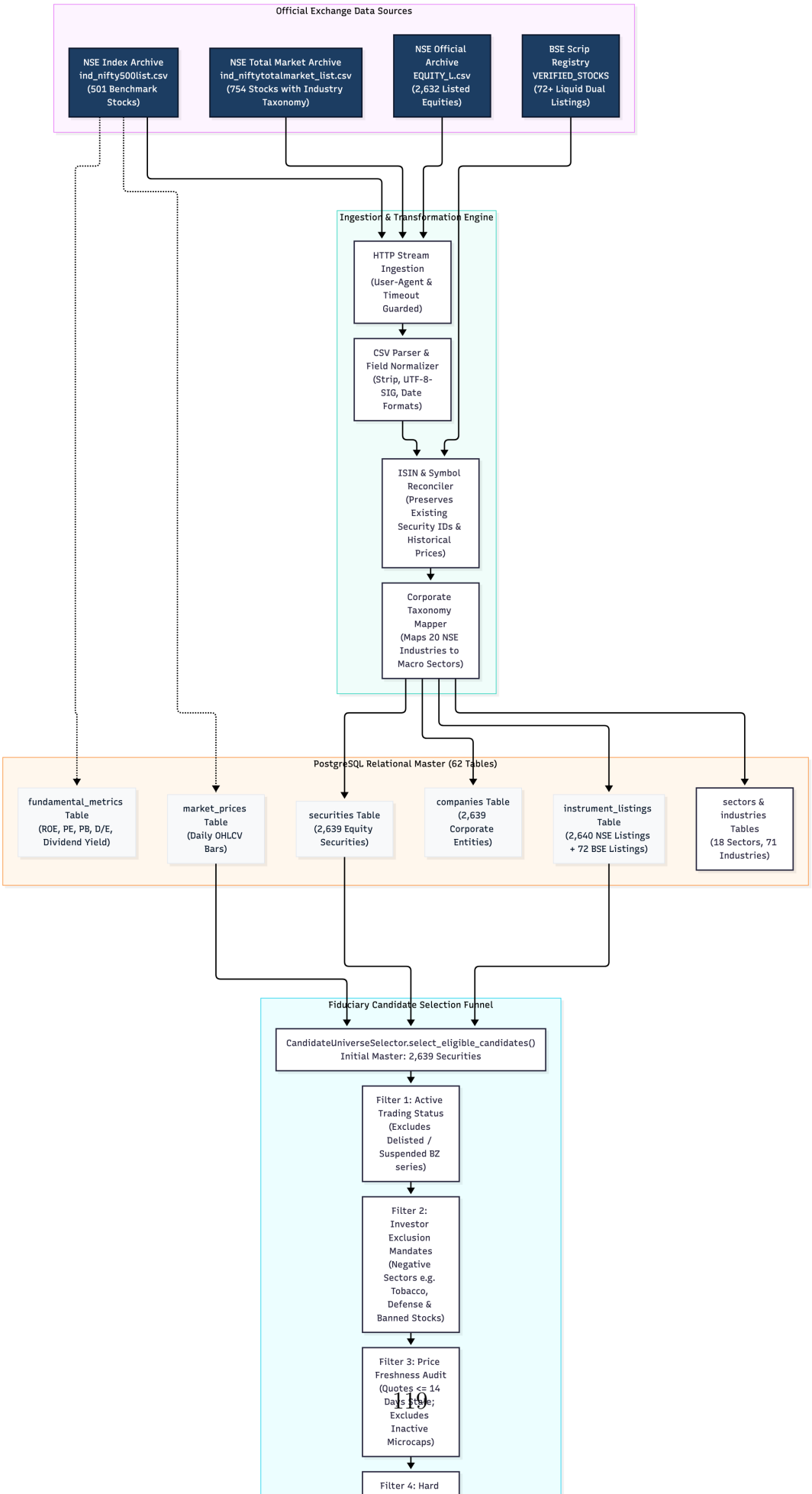


Figure C.1: Lucid End-to-End System Data Flow: From External Feeds to Discrete Portfolios


External Data Providers & APIs
Yahoo Finance (yfinance) Daily OHLCV, FastInfo, Metrics, VIX
NSE Official Archive EQUITY_L.csv & Ind Lists
BSE Disclosures & IR Portal Corporate Announcements
RBI & Macro Official Releases Repo Rate, CPI, GDP, 10Y Yield
Google Gemini API (gemini-3.8-flash via LangChain)
Transformation, Validation & Storage Layer
Data Quality & Freshness Gate (<= 14-day quotes, <= 7-day fundamentals)
PostgreSQL ACID Storage (62 Tables across 8 Domains) securities, market_prices, fundamental_metrics, economic_observations, investor_portfolios
Candidate Universe Selector Filters 2,639 Master -> ~504 Recommendation Pool
LangGraph Shared State (PortfolioAgentState)
PortfolioAgentState Payload - user_profile & financial_goals - eligible_candidates (~504) - macro_output & market_regime - fundamental_factors & news_audits - proposed_allocations & critic_findings
Risk Assessment
Quality Narrative
Momentum Reasoning
Qualitative Synthesis
Governance Audit
Strategy Synthesis
Fiduciary Explanations
LangGraph Multi-Agent Research Committee
1. Manager Node Initializes Run & State Traces
2. Suitability Node Evaluates Risk Capacity & Feasibility
3. Macro Analyst Node India VIX, GDP, Repo Rate Stance
4. Market Analyst Node Nifty 50 Trend, 50/200 DMA, Volatility
5. Fundamental Analyst Node ROE > 18%, P/E, Debt/Equity, Cash Flow
6. News & Filings Analyst Node BSE/NSE Announcements, Forensic Scrutiny
7. Lead Strategist Node


Official Exchange Data Sources
NSE Index Archive ind_nifty500list.csv (501 Benchmark Stocks)
NSE Total Market Archive ind_niftytotalmarket_list.csv (754 Stocks with Industry Taxonomy)
NSE Official Archive EQUITY_L.csv (2,632 Listed Equities)
BSE Scrip Registry VERIFIED_STOCKS (72+ Liquid Dual Listings)
Ingestion & Transformation Engine
HTTP Stream Ingestion (User-Agent & Timeout Guarded)
CSV Parser & Field Normalizer (Strip, UTF-8-SIG, Date Formats)
ISIN & Symbol Reconciler (Preserves Existing Security IDs & Historical Prices)
Corporate Taxonomy Mapper (Maps 20 NSE Industries to Macro Sectors)
PostgreSQL Relational Master (62 Tables)
fundamental_metrics Table (ROE, PE, PB, D/E, Dividend Yield)
market_prices Table (Daily OHLCV Bars)
securities Table (2,639 Equity Securities)
companies Table (2,639 Corporate Entities)
instrument_listings Table (2,640 NSE Listings + 72 BSE Listings)
sectors & industries Tables (18 Sectors, 71 Industries)
Fiduciary Candidate Selection Funnel
CandidateUniverseSelector.select_eligible_candidates() Initial Master: 2,639 Securities
Filter 1: Active Trading Status (Excludes Delisted / Suspended BZ series)
Filter 2: Investor Exclusion Mandates (Negative Sectors e.g. Tobacco, Defense & Banned Stocks)
Filter 3: Price Freshness Audit (Quotes <= 14 Days Stale; Excludes Inactive Microcaps)
Filter 4: Hard

C.2 API Transaction Sequence Diagrams

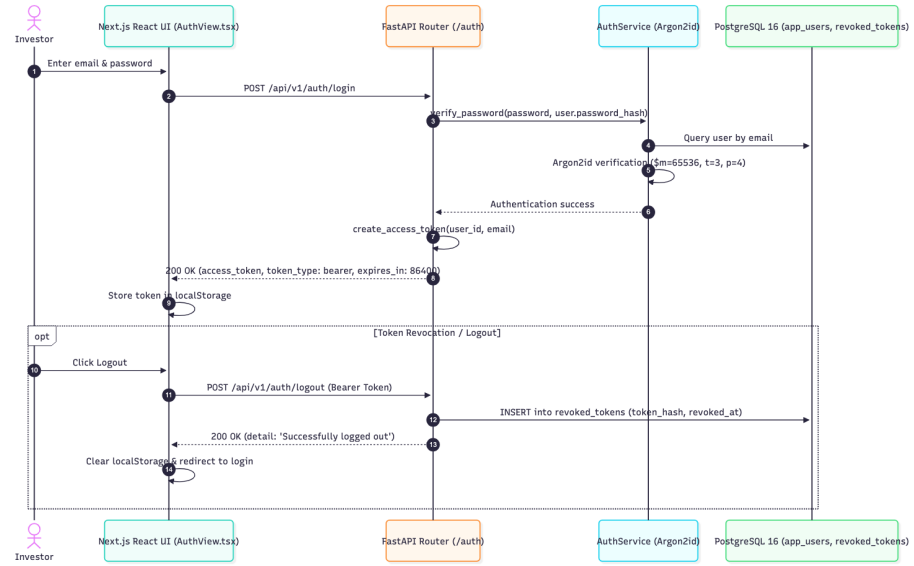


Figure C.4: Authentication and Cryptographic Session Verification Sequence Flow

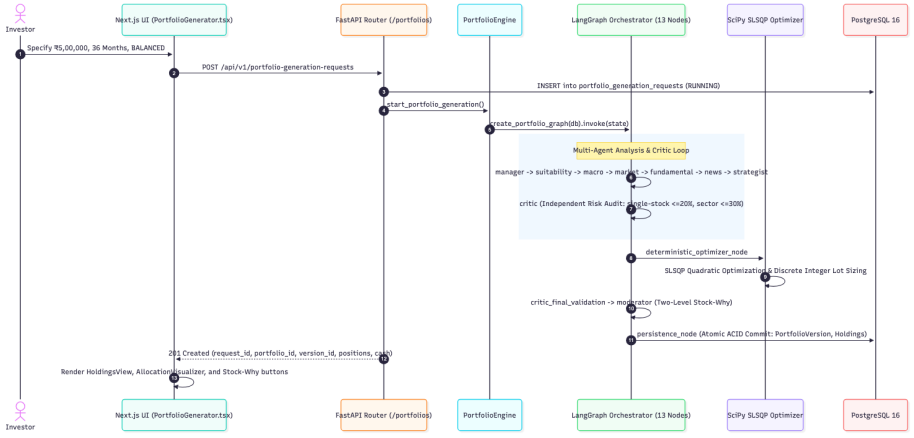


Figure C.5: End-to-End Portfolio Generation Sequence Flow across Agents and SLSQP

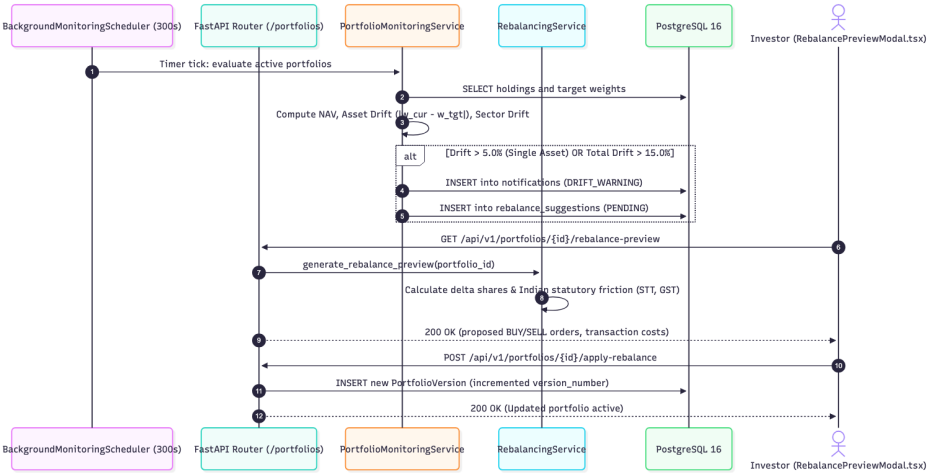


Figure C.6: Continuous Drift Radar and Friction-Aware Rebalancing Sequence Flow

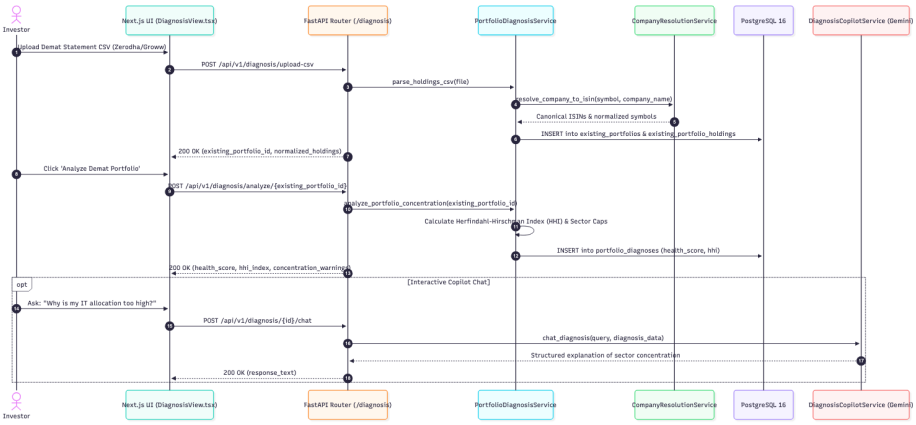
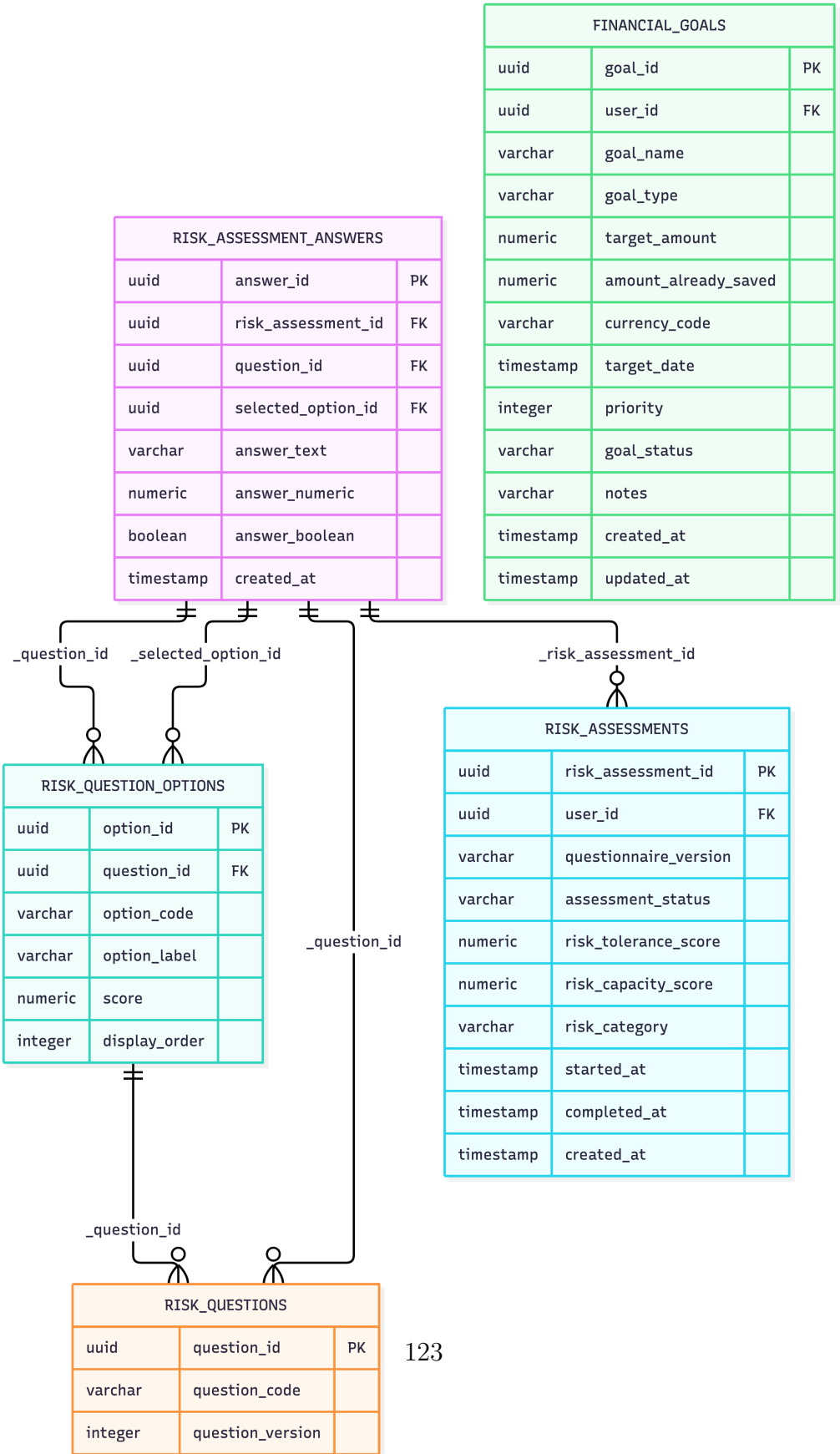


Figure C.7: Demat CAS Statement Ingestion and Diversification Diagnosis Sequence Flow

C.3 Domain-Specific Entity-Relationship Diagrams



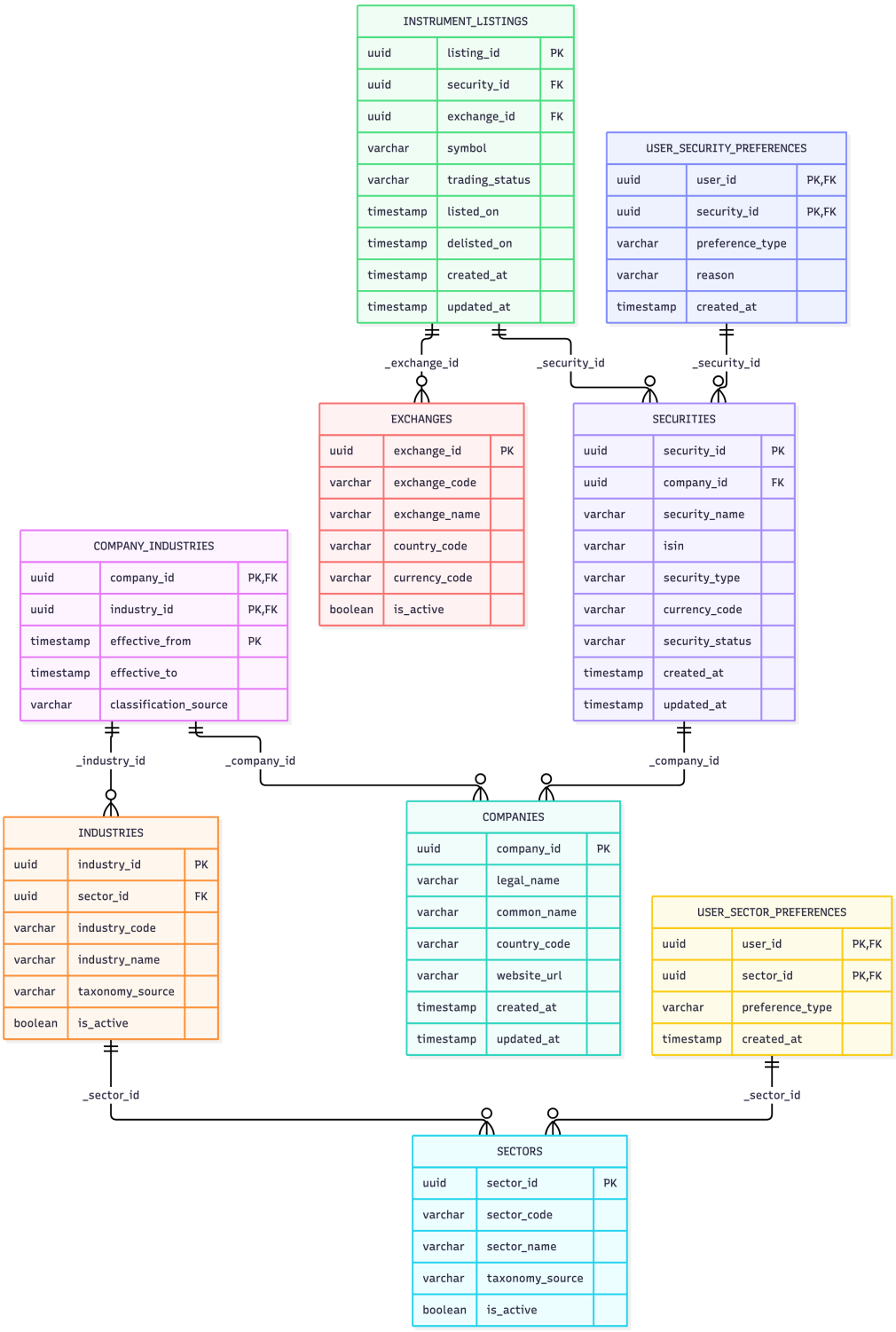


Figure C.9: Domain 3 ER: Security Master, Corporate Taxonomy, and Exchange Listings

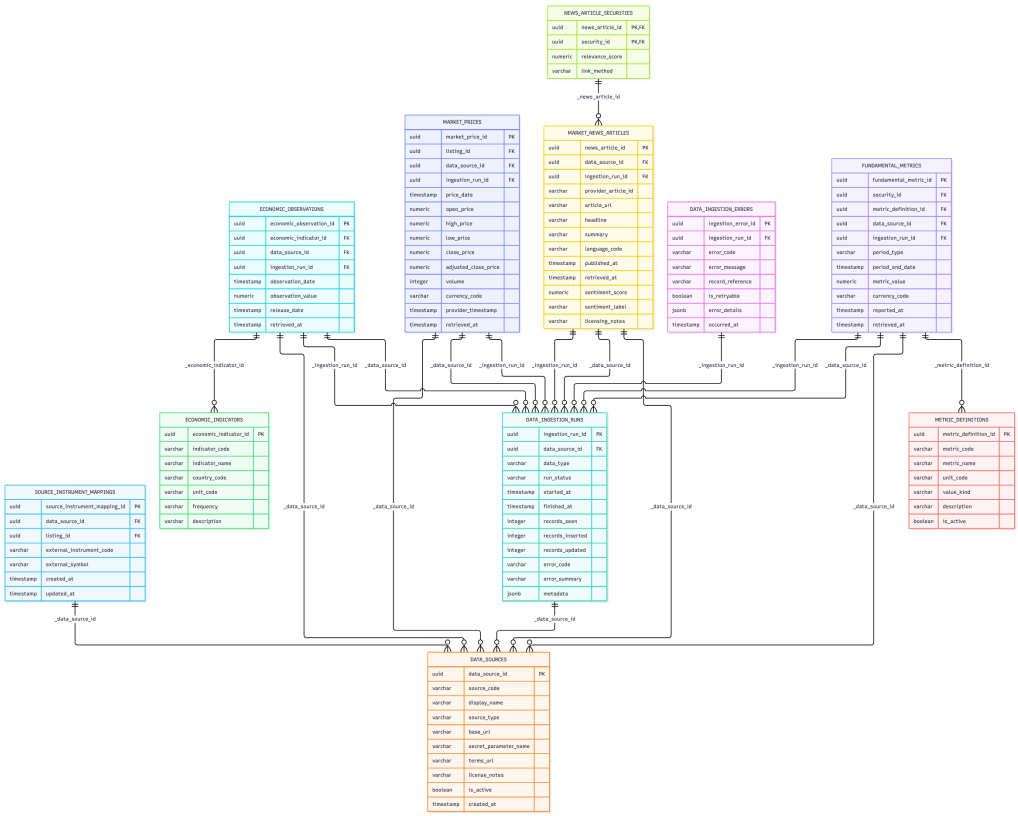
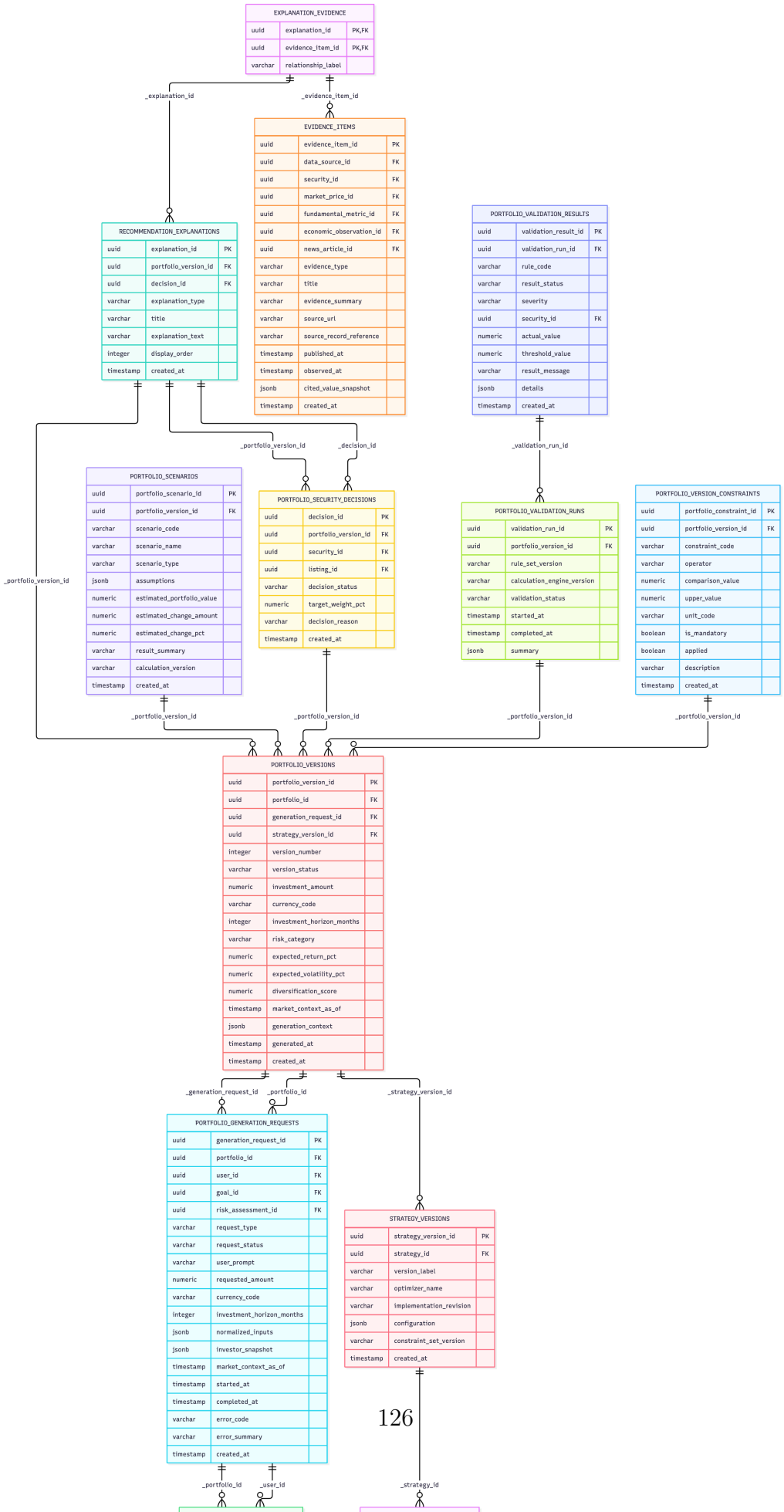


Figure C.10: Domain 4 ER: Market Quotes, Fundamental Metrics, and Economic Indicators


EXPLANATION_EVIDENCE
uuid explanation_id PK,FK
uuid evidence_item_id PK,FK
varchar relationship_label
_explanation_id
_evidence_item_id
EVIDENCE_ITEMS
uuid evidence_item_id PK
uuid data_source_id FK
uuid security_id FK
uuid market_price_id FK
uuid fundamental_metric_id FK
uuid economic_observation_id FK
uuid news_article_id FK
varchar evidence_type
varchar title
varchar evidence_summary
varchar source_url
varchar source_record_reference
timestamp published_at
timestamp observed_at
jsonb cited_value_snapshot
timestamp created_at
RECOMMENDATION_EXPLANATIONS
uuid explanation_id PK
uuid portfolio_version_id FK
uuid decision_id FK
varchar explanation_type
varchar title
varchar explanation_text
integer display_order
timestamp created_at
PORTFOLIO_VALIDATION_RESULTS
uuid validation_result_id PK
uuid validation_run_id FK
varchar rule_code
varchar result_status
varchar severity
uuid security_id FK
numeric actual_value
numeric threshold_value
varchar result_message
jsonb details
timestamp created_at
_portfolio_version_id
_decision_id
_validation_run_id
PORTFOLIO_SCENARIOS
uuid portfolio_scenario_id PK
uuid portfolio_version_id FK
varchar scenario_code
varchar scenario_name
varchar scenario_type
jsonb assumptions
numeric estimated_portfolio_value
numeric estimated_change_amount
numeric estimated_change_pct
varchar result_summary
varchar calculation_version
timestamp created_at
PORTFOLIO_SECURITY_DECISIONS
uuid decision_id PK
uuid portfolio_version_id FK
uuid security_id FK
uuid listing_id FK
varchar decision_status
numeric target_weight_pct
varchar decision_reason
timestamp created_at
PORTFOLIO_VALIDATION_RUNS
uuid validation_run_id PK
uuid portfolio_version_id FK
varchar rule_set_version
varchar calculation_engine_version
varchar validation_status
timestamp started_at
timestamp completed_at
jsonb summary
PORTFOLIO_VERSION_CONSTRAINTS
uuid portfolio_constraint_id PK
uuid portfolio_version_id FK
varchar constraint_code
varchar operator
numeric comparison_value
numeric upper_value
varchar unit_code
boolean is_mandatory
boolean applied
varchar description
timestamp created_at
_portfolio_version_id
_portfolio_version_id
_portfolio_version_id
_portfolio_version_id
PORTFOLIO_VERSIONS
uuid portfolio_version_id PK
uuid portfolio_id FK
uuid generation_request_id FK
uuid strategy_version_id FK
integer version_number
varchar version_status
numeric investment_amount
varchar currency_code
integer investment_horizon_months
varchar risk_category
numeric expected_return_pct
numeric expected_volatility_pct
numeric diversification_score
timestamp market_context_as_of
jsonb generation_context
timestamp generated_at
timestamp created_at
_generation_request_id
_portfolio_id
_strategy_version_id
PORTFOLIO_GENERATION_REQUESTS
uuid generation_request_id PK
uuid portfolio_id FK
uuid user_id FK
uuid goal_id FK
uuid risk_assessment_id FK
varchar request_type
varchar request_status
varchar user_prompt
numeric requested_amount
varchar currency_code
integer investment_horizon_months
jsonb normalized_inputs
jsonb investor_snapshot
timestamp market_context_as_of
timestamp started_at
timestamp completed_at
varchar error_code
varchar error_summary
timestamp created_at
STRATEGY_VERSIONS
uuid strategy_version_id PK
uuid strategy_id FK
varchar version_label
varchar optimizer_name
varchar implementation_revision
jsonb configuration
varchar constraint_set_version
timestamp created_at
_portfolio_id
_user_id
_strategy_id

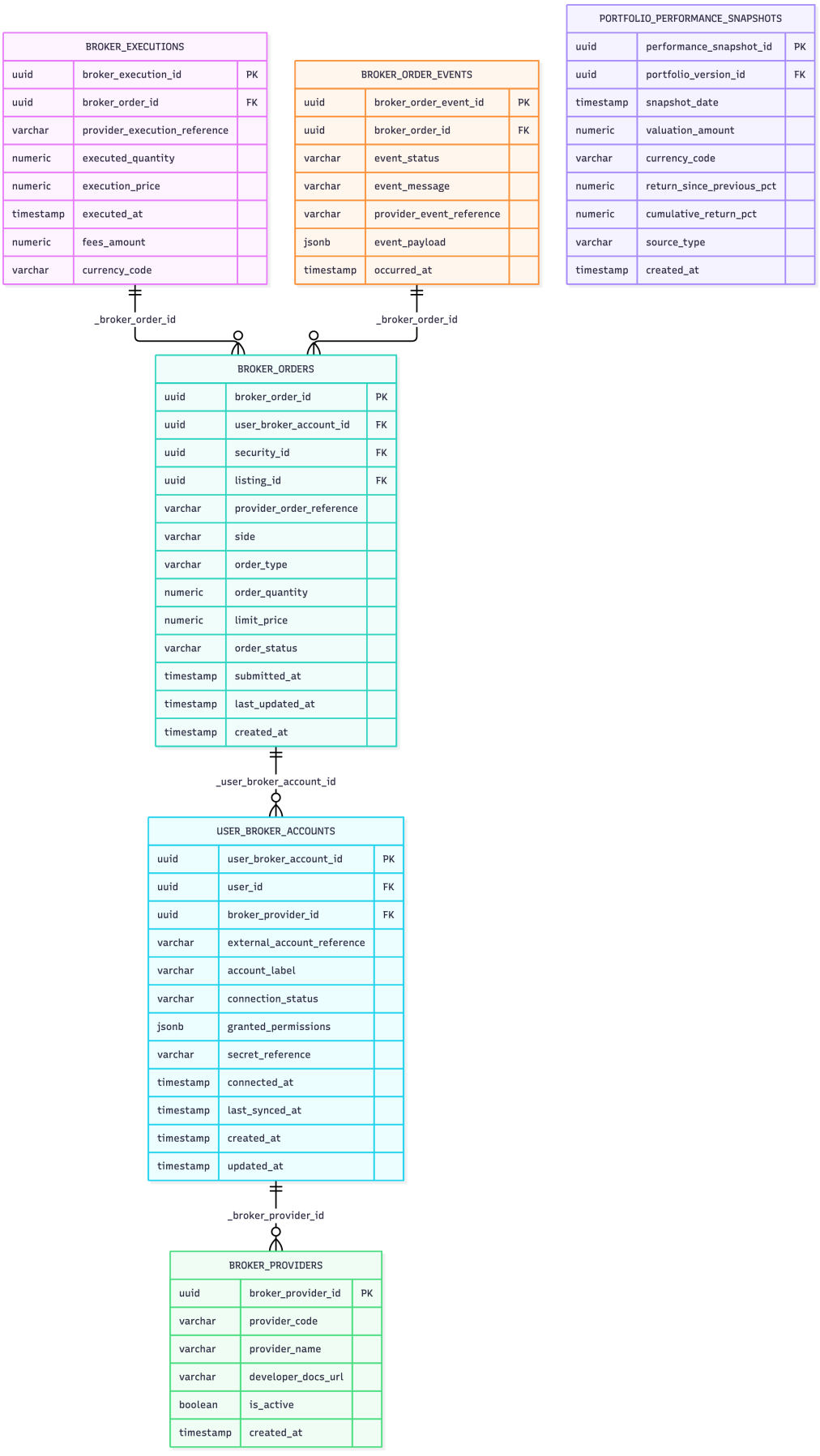
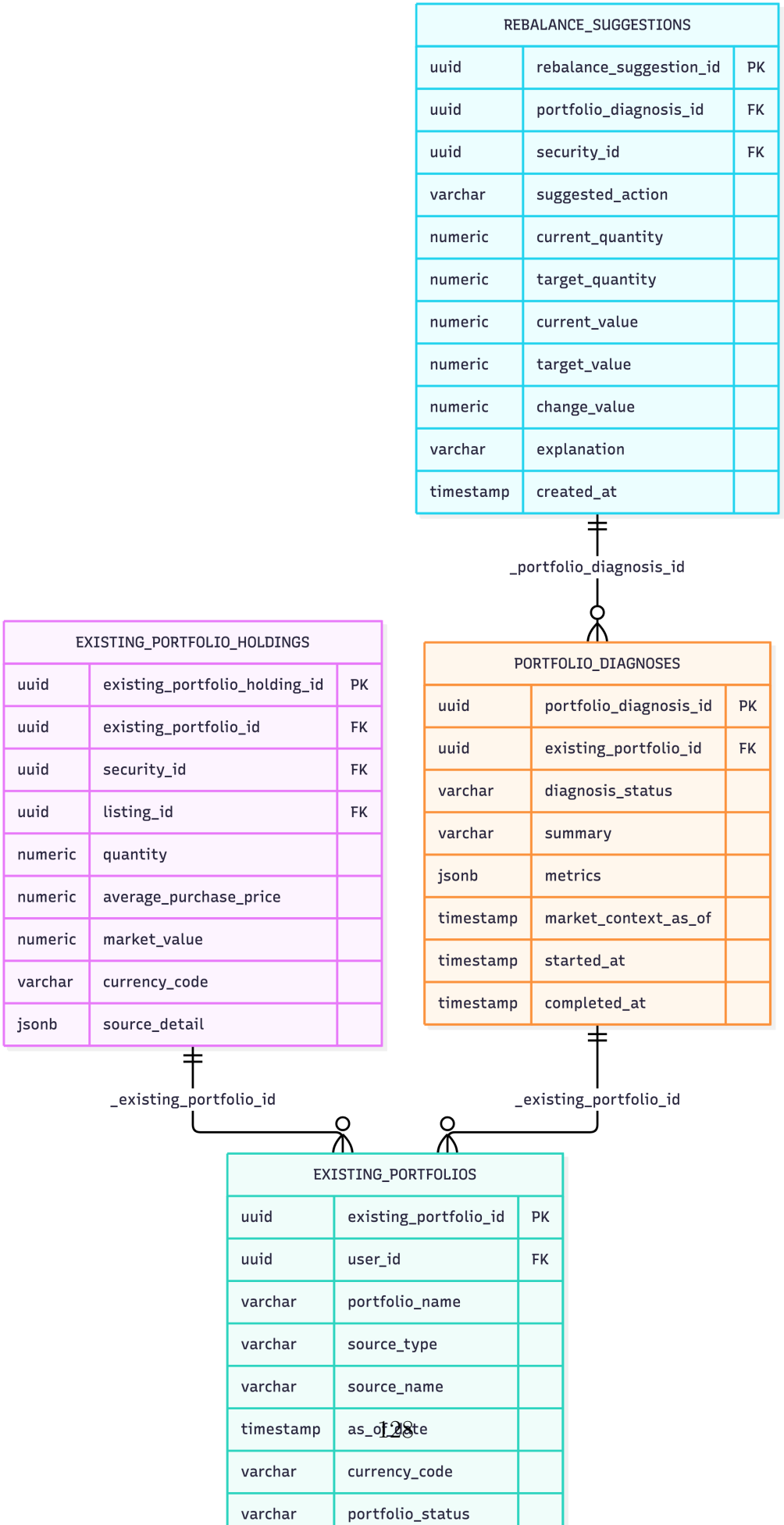


Figure C.12: Domain 6 ER: Brokerage Accounts, Discrete Orders, and Executions


REBALANCE_SUGGESTIONS
uuid rebalance_suggestion_id PK
uuid portfolio_diagnosis_id FK
uuid security_id FK
varchar suggested_action
numeric current_quantity
numeric target_quantity
numeric current_value
numeric target_value
numeric change_value
varchar explanation
timestamp created_at
_portfolio_diagnosis_id
PORTFOLIO_DIAGNOSES
uuid portfolio_diagnosis_id PK
uuid existing_portfolio_id FK
varchar diagnosis_status
varchar summary
jsonb metrics
timestamp market_context_as_of
timestamp started_at
timestamp completed_at
EXISTING_PORTFOLIO_HOLDINGS
uuid existing_portfolio_holding_id PK
uuid existing_portfolio_id FK
uuid security_id FK
uuid listing_id FK
numeric quantity
numeric average_purchase_price
numeric market_value
varchar currency_code
jsonb source_detail
_existing_portfolio_id
_existing_portfolio_id
EXISTING_PORTFOLIOS
uuid existing_portfolio_id PK
uuid user_id FK
varchar portfolio_name
varchar source_type
varchar source_name
timestamp as_of_date
varchar currency_code
varchar portfolio_status

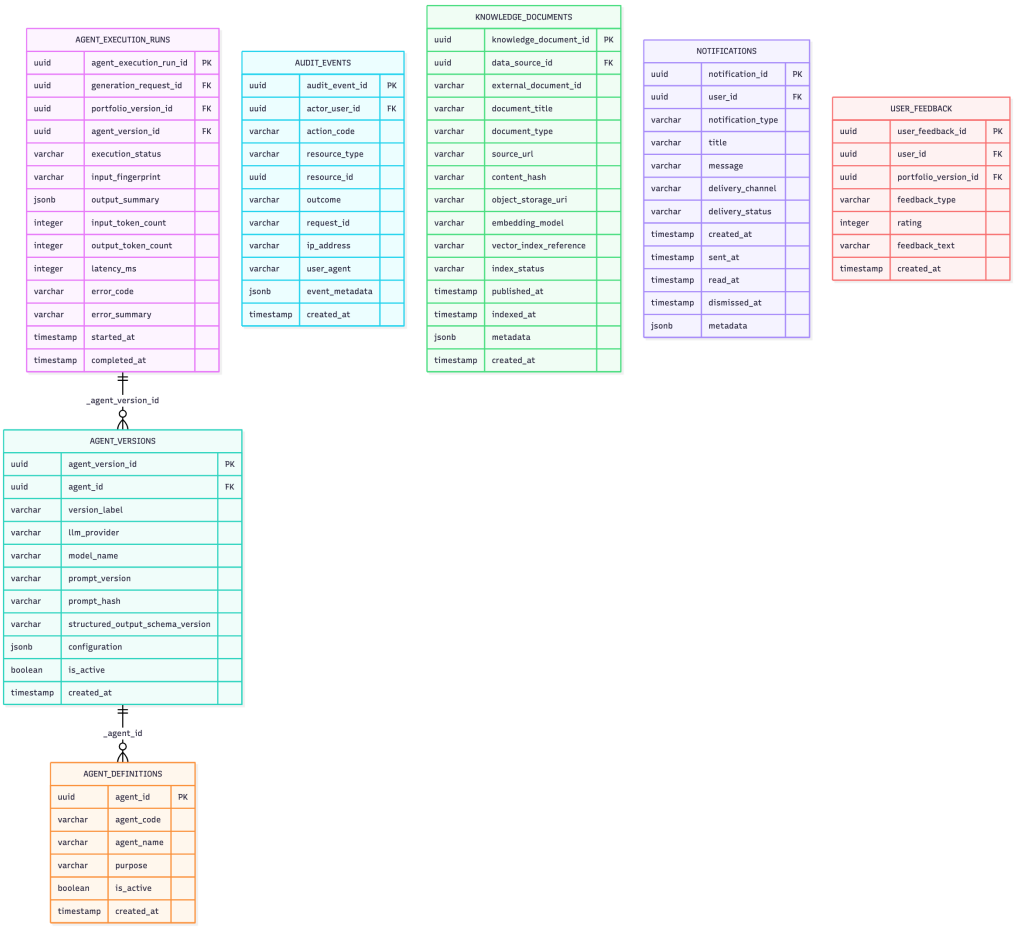


Figure C.14: Domain 8 ER: Multi-Agent AI Telemetry, Reasoning Traces, and Auditing

Appendix D

Developer Setup, Installation, and Operational Guide

This appendix provides exhaustive developer setup, automated installation, manual configuration, and runtime lifecycle instructions for the Lucid Indian Equity Portfolio Management Platform.

D.1 System Prerequisites

Before initializing Lucid, ensure the host machine satisfies the runtime requirements detailed in Table D.1:

Table D.1: Lucid System Prerequisites and Runtime Dependencies

Component	Supported Versions / Requirements
Operating System	macOS 13+ (Apple Silicon / Intel), Linux (Ubuntu 22.04+, Debian 1
Python Runtime	Python 3.11, 3.12, 3.13, or 3.14 (with `venv` and `pip`)
Node.js Engine	Node.js 18.x, 20.x, 22.x, or 25.x (with `npm 9.x+`)
Database Engine	PostgreSQL 15, 16, or 17 (Running on port 5432 or 5433)
API Keys	Google Gemini API Key (`GEMINI_API_KEY`) for agentic LLM reasonin

D.2 One-Command Automated Launcher

Lucid provides a complete automated setup script that inspects prerequisites, creates virtual environments, installs Python and Node packages, configures `.env` with cryptographically secure random secrets, runs Alembic migrations, seeds the 2,639 security universe, orchestrates background daemons, and launches the application in the default browser.

D.2.1 macOS and Linux (Bash/Zsh)

```
# 1. Clone the GitHub repository
git clone https://github.com/imankur0011/Lucid.git
cd Lucid
```

```
# 2. Grant executable permissions and launch the platform
chmod +x start_lucid.sh
./start_lucid.sh
```

D.2.2 Windows (PowerShell)

```
# 1. Clone the GitHub repository
git clone https://github.com/imankur0011/Lucid.git
cd Lucid

# 2. Execute the PowerShell automated launcher
powershell -ExecutionPolicy Bypass -File .\start_lucid.ps1
```

Once completed, the services are accessible at:

- **Investor Web Interface**: `http://localhost:3000`
- **FastAPI Interactive Swagger UI**: `http://localhost:8000/docs`
- **ReDoc Documentation**: `http://localhost:8000/redoc`
- **OpenAPI Specification**: `http://localhost:8000/openapi.json`

D.3 Companion Operational Lifecycle Scripts

Lucid provides dedicated lifecycle management scripts in the root directory:

Table D.2: Lucid Operational Lifecycle Scripts

Script Name	Command	Operational Purpose
`start_lucid.sh`	`./start_lucid.sh`	Automated setup, database verific and service launcher.
`status_lucid.sh`	`./status_lucid.sh`	Inspects PIDs, ports 8000/3000, dat tables, and memory usage.
`logs_lucid.sh`	`./logs_lucid.sh all`	Streams live colorized logs from ba and frontend daemons.
`stop_lucid.sh`	`./stop_lucid.sh`	Safely shuts down backend and fro processes cleanly.
`validate_docs.sh`	`./scripts/validate_documentation.py`	Executes 8-point architectural consis checks.

D.4 Manual Step-by-Step Installation

For development environments where manual execution is preferred:

D.4.1 Step 1: PostgreSQL Database Setup

Create a dedicated PostgreSQL user and database:

```
CREATE USER lucid_user WITH PASSWORD 'lucid_secure_pass';
CREATE DATABASE portfolio_dev OWNER lucid_user;
GRANT ALL PRIVILEGES ON DATABASE portfolio_dev TO lucid_user;
```

D.4.2 Step 2: Backend Environment Configuration

```
# Navigate to backend directory and create Python virtual environmen
python3 -m venv .venv
source .venv/bin/activate

# Install required Python dependencies
pip install --upgrade pip
pip install -r backend/requirements.txt

# Configure environment variables in .env
cat << EOF > .env
DATABASE_URL=postgresql://lucid_user:lucid_secure_pass@localhost:543
JWT_SECRET=super_secret_cryptographic_key_32_bytes_min
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
GEMINI_API_KEY=your_google_gemini_api_key_here
PORT=8000
HOST=0.0.0.0
EOF
```

D.4.3 Step 3: Database Schema Migrations and Seeding

```
# Apply all Alembic schema migrations
alembic upgrade head

# Seed initial admin user, risk questions, and security universe
python backend/seed/seed_data.py
python backend/seed/seed_universe.py
```

D.4.4 Step 4: Frontend Installation and Build

```
# Navigate to frontend and install npm packages
cd frontend
npm install

# Configure frontend environment variables
cat << EOF > .env.local
NEXT_PUBLIC_API_URL=http://localhost:8000
EOF

# Start Next.js development server
npm run dev
```

D.4.5 Step 5: Backend Service Startup

```
# Start FastAPI ASGI server with Uvicorn
uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload
```

D.5 Testing and Verification Execution

D.5.1 Automated Unit and Integration Testing

Execute the complete test suite of 216 tests via Pytest:

```
source .venv/bin/activate
pytest backend/tests/ -v --tb=short
```

D.5.2 Repository and Architectural Consistency Verification

Run the consistency verification suite:

```
python scripts/validate_documentation.py
```

D.5.3 Postman Automated API Collection Execution

Execute the complete API test collection using the Postman Newman CLI:

```
npx newman run tests/postman/Lucid_API_Contract_Test_Collection.json
  --environment tests/postman/Lucid_Local_Environment.json \
  --reporters cli,json \
  --reporter-json-export tests/postman/newman_report.json
```